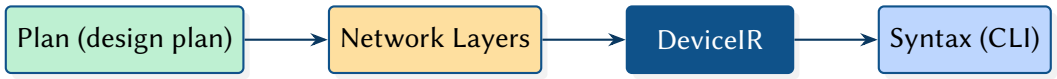
NetCfg: A Declarative Network Configuration Compiler

Formal Methods in Configuration Synthesis

Simon Knight, Ph.D.

March 2026 • Version 4.0.0

Last updated: March 17, 2026



Abstract. NetCfg is an architectural network configuration engine that takes a Blueprint (annotated topology)—the "source code" of the network—and compiles it into verified, target-specific device configurations. A companion design plan selects which compilation strategies to apply. The synthesis engine processes these inputs through a four-stage compilation pipeline—Intent Synthesis, DeviceIR Extraction, Target-Specific Lowering, and Syntax Serialisation—with cross-cutting assertion gates that validate the model at each stage boundary. This report documents the data model, the transformation DSLs, and the structural model-based lowering mechanism that decouples architectural logic from physical hardware constraints.

Contents

1 Introduction

Large-scale network configuration is a compilation problem.

Insight: *Manual CLI entry is the "assembly language" of networking; NetCfg provides the high-level language and compiler.*

This report documents NetCfg, an architectural network configuration engine that transforms a Blueprint (annotated topology) into verified, target-specific device configurations. By treating network design as a structured compilation process rather than a text-templating exercise, NetCfg enables operators to validate their architectural intent *before* a single line of configuration is pushed to the physical hardware.

1.1 The Problem: Protocols without Abstractions

Shenker [1] diagnosed the foundational crisis in modern networking: the industry adds protocols rather than abstractions. This produces a "bag of protocols" with no composable interfaces, where every new feature increases the O(N!) complexity of the interaction space.

Gill et al. [2] quantified the cost of this complexity: misconfiguration is the primary driver of customer-impacting incidents in cloud-scale data centres. As configurations grow to millions of lines, the reliance on ad-hoc Python scripts and Jinja2 templates has reached its limit. These tools lack a formal understanding of the underlying topology; they operate on strings, not models.

1.2 The Thesis: Topology as the Root Abstraction

Recently, large-scale topology systems [3] have introduced *multi-abstraction-layer topology* (MALT) modelling. The core insight is that almost all network management data either *derives* from topology, *constrains* how to use a topology, or *associates* resources with specific places in a topology.

NetCfg adopts this graph-centric worldview. It provides three core abstractions to decouple intent from mechanism:

1. **The Blueprint:** A formal graph model of the network's semantic intent.
2. **Layered Topology:** A multi-plane graph stack (Physical, OSPF, BGP) where state flows between layers via formal projections.
3. **The Design Plan:** A declarative specification of the compilation strategies applied to the graph.

Where OpenConfig [4] standardises the *schema* of per-device state, NetCfg standardises the *derivation* of that state from graph-wide topological relationships.

Design Rule 1: Hardware Independence

Architectural logic must lower to a vendor-neutral Intermediate Representation (DeviceIR) before any target-specific syntax is generated. This decouples policy intent from physical ASIC constraints.

1.3 The NetCfg Approach

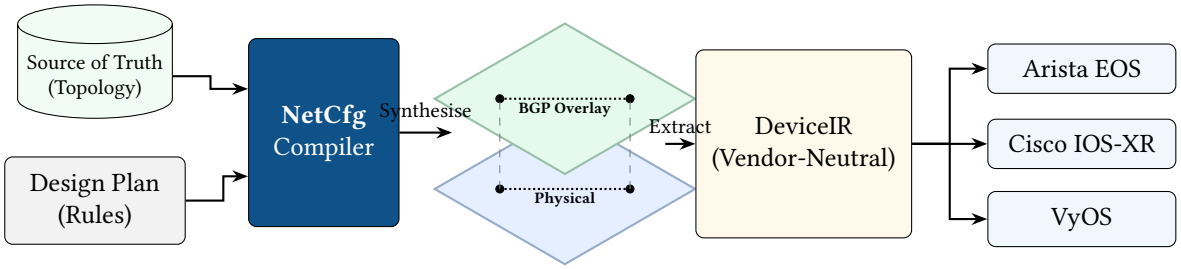


Figure 1. NetCfg acts as the compilation bridge between Source of Truth systems and physical network hardware. Powered by its declarative query language, it synthesises the physical inventory and design rules into logical **Network Layers** (e.g., BGP, OSPF). This allows operators to validate their architectural intent *before* extracting the Vendor-Neutral Intermediate Representation (DeviceIR).

NetCfg is a command-line tool and synthesis engine that compiles an annotated topology (the *Blueprint*) into per-device configuration files. Three input documents drive the pipeline (§4.1): the Blueprint declares *what* the network contains, a design plan declares *which* compilation strategies to apply, and an optional target backend declares *how* to render per-device syntax. The synthesis engine produces deterministic, diffable, auditable configuration artefacts for every device in the network.

NetCfg replaces the manual "glue" layer between source-of-truth systems (NetBox [5], Nautobot [6]), automation frameworks (Ansible [7], Nornir [8]), and network modelling tools (Batfish [9]). It treats network configuration as a *compilation problem*: topology intent is compiled through a typed pipeline, not interpolated through ad-hoc templates.

> **What NetCfg is *not***
>
> To understand the architecture, define its boundaries:
> - **It is not a Source of Truth.** It does not store inventory or manage IPAM databases. It *consumes* data from systems like NetBox and projects it into a layered graph.
> - **It is not a Deployment Engine.** It does not SSH into routers or push configuration via NETCONF/gNMI. It *compiles* the declarative artefacts that tools like Ansible then deploy.
> - **It is not a Data-Plane Verifier.** It does not simulate packet flow like Batfish. It proves *structural intent* (e.g., "Are these zones isolated in the model?") rather than simulating routing tables.

Where OpenConfig standardises the *schema* of device state, NetCfg standardises the *derivation* of that state from topology intent. Earlier systems provided a shared topology *representation*; NetCfg provides a topology *compiler* that transforms intent into per-device configuration.

The current system should be read as a production-oriented research prototype. The core compilation pipeline, assertion model, and multi-vendor rendering path are implemented; incremental compilation, runtime drift detection, and full mapping-document tooling remain explicit future work (§G).

Design philosophy

Three principles explain every design decision in this document:

1. **Lowering over templating.** The operator's intent is lowered through successive abstraction stages, mirroring the multi-stage lowering architecture of LLVM [10]. LLVM lowers high-level source through a language-independent IR (LLVM IR) to target-specific machine code; NetCfg lowers topology intent through a vendor-neutral DeviceIR to platform-specific CLI syntax. The analogy is precise: LLVM's frontend/IR/backend separation corresponds to NetCfg's design plan/DeviceIR/template separation. Hardware quirks and vendor-specific logic are handled via structured Model-to-Model transformations, not ad-hoc string concatenation. Without this separation, a vendor firmware upgrade forces changes across every template that touches the affected feature—the kind of cascading edit that causes outage-inducing typos.
2. **Separation of what from how.** The Blueprint (annotated topology) declares *what* the network contains—devices, links, and their semantic properties. The design plan declares *which* design rules to apply when compiling that topology (meshing style, allocation strategy, assertion constraints). The mapping document and templates declare *how* to render the enriched model into per-device configuration. Platform-specific rules are injected via `imports: ['hardware-lowering.yaml']` and isolated behind `when: device_os == ...` predicates. The same topology and design plan can produce Cisco IOS, Arista EOS, or Juniper JunOS output by swapping only the mapping document and templates. This means a campus migration from Cisco IOS to Arista EOS changes the mapping document and templates—not the design plan or topology. Intent survives platform churn.
3. **Topology as the single source of truth.** Topology is the root from which all other network-management data derives. NetCfg treats the graph as the canonical representation. IP addresses, protocol sessions, policy rules, and security zones are *computed* from topology relationships, not declared independently. When addresses, sessions, and policies are computed from relationships

rather than declared independently, the system eliminates an entire class of stale-reference bugs: if a link disappears from the topology, its addresses and peering sessions vanish automatically.

Audience

This document is intended for:

- Engineers integrating NetCfg into a network automation pipeline.
- Contributors to the NetCfg and NTE codebases.
- Researchers evaluating graph-based approaches to network configuration.

Familiarity with YAML, basic graph theory, and IP networking is assumed. Experience with programming is helpful but not required to extend the engine.

> **Companion report: NTE-TR-001**
>
> NetCfg implements the compilation leg of the deployment strategy defined in NTE-TR-001 §Core Architectural Insights. NTE provides the graph substrate; NetCfg compiles annotated topologies into verified device configurations. NetCfg operates exclusively in NTE's Strict Mode (design-authority, schema-validated compilation).

How to read this document

The report follows three reading arcs rather than three separately typeset parts. **Architecture & Insights** introduces the conceptual model, the network data model, intent and policy expression, and the compilation pipeline—purely conceptually, with implementation detail deferred. **The Design Plan Language** specifies the declarative surface: plan structure, selector syntax (NTE-QL), named groups and objects, the primitive catalogue, and target-specific transforms. **Engine Implementation** then shows how those abstractions execute in practice: pipeline internals, core mechanisms (IPAM, protocol overlay, DiffEngine, TSDM), the CLI, an end-to-end worked example, the error model, editor integration, and current limitations.

If you are evaluating the idea, read Sections 1–3 first. If you are authoring plans, jump next to Section 4. If you are integrating or extending the tool, Sections 5–8 and the appendices are the operational reference.

1.4 Case Study: From PhD to Production (Architectural Synthesis)

The NTE design philosophy is rooted in the **Architectural Synthesis** strategy developed during PhD research into automated network design. This framework treats network design as a three-stage transformation pipeline where the `ank_netcfg` compiler acts as the synthesiser, lowering high-level intent into the rigorous NTE topology model: **Design Intent → Synthesis Plan → Layered Topology**.

Figure 2. The Architectural Synthesis Pipeline. `ank_netcfg` aggregates intent from abstract human domains into a formal plan, then lowers it into the NTE graph representation.

1.4.1 Case Study: Segment Routing (SR-TE) Migration

Consider a large-scale migration from RSVP-TE to Segment Routing (SR-TE) in a service provider core. In the PhD thesis, this migration spans two forwarding paradigms over a shared physical underlay.

1. The Design Intent. The architect defines high-level constraints: "Gold traffic must avoid PoP-X and use low-latency links." This intent is captured in an abstract specification mapping service classes to topological constraints.

2. The Synthesis Plan. The `ank_netcfg` synthesis engine aggregates these constraints into a singular `SR_Policy_Plan`. NTE represents this as a set of high-level `Semantic` edge declarations targeted for expansion.

```yaml
nodes: [R1, R2, R3, R4]
sr_policies:
  - id: "GOLD-PATH"
    head: R1
    tail: R4
    constraints: { avoid: [PopX], metric: latency }
```

3. The Layered Transformation. The `ank_netcfg` compiler lowers this Synthesis Plan into the formal NTE graph model:

- **Underlay (L1/L2)**: NTE verifies the physical fibre and IGP adjacencies (IS-IS).
- **Shadow Layer**: The synthesis engine creates **Shadow Nodes**—synthetic vertices representing logical constructs (here, Segment IDs) not yet present in the discovered topology—for the required SIDs.
- **Overlay (SR-TE)**: The compiler lowers the `Semantic` edge (an intent-level link carrying policy metadata) into `Parent` edges (hierarchical cross-layer bindings) linking the policy to the underlying physical path.

NTE-TR-001 [11] provides full definitions of NTE graph primitives (Shadow Nodes, Semantic edges, Parent edges, and Orthogonal Graph Planes).

This synthesis framework lets architects reason at the intent level while the NTE engine enforces production-grade cross-layer mapping and verification.

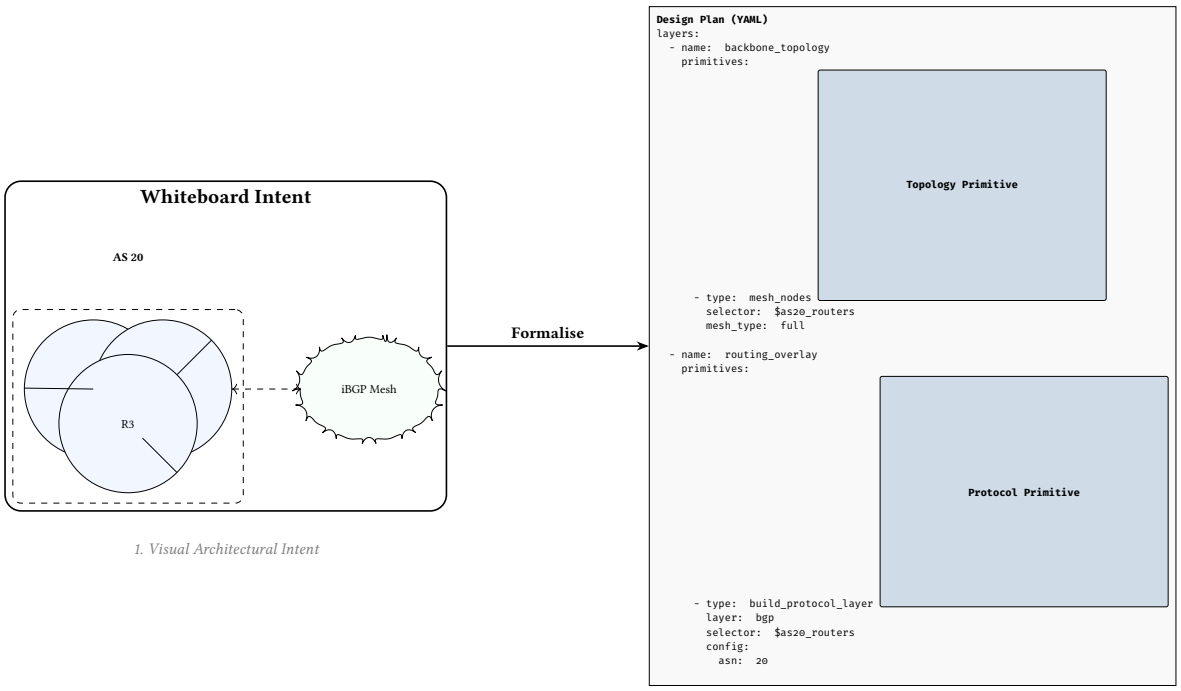
2 Architecture

Three ideas underpin every NetCfg design plan: (1) the network model is a *layered graph*; (2) design plans *compose* the model declaratively, layer by layer; (3) every primitive follows the same *select* → *enrich* → *validate* pattern. This section establishes the mental model; the formal data model (§3) and DSL syntax (§4) build on it.

Keep the mental model, postpone the mechanisms. Treat NetCfg as a compiler over a graph: selectors identify where to act, primitives add derived state, and assertions gate the output. Implementation details (engine internals, NTE-QL compilation, DataFrames) make this model fast and safe but are not needed to follow the flow.

2.1 The Intent Formalization Boundary

The NetCfg workflow begins at the *Intent Formalization Boundary*. Visual intent (sketches, topology diagrams, or design requirements) is formalised into a standardised **Blueprint** and a corresponding **Design Plan**. This process (Figure 3) anchors the abstract architectural design in a machine-readable model.



1. Visual Architectural Intent

2. Machine-Readable Design Plan

Figure 3. The Intent Formalization Boundary. Architectural intent is captured visually and then formalised into a declarative design plan. Logical constructs like "iBGP Mesh" map directly to high-level protocol primitives in the NetCfg DSL.

2.2 The conceptual workflow

NetCfg treats network configuration as a pure compilation problem. Instead of writing per-device stanzas, operators declare design rules in a **Design Plan** that the synthesis engine processes through a four-stage pipeline. The pipeline separates domain knowledge (the network architecture and vendor mappings) from engine mechanics (graph mutations and DeviceIR generation). This funnel progressively refines high-level abstract intent down to concrete, hardware-specific syntax.

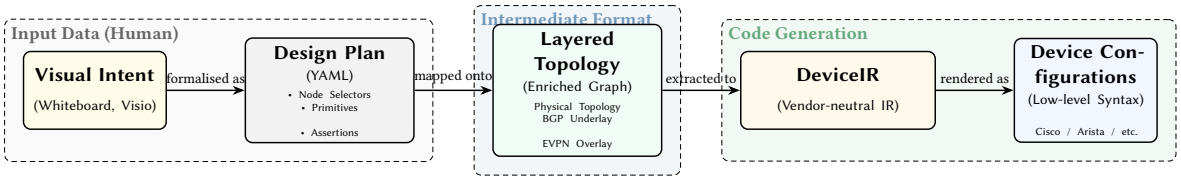


Figure 4. The Configuration Synthesis Pipeline. The operator formalises high-level **Visual Intent** into a declarative **Design Plan**. The primitives in this plan are mapped onto a **Layered Topology**, which is extracted into a **DeviceIR** and rendered as low-level **Device Configurations**.

Because state can only flow forward through the pipeline, the final syntax is strictly a mathematical derivative of the design intent, eliminating the drift and "spaghetti" configuration inherent to manual device-by-device workflows.

Case study: multi-tenant isolation

The multi-tenant data-centre case study (`examples/case_studies/04_multi_tenant_dc.yaml`) illustrates how these concerns compose. The operator declares: (1) groups that name device roles and per-tenant link populations; (2) a hub-and-spoke mesh for the fabric; (3) per-VRF address pools; (4) access policies that deny cross-VRF traffic; (5) assertions that every device has a tenant assignment and that each tenant's subgraph is connected; and (6) a blast-radius limit. At no point does the operator name a specific device or IP address—the compiler derives all concrete state from the declared populations and intent.

Assertions, rules, and primitives can be factored into reusable libraries and shared across design plans via `imports:`. §4.6.32 catalogues the standard libraries that ship with NetCfg.

2.2.1 The Compilation Pipeline

This section describes the pipeline conceptually; Part III (§A) covers each layer's implementation.

NetCfg's compilation pipeline has four stages, each corresponding to a distinct level of abstraction, with assertions acting as cross-cutting quality gates rather than a separate stage. Figure 5 shows the data flow from abstract intent to concrete syntax.

A key property of this pipeline is **determinism**: configuration is a pure function $f(\text{Annotated Topology}, \text{Design Plan}) = \text{DeviceIR} \to \text{Config}$. If neither the topology annotations nor the design plan strategies have changed, regenerating the configuration produces a bit-identical result, making any divergence immediately detectable via diff. Combined with the structural diff engine (§A.9), this enables a "regenerate and compare" workflow for detecting configuration drift without runtime monitoring.

> **Pipeline granularity**
>
> The conference paper describes six implementation steps (parse, shadow, execute, map, render, write). This report groups these into four conceptual stages: (1) Intent Synthesis (parse + shadow + execute), (2) DeviceIR Extraction (map), (3) Target-Specific Lowering (TSDM transforms), and (4) Syntax Serialisation (render + write). Assertions are evaluated after all primitives complete but before DeviceIR extraction; mid-pipeline `assert_state` primitives provide earlier feedback within Stage 1.

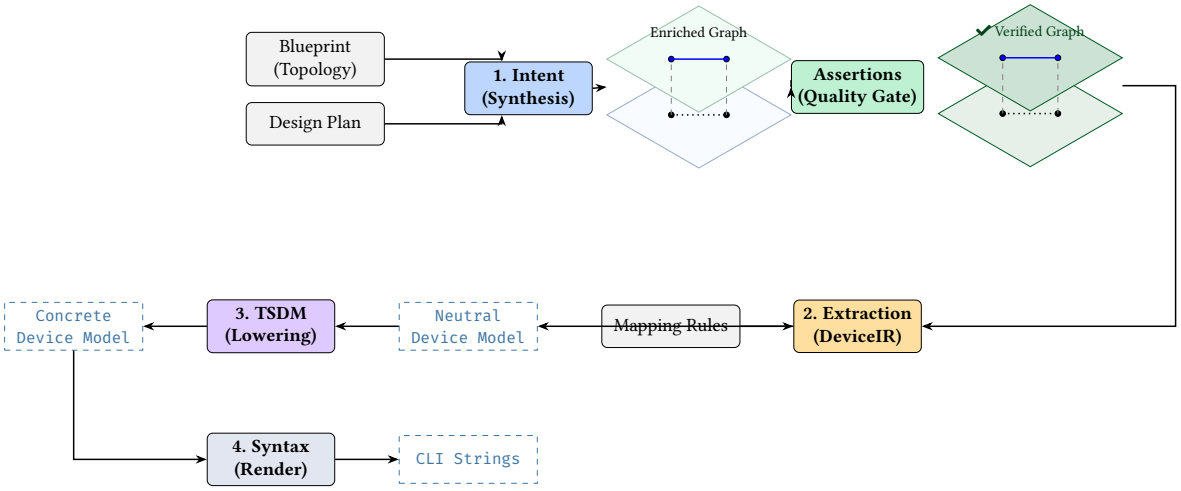


Figure 5. NetCfg compilation pipeline. Four stages lower intent through three mathematical representations (Enriched Graph, Neutral Device Model, Concrete Device Model). Assertions act as a cross-cutting quality gate between Stage 1 and Stage 2, not a separate pipeline stage.

3 The Topological Ontology (Data Model)

To synthesise configuration across a multi-vendor landscape, NetCfg relies on a highly structured internal data model. This section defines the ontology that later sections assume: what counts as a node, how layers relate, where renderable state lives, and why a graph model is necessary rather than incidental.

3.1 Design choices

The classical graph $G = (V, E)$ is insufficient for multi-vendor configuration synthesis. Real hardware hierarchies, vendor schema differences, and multi-layer protocols require four foundational design choices, each solving a specific modelling failure.

Endpoint ontology. In classic computer science graph theory, a network is often modelled as a simple graph $G = (V, E)$, where V are routers and E are the cables connecting them. This abstraction breaks down at scale. In reality, cables do not plug into "routers"; they plug into interfaces, which are hosted on transceivers, which reside on line cards, which are inserted into chassis slots. To accurately model physical realities and ensure deterministic algorithmic traversal, NetCfg abandons the naive $G = (V, E)$ approach. Instead, it enforces a strict **Endpoint Ontology**: Nodes connect via Structural edges to Endpoints, and Endpoints connect via Connectivity edges to other Endpoints (`Node` → `Structural` → `Endpoint` → `Connectivity` → `Endpoint` → `Structural` → `Node`). This provides a mathematically uniform canvas for algorithms, freeing them from parsing custom vendor chassis logic. This endpoint ontology provides the uniform traversal canvas that selectors (§4) rely on.

Fractal containment. Network devices are not atomic black boxes; they are networks unto themselves. A single modular core router contains its own internal fabric, line cards, and VRFs. The NetCfg ontology treats topology as fractal: the exact same graph primitives (Nodes and Structural edges) used to build a macro data centre topology are used to build the internal containment hierarchy of a single switch. Because containment uses the same graph primitives as inter-device topology, a query written to trace a WAN path works unmodified inside a modular chassis.

Property flattening. YANG models and NetBox store state in nested, schema-specific trees. When data is buried inside a vendor's JSON payload, generic graph algorithms cannot reach it—a shortest-path query has no way to parse an Arista-specific nested block to extract link delay. NetCfg enforces **Property Flattening**: all domain semantics and state are stored as flat key-value pairs directly on the topological primitives (Nodes and Edges). This decoupling creates an Algorithmic Canvas: engineers can write generic Cypher-based graph queries (e.g., `MATCH (n) WHERE n.status == 'down'`) without needing to understand the underlying schema. Property flattening is what makes the NTE-QL selector language (§4.6.2) possible: queries match flat key–value predicates rather than parsing nested vendor JSON.

Orthogonal layering. Networks span multiple protocol layers. A physical fibre cut (Layer 1) causes an IP adjacency drop (Layer 3), which drops a BGP session (Layer 4), which withdraws a VPN route (Layer 7). In NetCfg, these layers are not just tags; they are Orthogonal Graph Planes bound by strict hierarchical dependencies (Parent edges). The entire digital twin is modelled as a **Directed Acyclic Graph (DAG) of Graphs**. This allows the engine to treat the BGP topology as a completely pure, mathematically sound graph for BGP algorithms, while still retaining the deterministic physical lineage back to the hardware. Parent mappings between layers are the mechanism that `build_protocol_layer` (§4) uses to project physical nodes into logical overlays while preserving hardware lineage.

3.2 Per-node state: DeviceContext

§3.3 explained that all renderable state lives on nodes. Formally, each node's properties are stored as structured metadata in the columnar property store D, keyed by node type. NTE-QL expressions evaluate predicates against this store to select node populations on demand. The synthesis engine exposes a typed *device context* whose fields are progressively enriched by primitives:

The choice to centralise all mutable state on nodes (rather than edges or external stores) has three engineering consequences. First, producing a device's configuration requires reading exactly one node—the mapping evaluator has no joins or cross-references. Second, primitives compose without coordination: each reads the context its predecessor left and appends new fields, so adding a new primitive cannot break an existing one. Third, deterministic rendering follows directly: the same enriched node always produces the same output file.

Table 1. Per-node device context: fields populated by the synthesis engine pipeline. Fields marked with † are written by specific primitives; all others originate from the Blueprint.

Property	Architectural Source
`hostname`	Extracted from the digital twin node identity
`device_os`	Declared as an immutable topology annotation
`management_ip`	Declared as an immutable topology annotation
`src_ip`, `dst_ip`, `subnet`	Synthesized dynamically by the `provision_ips` design rule †
`src_ipv6`, `dst_ipv6`, `subnet_v6`	Synthesized dynamically by the `provision_ips` design rule † (dual-stack)
`vrf`	Inherited from topology or dynamically allocated †
`peer_interfaces`	Topologically computed by the `mesh_nodes` design rule †
`next_interface_index`	Monotonic counter for abstract interface indices †
`loopback0`	Loopback address (IPv4, no prefix length), from IPAM †
`vtep_ip`	VXLAN VTEP tunnel endpoint address †
`bgp_as`	BGP autonomous system number for this device †
`router_id`	OSPF/IS-IS router ID (IPv4 address string) †
`security`	Typed security IR: zone policies and NAT rules (`SecurityModel`) †
`qos`	Typed QoS IR: classification, marking, shaping (`QosModel`) †
`stanzas`	The accumulated logical configuration model †
`(any other key)`	Free-form topological metadata or enriched facts

Primitives progressively enrich this context: `mesh_nodes` establishes edges and writes `mesh_type`/`peer_count` into metadata; `provision_ips` writes IP fields; `build_protocol_layer` deep-merges config overlays into the cloned node's metadata. By the time rendering occurs, each node's device context contains all information needed to produce its configuration file.

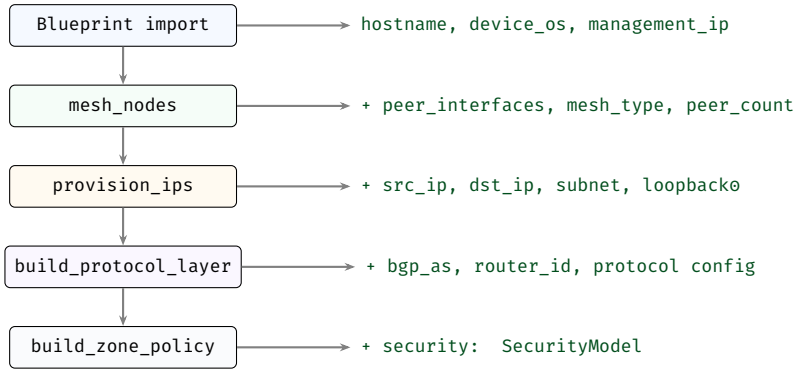


Figure 6. Progressive `DeviceContext` enrichment. Each primitive reads the context left by its predecessor and writes additional fields. By rendering time, the context carries all information needed for configuration generation.

Note

Metadata flattening. Any topology annotation that does not match a named field is captured in the node's metadata map. The metadata map uses `#[serde(flatten)]`—when `DeviceContext` is serialised (e.g. for the `TargetDeviceModel`), metadata keys appear as *top-level keys*, not nested under a `metadata` object. Template authors access `bgp_as` directly, not `metadata.bgp_as`. This makes the device context extensible without schema changes—operators can attach arbitrary properties in the topology and reference them from selectors or templates.

Note

Graph engine dependency. The NTE topology engine [12] (NTE-TR-001) provides the directed graph with columnar (Polars DataFrame) property storage. NetCfg uses NTE for the graph, the

mapping evaluator, and DeviceIR types. NetCfg's `mesh_nodes` and `build_protocol_layer` primitives rely on the Endpoints Model—a 3-hop traversal pattern (NTE-TR-001 §Data Model) that discovers adjacent devices via the DWC (Device–Wire–Cable) abstraction.

3.3 The network model as a Layered Topology

NetCfg represents the network-wide state using a **Layered Topology**. The topology is a graph of *nodes* (devices) and *edges* (links). The overall network state is captured in a stack of these planes, starting from the physical cabling up through the logical protocol overlays.

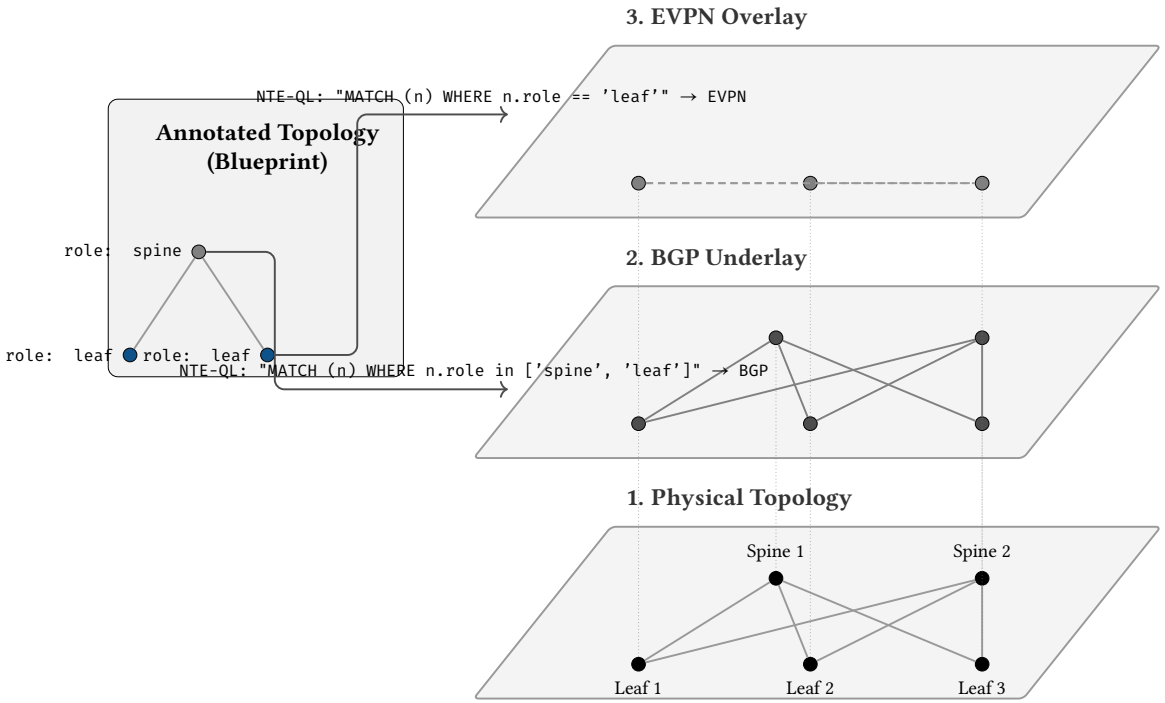


Figure 7. Intent Mapping to Layered Topology. Rather than compiling unstructured templates, operators declare intent in a Design Plan. Primitives in the plan directly map logical intent onto orthogonal topology layers.

Mapping Intent into Layer Projections. A source of confusion in network automation is the distinction between what an operator *declares* and what the engine *computes*. In NetCfg, we use specific terminology:

- **Mapping (Human domain):** The operator writes a Design Plan (YAML). This plan *maps* intent via Primitives (like `build_protocol_layer`) to a target population (like "all leaves").
- **Projection (Engine domain):** The synthesis engine reads the plan and mathematically *projects* nodes from the physical layer upwards into logical protocol layers (e.g. creating BGP nodes from physical nodes).

When a primitive executes, it maps the user's intent directly onto the topology. As nodes are projected into a logical layer, the primitive applies structural *transformations*, merging the intended configuration (such as AS numbers or VPN address families) onto the newly created overlay nodes.

A third use of "mapping" in this report refers to the *mapping document*—the YAML rule set that converts DeviceIR fields into vendor-specific configuration stanzas (see Appendix L). Context disambiguates: "parent mapping" always means the `_source_id` lineage link; "mapping document" or "mapping rules" always means the DeviceIR-to-stanza conversion rules.

Because every projected node retains a parent mapping back to its physical origin, the logical overlay implicitly inherits the physical truths of the network without redundant data entry. The synthesis engine does not need to separately map IP addresses into EVPN configurations; the EVPN node simply looks down its parent mapping to retrieve the hardware's allocated addressing.

> **Note**
>
> **Design note: logical edges do not mirror physical edges.** Because protocols exist in their own structural planes, their topologies can differ entirely from the physical hardware. An EVPN layer might project a full mesh of VXLAN tunnels between Leaf nodes, ignoring the physical Spines entirely, while an OSPF layer might strictly follow every physical point-to-point cable. Layer projection decouples the logical network architecture from the physical cable plant.

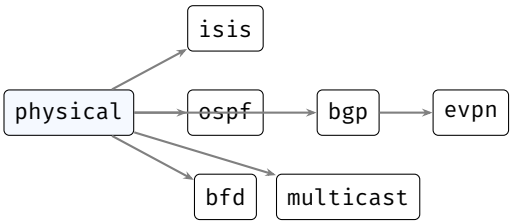
Kinds of layers (a practical taxonomy)

Layers are intentionally lightweight: they let the model hold multiple simultaneous views of the same network without conflating their semantics. In practice, most NetCfg design plans use a small, repeatable taxonomy:

1. **Physical layer.** Nodes represent devices; edges represent physical links. Typical state: interface naming, link addressing, management reachability, hardware inventory.
2. **Protocol layers (routing/switching overlays).** Nodes represent a device's participation in a protocol (one node per device per protocol); edges represent logical adjacencies or sessions. Typical state: neighbours, timers, areas/ASNs, policy attachments, protocol feature flags.
3. **Policy and service layers (optional).** Some organisations model policy and services as dedicated layers when the relationships are graph-shaped (e.g., security-zone bindings, NAT relationships, telemetry subscriptions). Others keep these as node metadata attached to the physical or protocol nodes. The design rule is the same either way: *renderable state stays on nodes; edges express relationships.*

This taxonomy is not prescriptive: the point is that layering preserves separation of concerns while keeping every derived fact traceable back to the physical topology via the parent mapping.

The layer DAG. Layers form a directed acyclic graph (DAG) rooted at the physical layer. In a typical datacentre fabric design, the structure is:



Every overlay node carries `_source_id` (the parent physical node ID) and `protocol_layer` (the layer name). This follows the MALT pattern [3] of typed cross-layer relationships, though with a simpler two-field parent mapping rather than MALT's full relationship-kind taxonomy. The EVPN–BGP parent link is a design-plan choice: if `build_protocol_layer` for EVPN selects from the BGP layer, EVPN nodes parent to BGP nodes; if it selects from the physical layer, they parent to physical nodes instead.

Design rules that build overlays

Layering alone provides representation; *design rules* provide synthesis. In NetCfg, design rules are implemented as primitives that read from the current model (often the physical layer) and then materialise overlay structure and per-device stanzas. They are the bridge from plan topology to protocol overlays.

Two common classes of design rule are:

1. **Overlay construction rules.** `build_protocol_layer` clones selected physical nodes into a protocol layer, records the parent mapping (`_source_id`), and deep-merges protocol defaults into the clones. With `clone_underlying: true`, it also seeds the overlay with a derived copy of the physical connectivity.

2. **Session synthesis rules.** Protocol session primitives (e.g. `build_bgp_session`, `build_ospf_session`, `build_isis_session`)[1] consume the overlay graph and emit configuration stanzas on the participating nodes. These rules are where operator intent becomes concrete neighbours, interfaces, and per-peer parameters.

3.4 Modelling Real-World Protocols

Insight: *The operator does not "mesh in the BGP layer" manually. They declare intent; the design rules materialise the overlay relationships and the resulting per-device stanzas in the appropriate layer.*

The critical design insight is that *all protocols fit the same modelling pattern*: nodes represent routing entities, edges represent sessions or adjacencies, and layers isolate protocol concerns. This uniformity means the synthesis engine needs no protocol-specific code paths—`build_protocol_layer` handles OSPF, BGP, IS-IS, EVPN, BFD, and multicast through a single typed-schema mechanism. The following catalogue shows how each protocol maps to this universal pattern. The concrete design-rule syntax, strict validation schemas, and plan schemas for each protocol are given in §A.8.

3.4.1 Layer 3 Foundations: OSPF and IS-IS

In the IP routing layer, nodes represent routers or virtual routing instances (VRFs).

Adjacencies are modelled as `Connectivity` edges between `LogicalEndpoint` vertices representing routed interfaces (e.g., /30 or /31 point-to-point links). This supports both OSPF [13] and IS-IS [14] adjacency topologies.

Areas/Levels are modelled as `LogicalNode` vertices. A router participating in OSPF Area 0 has a `Parent` edge to the `Area0` LogicalNode.

LSDB Visibility Protocol-specific state (e.g., OSPF Router IDs, IS-IS System IDs) is stored in the property map of the `Node` or `LogicalEndpoint`.

3.4.2 BGP Control Plane

BGP [15] is modelled as a logical overlay on top of IP reachability.

BGP Sessions are modelled as `Connectivity` edges at the `bgp` layer. For multi-hop eBGP, these edges connect `LogicalEndpoint` vertices (Loopbacks), while for single-hop eBGP, they may connect physical `Endpoint` vertices.

AS-Path and Communities are stored as columnar attributes on the BGP `Connectivity` edges, enabling high-performance SIMD filtering (e.g., "Find all sessions with AS 65000 in the path").

Route Reflectors are modelled using `Semantic` edges to represent the client-to-RR relationship without requiring explicit endpoint pinning.

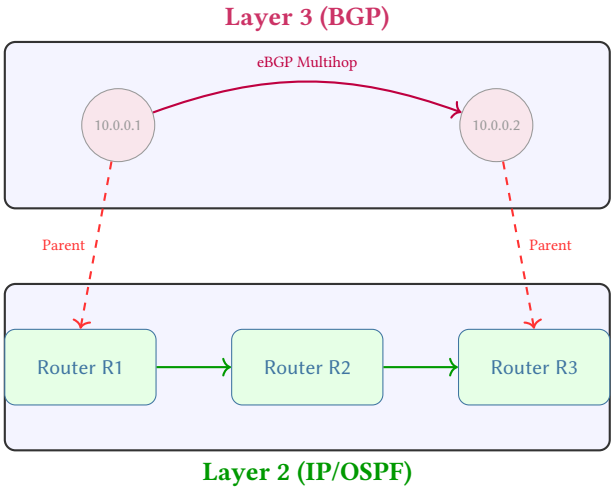


Figure 8. Multi-hop BGP dependency mapping. A logical BGP session at L3 is anchored to IP endpoints on R1 and R3. The session's existence depends on the multi-hop path (R1-R2-R3) in the underlay IP layer.

[1] Session synthesis primitives are planned but not yet in the catalogue; the current workflow uses `build_protocol_layer` with a config overlay to achieve the same result.

3.4.3 MPLS and Segment Routing (SR)

Multiprotocol Label Switching (MPLS [16]) introduces label-switched paths (LSPs) that may not follow the shortest IGP path.

Label Switched Paths (LSPs) are modelled as `Semantic` edges at the `mpls` layer. An RSVP-TE tunnel from R1 to R5 is a single Semantic edge with an attribute containing the full `hop_list`.

Segment Routing (SR-TE) is modelled using **Shadow Nodes** for the segment identifiers (SIDs). The engine can verify if an SR policy's SID-list is topologically valid by traversing the underlying physical layer.

LDP/RSVP Sessions are modelled as `Connectivity` edges at the `mpls_control` layer, sharing `Parent` edges with the physical interfaces they run on.

3.4.4 Network Virtualization: VXLAN and EVPN

Modern data centre fabrics use VXLAN [17] encapsulation orchestrated by BGP-EVPN [18]. The typed overlay rules and strict schemas for VXLAN and EVPN are shown in §A.8.

VTEPs (VXLAN Tunnel Endpoints) are modelled as `LogicalEndpoint` vertices on the `vxlan` layer.

VNIs (VXLAN Network Identifiers) are modelled as `LogicalNode` vertices. All VTEPs participating in a specific VNI have a `Parent` edge to that VNI node.

EVPN Control Plane is modelled as a sub-family of the `bgp` layer. MAC/IP reachability is stored as property metadata on the `vxlan` Connectivity edges.

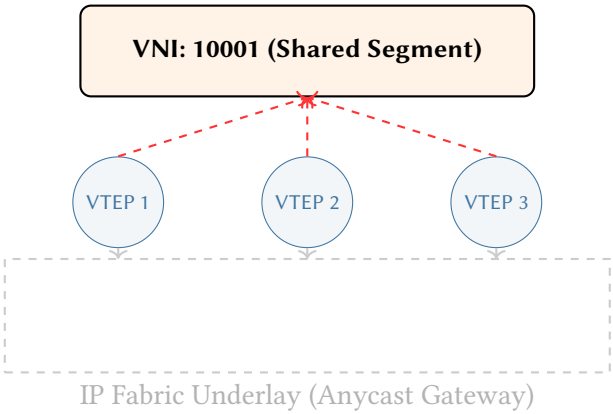


Figure 9. VXLAN VNI as a logical shared segment. Individual VTEPs are mapped to a central `LogicalNode` representing the VNI. This allows NTE-QL to treat the complex underlay fabric as a single broadcast domain for overlay verification.

3.5 Topology provenance

The Blueprint can enter NetCfg through three channels, each preserving the node annotations that selectors and primitives operate on:

1. **Hand-authored structured formats.** The operator writes the topology file directly, placing each node's annotations under a `properties:` block. This is the most explicit channel and is common during greenfield design or lab work.
2. **Import from existing configurations** (`netcfg import`, §D.6). Parses a directory of device configurations (or a device inventory file) and produces a topology archive (`.nte`), inferring hostnames, links, and device platforms from the configuration data. Operators then enrich the imported topology with additional annotations (roles, sites, trust levels) before compiling.
3. **NSoT synchronisation** (`netcfg sync pull`, §D.12). Pulls a live topology from a network source of truth such as NetBox [5] via its GraphQL API, mapping inventory fields to topology annotations. This channel keeps the Blueprint in sync with the production network.

Regardless of channel, the result is the same data structure: a directed graph with a structured property document per node. Primitives and selectors treat all topologies uniformly—one imported from NetBox compiles identically to one hand-authored in YAML.

With the data model established, we turn to how operators express intent over it.

4 The Design Plan (DSL)

This section turns the conceptual model into an authoring surface. It starts with the three input documents, then narrows to plan structure, selectors, named abstractions, and the primitive catalogue that composes the layered network model.

4.1 Three input documents

An operator supplies three documents to the synthesis engine:

1. **Annotated topology** — the "source code" of the network. A graph of devices (nodes) and links (edges), where each node carries rich semantic annotations: `hostname`, `role`, `site`, `device_os`, trust levels, address pools, protocol flags, and any other properties the operator considers part of the network's design intent. The topology may be hand-authored (structured formats), imported from a network inventory system via `netcfg import` (§D.6), or pulled live from an NSoT such as NetBox [5] via `netcfg sync pull` (§D.12).
   These annotations are the data that the synthesis engine's selectors (e.g. `MATCH (n) WHERE n.role =='core'`) and primitives operate on. The richer the annotations, the more the synthesis engine can derive automatically. Ideally, the topology captures *all* network-wide design intent; the design plan then reduces to a thin set of compilation strategy choices.
2. **design plan** — *which* compilation strategies to apply. A declarative specification declaring layers of primitives that select subsets of the Blueprint and derive new state: meshing patterns, IP allocation pools and strategies, protocol overlay construction, policy rules, and assertion constraints. The design plan does not duplicate what is already in the topology; rather, it names the *design rules* that transform topology-level annotations into device-level state.
   The relationship is analogous to a traditional compiler: the Blueprint is *source code* and the design plan is a *compilation profile* (flags such as `-O2` or target architecture). The source code carries the program's semantics; the compilation profile selects *how* those semantics are lowered. Similarly, the topology carries network intent; the design plan selects which primitives lower that intent into device-level state.
3. **Target backend** — *how* to turn the enriched model into per-device configuration. In practice this is a small directory containing (i) target-specific transforms that lower vendor-neutral intent into platform-specific structure, and (ii) templates that serialise that structure into concrete CLI or configuration syntax. Most organisations author this once per platform and reuse it across sites.

> **Note**
>
> **Where does intent live?** The boundary between topology annotations and design plan declarations is a design choice that operators can tune. At one extreme, the topology carries only physical inventory (hostnames and cabling) and the design plan encodes all design rules. At the other, the topology carries rich intent (address pools, protocol parameters, zone memberships) and the design plan is a thin strategy selector. NetCfg supports both styles; the general trajectory is toward richer topologies with thinner design plans, because this keeps intent closer to the source-of-truth and reduces the coupling between the design plan and a specific topology's node schema.

4.1.1 Canonical annotation vocabulary

Operators may attach *any* key-value pair to topology nodes; a set of well-known keys (`hostname`, `role`, `device_os`, `zone`, etc.) receive special treatment from selectors, primitives, and the rendering engine. Appendix K catalogues the full annotation vocabulary (Table 25).

Any key not listed there is stored in the node's metadata map and remains accessible to selectors and templates. For example, an operator might annotate nodes with `trust_level: high` or

`loopback_pool: 10.255.0.0/24`; these values are available to NetCfg queries and MiniJinja templates without any schema changes.

4.1.2 Target backend structure

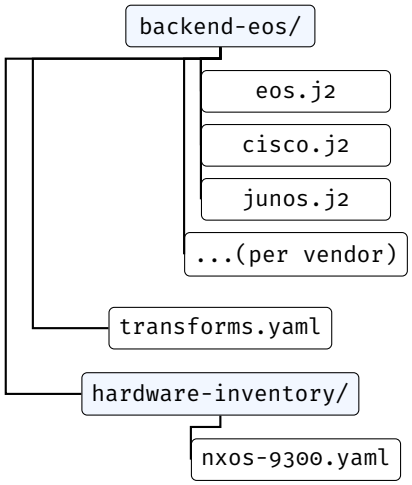


Figure 10. Target backend package structure. Templates render per-vendor syntax; `transforms.yaml` declares TSDM lowering rules; `hardware-inventory/` maps chassis models to slot layouts. Most organisations author this once per platform and reuse it across sites.

4.2 The five concerns of a network design plan

Every design plan addresses five orthogonal concerns that must be mapped from the architect's intent into the final configuration of the network. Table 2 serves as the Traceability Matrix, mapping human intent to pipeline mechanisms.

Table 2. The Traceability Matrix: mapping the five concerns of a network design plan to final configuration state.

Concern	DSL construct	Question answered	Configuration impact
Population naming	`groups:`	Which devices belong to which roles?	Determines target nodes for rendered configuration stanzas.
Topology intent	`layers:` / primitives	How should they be connected and what state should they carry?	Generates interfaces, IPs, and routing protocol sessions.
Policy constraints	routing/access/prefix-list primitives	What traffic rules should be enforced?	Synthesizes route-maps, prefix-lists, and ACLs.
Design invariants	`assertions:` + `rules:`	What properties must always hold?	Provides pre-flight validation (no direct config output).
Change safety	`blast_radius:`	How much change is acceptable per commit?	Acts as deployment pipeline gates (no direct config output).

- **Population naming** (`groups:`) — which devices belong to which roles.
- **Topology intent** (`layers:`/primitives) — how devices connect and what state they carry.

- **Policy constraints** (routing/access/prefix-list primitives) — what traffic rules to enforce.
- **Design invariants** (`assertions:`) — properties that must always hold.
- **Change safety** (`blast_radius:`) — how much change is acceptable per commit.

These concerns are not independent—they form a dependency chain. Population naming must precede topology intent (you cannot mesh nodes you have not named). Topology intent must precede policy constraints (you cannot filter traffic on links that do not exist). Design invariants span all preceding concerns (they verify the accumulated model). Change safety gates the entire pipeline. This ordering is why the design plan's `layers:` construct enforces declaration order: earlier layers establish the structure that later layers constrain.

The key insight is that every concern operates on the *network model*—the layered graph described in §3. The operator declares populations, intent, policy, and invariants; the synthesis engine maps these into graph mutations that progressively enrich the model's nodes, ultimately generating the final configuration artefact for each device.

4.3 From plan to network model: declarative composition and Tiered Execution

§4.4 below formalises the five primitive tiers and their mutation semantics. This subsection previews how those tiers interact when composing a design plan.

A fundamental mathematical property of the synthesis engine is **Tiered Execution**. The synthesis pipeline resolves dependencies by evaluating primitives in a strict, forward-only Directed Acyclic Graph (DAG):

1. **Topology:** Establishes the physical and logical edges required for connectivity.
2. **Allocation:** Assigns resources (IPs, ASNs, VNIs) strictly to the edges created in Phase 1.
3. **Protocol:** Binds routing paradigms (OSPF, BGP) over the allocated addresses from Phase 2.
4. **Policy:** Implements security and reachability constraints over the established protocols in Phase 3.

This Tiered Execution model guarantees that identifiers required by policy are always deterministically allocated before policy compilation begins.

`provision_ips` iterates edges to determine which node pairs need addresses, but writes `src_ip` and `dst_ip` onto the respective *nodes*—not onto the edge itself. `build_protocol_layer` clones selected nodes into a new graph layer and merges protocol configuration into them via deep merge. Each primitive sees the enriched model left by its predecessors, but never needs to know what they did.

> **Design Rule 2: Node-Centric State**
>
> All per-device state is written to nodes via `DeviceContext`. Edges carry no mutable state; they exist solely to express relationships between nodes. This invariant simplifies the mapping stage: to produce a device's configuration, the mapping evaluator reads exactly one node.

Topology projections are the mechanism that links intent to the graph. Rather than binding configuration templates to device names via ad-hoc scripts, NetCfg uses declarative queries (NTE-QL) to project physical node populations into logical layers. For example, a design rule selects all `role == 'leaf'` nodes and projects them into a BGP layer. This layer-scoping ensures that protocol design rules only operate on relevant network segments, treating the network as a coherent mathematical structure rather than a collection of independent text files.

4.4 Design plans compose the model declaratively

A design plan organises primitives into ordered *layers*. Each layer declares a set of primitives; each primitive selects a subset of the graph and enriches the nodes it touches. The result is a progressively richer model, as Figure 11 illustrates.

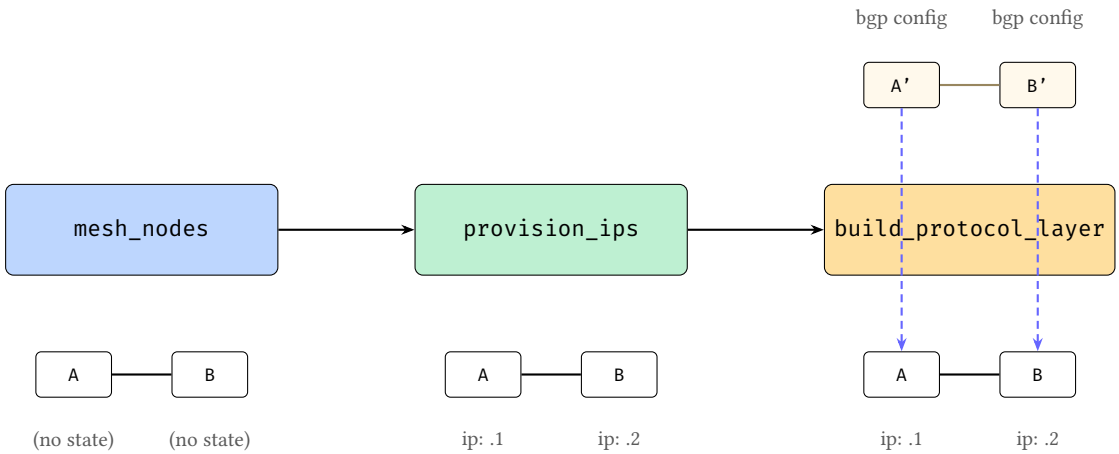


Figure 11. Progressive model enrichment. Left: `mesh_nodes` establishes edges between bare nodes. Centre: `provision_ips` annotates nodes with IP addresses. Right: `build_protocol_layer` clones nodes into a protocol layer and deep-merges a protocol overlay into the clones; dashed arrows show the parent mapping back to the physical devices. At each stage the model grows richer; no primitive needs to know what others have done.

Three properties enable composition:

1. **State lives on nodes, not edges.** Edges define relationships—physical links, peering sessions, policy bindings. Each primitive writes mutable, renderable state to the nodes at each end of an edge. Edges carry static relationship metadata (e.g., labels) but never configuration-bearing state.
2. **Primitives compose, not override.** Each primitive enriches the model additively. No primitive needs to know what other primitives have done; the model accumulates state layer by layer.
3. **Primitives are typed model transforms.** Each primitive belongs to one of five tiers, distinguished by what it mutates: *Topology* (new nodes and edges), *Allocation* (IP addresses, ASNs, VLANs), *Policy* (firewall rules, route-maps), *Overlay* (VXLAN VNIs, EVPN route-targets), or *Meta* (pipeline annotations and gates). Primitives never name specific devices; they reference *groups* and *selectors* (graph queries that resolve against the current model state), so the same design plan can operate on topologies of different sizes.

4.5 Mini worked example (conceptual)

The following design plan fragment illustrates the layered-model flow without requiring knowledge of the underlying implementation.

Listing 1. A minimal design plan fragment (layered model + design rules).

```
layers:
  - name: physical
    primitives:
      - type: mesh_nodes
        selector: "MATCH (n) WHERE n.role =='core'"
        mesh_type: full

      - type: provision_ips
        selector: "MATCH ()-[e]->()"
        pool: 10.0.0.0/24
        subnet_size: 30

  - name: bgp
    requires: [physical]
    primitives:
      - type: build_protocol_layer
        selector: "MATCH (n) WHERE n.layer =='physical' && n.role =='core'"
        layer: bgp
        clone_underlying: true
        config: { bgp: { enabled: true } }

      # With clone_underlying: true, BGP overlay edges are derived
```

```
    # automatically from the physical topology.  No separate
    # session primitive is needed.

assertions:
  - name: core-has-link-address
    severity: error
    select: "MATCH (n) WHERE n.role =='core'"
    check: { type: field_exists, field: src_ip }
```

Conceptually, this executes as follows:

1. **Start with a Blueprint** containing core routers as physical nodes.
2. **`mesh_nodes`** creates physical edges between those nodes.
3. **`provision_ips`** examines the edges to decide which links need subnets, then writes the resulting link addresses onto the endpoint *nodes* (not onto the edges).
4. **`build_protocol_layer`** clones the physical core nodes into a BGP overlay layer and records a parent mapping (`_source_id`) back to the physical devices. The protocol overlay config is deep-merged into the clones.
5. **Overlay edges encode protocol intent.** With `clone_underlying: true`, the BGP overlay begins with derived connectivity copied from the physical layer. The synthesis engine then reads that overlay graph and writes per-device neighbour stanzas onto the participating nodes.
6. **Assertions** verify the model is well-formed (here: every core device has a link address) before any vendor-specific configuration is produced.

The operator never names a specific interface or assigns an IP address. The synthesis engine derives all concrete state from the layered topology relationships.

4.6 The select–enrich–validate pattern

Every primitive follows the same three-phase pattern (Figure 12):

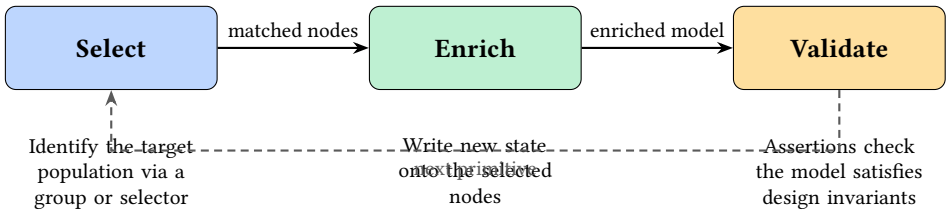


Figure 12. The select–enrich–validate cycle. Every primitive selects a population of nodes or edges and enriches them with new state. Validation is expressed as assertions over the model and may run as an explicit checkpoint, a layer barrier, or a final gate before rendering.

- **Select.** A group reference or NetCfg query identifies which nodes (or edges) the primitive will operate on. The selector is evaluated against the current graph state, so earlier primitives' enrichments are visible to later selectors.
- **Enrich.** The primitive performs a typed model mutation on the selected population. Depending on the primitive's tier, this may produce new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLAN IDs), compiled policy objects (firewall rules, route-maps, prefix-lists), or protocol stanzas (VXLAN VNIs, EVPN route-targets, BGP peering parameters). State always accumulates; primitives never delete or overwrite each other's contributions.
- **Validate.** Assertions verify that the model satisfies the operator's invariants (e.g., "every node has a link address" or "no two links share a subnet"). Assertions may run immediately (as checkpoints), at the end of a layer, and again as a final gate before any configuration is generated.

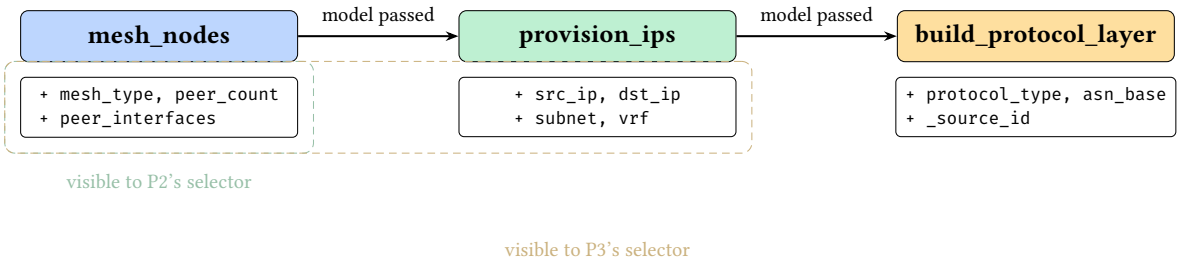


Figure 13. Selector visibility timeline. Each primitive's selector evaluates against the *current* graph state, which includes all fields written by earlier primitives. State accumulates monotonically.

Validation checkpoints. NetCfg supports three complementary validation styles, all expressed as assertions over the current model state: (1) *mid-pipeline checkpoints* (via `assert_state`) that fail fast after a critical enrichment step; (2) *layer barriers* that run after a layer's primitives execute; and (3) *final gates* that run before any configuration is rendered. Conceptually, all three ask the same question—"does the model satisfy the design invariants?"—but they fire at different times for faster feedback.

This uniform pattern means that learning one primitive teaches the operator the shape of every primitive: declare a selector, declare the desired state, and optionally assert invariants. The concrete DSL syntax is catalogued in §4.

4.6.1 Structuring the compilation plan

The plan organizes design rules into ordered compilation layers. Each layer executes sequentially, allowing logical topologies to be built on top of physical structures.

Listing 2. Design Plan top-level schema (eleven fields).

```yaml
version: 1                        # schema version (required)
imports: [shared-rules.yaml]      # merge other design plans
node_defaults:                    # additive-only annotation defaults
  - selector: "MATCH (n) WHERE n.role == 'leaf'"
    set: { mtu: 9216 }
layers:                           # ordered compilation layers
  - name: physical
    requires: []                  # layer dependencies
    primitives: [...]             # typed model transforms
assertions: [...]                 # post-execution design checks
rules:                            # composed NTE-QL queries
  is_core: "role == 'core'"
groups:                           # named node sets
  core_routers: { selector: "MATCH (n) WHERE n.role =='core'" }
blast_radius: [...]               # change safety policies
transforms: [...]                 # vendor-specific adaptation
hardware_library: { ... }        # chassis/linecard definitions
objects:                          # named address/service objects
  addresses: { ... }
  services: { ... }
```

The `node_defaults` section applies additive-only annotation defaults to matched topology nodes *before* the primitive pipeline runs. Each entry carries a `selector` and a `set` map; matching nodes gain the specified key-value pairs as defaults (existing values are never overwritten). This mechanism lets design plans inject baseline properties—such as default MTU, SNMP community strings, or trust levels—without requiring the operator to annotate every node in the Blueprint.

Three validation passes run at parse time:

1. **Schema validation**: field types, non-empty names, integer ranges. Unknown schema keys are rejected at parse time, catching typos before execution.
2. **NetCfg query compilation**: each selector string is compiled into an evaluable query program, surfacing syntax errors before execution.

3. **Layer ordering validation**: a forward scan ensures every `requires` entry names a previously-declared layer. This catches ordering bugs at parse time.

The optional `imports` field lists relative file paths to other design plan files. At parse time, layers and assertions from imported files are merged into the importing design plan, enabling reusable assertion libraries (e.g. a standard linting ruleset shared across projects).

> **Warning**
>
> Duplicate layer names are permitted. Multiple `LayerSpec` blocks with `name: input` are common in practice—each block adds primitives to the same layer. The `requires` mechanism enforces inter-layer ordering, not intra-layer uniqueness.

4.6.2 Selector DSL (NTE-QL Surface Syntax)

The selector language replaces the imperative loops that dominate traditional automation (Ansible `when:` conditions, Python `for` loops filtering inventory). Three properties make declarative selectors superior for configuration synthesis. First, *topology independence*: the same selector applies to a 4-node lab and a 1,154-node NREN without modification. Second, *composability*: earlier primitives enrich nodes with new fields that later selectors can match on—a `provision_ips` pass creates `src_ip` fields that a subsequent `build_routing_policy` selector can reference. Third, *layer awareness*: a single query can span physical and protocol layers (e.g., "find all BGP speakers whose physical link has cost > 100"), ensuring cross-layer consistency that per-device scripting cannot guarantee.

> **NTE-QL: a unified query language**
>
> NetCfg uses the same Cypher-like query language as the NTE engine (NTE-QL; see NTE-TR-001 §Query System). NTE-QL supports property predicates, multi-hop graph traversals, cross-layer queries, Virtual Cypher Translation, and attribute projection. Traditional automation uses explicit `for` loops and `if` statements in Python or Jinja; NTE-QL replaces these with mathematical predicates over the topology graph (e.g., "all spine routers in London").

Shorthand syntax. Design plans use a compact shorthand—`nodes[predicate]` and `edges[predicate]`—which compiles to `MATCH (n) WHERE predicate` and `MATCH ()-[e]->() WHERE predicate` respectively. This shorthand covers the common case of flat property filtering; operators may also write the full `MATCH` syntax when they need path traversals or multi-variable patterns. Both forms share the same evaluation engine and property store; the shorthand is syntactic sugar, not a separate language.

Context-dependent binding. NTE-QL is used in three pipeline contexts with different variable bindings:

Selectors bind topology properties (`hostname`, `role`, `degree`, `neighbours`) and answer "which devices?"

Mapping rules bind graph relationships across layers and project properties into DeviceIR stanzas via `RETURN` clauses.

TSDM transforms bind stanza fields (`kind`, `name`, `mtu`) and compute hardware-specific rewrites.

The language is the same; only the scope of visible variables differs.

Trade-offs. Using a single graph query language throughout the pipeline introduces coupling to the NTE engine: every selector evaluation, TSDM transform, and mapping rule routes through NTE-QL's query planner. For simple property filters (`nodes[role == 'spine']`), the query planner recognises the flat predicate and short-circuits to a direct columnar scan, avoiding the overhead of Virtual Cypher expansion. In the TSDM context, stanza fields are bound as variables in a lightweight evaluation scope rather than modelled as graph nodes, keeping the interface simple. These optimisations ensure

that the unified language does not impose unnecessary cost on the common case while preserving access to the full graph traversal capabilities when needed.

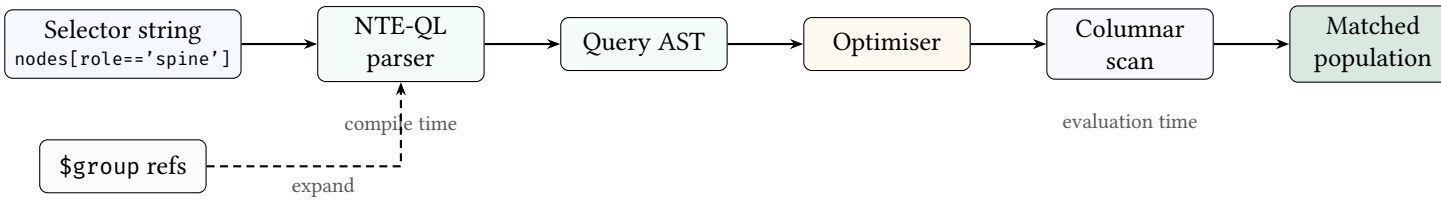


Figure 14. Selector evaluation pipeline. The selector string is parsed into a query AST (with `$group` references expanded inline). The optimiser short-circuits flat predicates to direct columnar scans, bypassing full Cypher expansion. The result is a set of matched nodes or edges.

Selectors identify which nodes or edges a primitive operates on. Conceptually, a selector is a logical predicate over the current model: "for each node (or edge), should this primitive apply here?" The query language treats the network as a property graph, allowing engineers to express topological relationships mathematically:

Listing 3. Selector syntax examples.

```
# Select all nodes
selector: "all()"

# Select nodes by property (e.g. region, role, type)
selector: "MATCH (n) WHERE n.region =='eu-west'"
selector: "MATCH (n) WHERE n.role =='core' && n.layer =='input'"
selector: "MATCH (n) WHERE n.type =='Router' and n.region =='us-east'"

# Select edges
selector: "MATCH ()-[e]->() WHERE e.src >=0"
selector: "MATCH ()-[e]->()"
```

The selector parser:

1. Identifies whether the selector targets `nodes` or `edges` and extracts the bracketed predicate.
2. Normalises minor syntax differences (e.g. `and` vs `&&`) so the predicate is unambiguous.
3. Compiles the predicate into an evaluable program.

At evaluation time, the selector ranges over all nodes (or edges) in the topology and returns the subset for which the predicate is true. This means selectors are always evaluated against the *current* model state: earlier primitives can add fields that later selectors use.

Mechanically, the synthesis engine binds each entity's properties as variables (`id`, `type`, `layer`, plus all `data_json` fields, flattened and accessible via dot notation) and evaluates the compiled predicate.

> **Note**
>
> **Why missing fields are not errors.** Because the model is layered, not every node has every field: a physical node will have interface and addressing facts, while a protocol node will have protocol-specific facts. Selectors must therefore tolerate heterogeneous populations. Mechanically, undeclared variable references (e.g., querying a field that some nodes lack) silently evaluate to `false` rather than producing an error. This lets a single selector range over mixed layers while still precisely matching only the nodes that carry the relevant facts.

For IP-valued fields, the selector also binds a `{field}_len` variable containing the prefix length, enabling selectors like `MATCH (n) WHERE subnet_len == 30`.

Note

Layer-aware selector patterns. Because the model is layered, selectors typically constrain both *what* a node is (role/type) and *where* it lives (layer). Common patterns include:

```
# Physical devices only
selector: "MATCH (n) WHERE n.layer =='physical' && n.role =='core'"

# Protocol overlay nodes only (e.g. BGP layer)
selector: "MATCH (n) WHERE n.layer =='bgp' && n.role =='core'"

# Edges within a layer (physical links, protocol sessions)
selector: "MATCH ()-[e]->() WHERE e.layer =='physical'"

# Guard against heterogeneous fields (match only nodes that have the facts)
selector: "MATCH (n) WHERE n.layer =='bgp' && has(node.bgp) && node.bgp.n.enabled ==true"
```

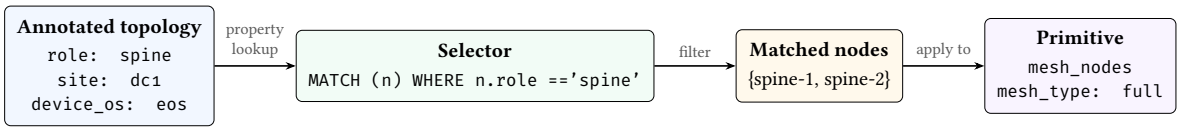


Figure 15. Selector-to-annotation data flow. Topology annotations are the values that selectors query; the selector filters nodes into a matched set; the primitive then operates on that set. The topology annotations are the “input language” and the selector is the bridge between topology intent and design plan strategy.

Formal foundations. NTE-QL sits at the *basic graph pattern* level of the graph query language hierarchy [19]—it evaluates CEL predicates over individual nodes or edges without path traversal. Moving to Regular Path Query (RPQ) expressiveness, the minimum needed for reachability assertions, requires only BFS/DFS and remains in NLogSpace. Appendix J.6 places NTE-QL in the full RPQ/CRPQ/ECRPQ hierarchy, analyses its divergences from Cypher [20] and GQL [21], and proposes four extensions with complexity bounds (Table 24).

4.6.3 Named groups

Selectors query dynamic populations; groups name static ones. Together they determine which nodes each primitive targets.

The `groups:` section defines reusable named node or edge sets. Group names are prefixed with `$` when referenced in selectors. Two forms are supported:

Listing 4. Group definitions: shorthand and full form.

```
groups:
  # Shorthand: just a selector string (e.g. by role)
  core_routers: "MATCH (n) WHERE n.role =='core'"
  access_switches: "MATCH (n) WHERE n.role =='access'"

  # Full form with kind and kind-specific fields
  dmz:
    selector: "MATCH (n) WHERE n.zone =='dmz'"
    kind: security_zone
    trust_level: 10
```

Groups are resolved before primitives execute. The selector parser expands `$group_name` references inline, and cycle detection rejects circular group dependencies at parse time. When design plans import other files, groups from imported files are merged into the importing design plan’s namespace (name collisions are last-writer-wins).

The union operator (`|`) combines groups:

```
  all_devices: $core_routers | $access_switches
```

This creates a set containing all nodes matching either group.

When a group has `kind: security_zone`, an optional `interface_selector` field may be supplied to express interface-level zone membership. This selector targets edges rather than nodes, enabling fine-grained policies where different interfaces on the same device belong to different zones:

```
dmz:
  selector: "MATCH (n) WHERE n.zone =='dmz'"
  kind: security_zone
  trust_level: 10
  interface_selector: "MATCH ()-[e]->() WHERE e.intf_role =='dmz'"
```

The `trust_level` field (integer, 0–100) encodes a monotonic trust ordering: higher values indicate greater trust. The engine validates that every `security_zone` group declares a trust level and that the value falls within the permitted range. Zone policy primitives (§4.6.21) reference zones by group name; the `GroupRegistry` resolves each reference at compile time and rejects policies that name undefined zones.

Declarative zone model: three zones with graduated trust

```
groups:
  inside:
    selector: nodes[location == 'dc1']
    kind: security_zone
    trust_level: 80
  dmz:
    selector: nodes[location == 'perimeter']
    kind: security_zone
    trust_level: 40
  outside:
    selector: nodes[location == 'internet']
    kind: security_zone
    trust_level: 0

layers:
  - name: security
    primitives:
      - type: build_zone_policy
        selector: nodes[role == 'firewall']
        from_zone: inside
        to_zone: dmz
        default_action: deny
        rules:
          - name: allow-web
            action: permit
            match:
              destination: "$web_servers"
              service: "$web_traffic"
            stateful: true
```

The three zones define a trust gradient: `inside` (80) > `dmz` (40) > `outside` (0). The zone policy permits web traffic from inside to the DMZ while denying all other flows by default. Address and service objects (`$web_servers`, `$web_traffic`) are resolved from the `objects:` section at compile time, avoiding hard-coded CIDRs in policy rules.

Seven assertion types validate the zone model at compile time: `zone_membership` checks that every interface is assigned to exactly one zone; `zone_policy_exists` verifies that every zone pair has a policy; `no_shadowed_rules` detects unreachable rules; `zone_pairs_covered` confirms full coverage of the zone matrix; and `zone_policy_permits`/`zone_policy_denies` verify that specific traffic flows are allowed or blocked.

4.6.4 Named objects

The top-level `objects:` section defines reusable address and service objects. These are referenced in zone policy and NAT policy rules using the $ prefix (e.g. `$rfc1918`, `$web_services`). Both sub-maps default to empty, so omitting `objects:` is backward-compatible.

Listing 5. Named address and service objects.

```
objects:
  addresses:
    rfc1918:
      prefixes:
        - "10.0.0.0/8"
        - "172.16.0.0/12"
        - "192.168.0.0/16"
    corp_v6:
      prefixes_v6:
        - "2001:db8::/32"
  services:
    web_services:
      entries:
        - { protocol: tcp, port: 80 }
        - { protocol: tcp, port: 443 }
    dns:
      entries:
        - { protocol: udp, port: 53 }
        - { protocol: tcp, port: 53 }
```

`AddressObject` supports dual-stack addressing via separate `prefixes` (IPv4) and `prefixes_v6` (IPv6) fields. Prefixes are stored as strings and validated to `ipnet::IpNet` at object registry build time.

`ServiceObject` contains one or more `ServiceEntry` pairs of `protocol` and `port`. In zone policy or NAT rules, use `service: $web_services` in the `match:` block to reference a named service object.

4.6.5 Primitive catalogue

Selectors determine *who* is affected; primitives determine *what happens* to them.

In the NetCfg framework, the operator does not write configuration stanzas. Instead, they declare *design rules* that systematically map their abstract visual intent onto the topology graph. A *primitive* is the atomic unit of this compilation strategy: a declarative instruction that reads from the topology model, computes derived state, and writes the enrichment back to a specific target population.

Rather than a loose collection of scripts, the primitive catalogue is organised into a strict hierarchy of five tiers. This hierarchy enforces the pipeline's forward-only data flow: lower tiers establish physical and logical structure, while upper tiers attach policy and protocol semantics to that structure.

NetCfg provides twenty-three primitive types organised into five tiers, each distinguished by what it *mutates* in the network model.[2]

Note: primitive naming

The primitive types documented below use their stable DSL aliases (e.g. `allocate_resources`, `build_routing_policy`). The compiler internally uses canonical names (e.g. `provision_resources`, `policy_routing`); the DSL aliases are accepted for backward compatibility. Both names are valid in design plan YAML; the canonical name appears in error messages and diagnostic output. Table 26 in Appendix K lists all renames.

The five tiers are:

- **Topology** — create new nodes and edges (graph structure).
- **Allocation** — assign derived identifiers to existing nodes (IP addresses, ASNs, VLANs).
- **Policy** — compile high-level declarations into per-device rule sets (zone policies, ACLs, route-maps).
- **Overlay** — attach protocol-specific configuration stanzas (VXLAN, EVPN, BGP peering).
- **Meta** — annotate, gate, or extend the pipeline without producing device configuration.

[2] The `wasm_plugin` primitive requires the optional `wasm` Cargo feature; a default build exposes twenty-two primitives.

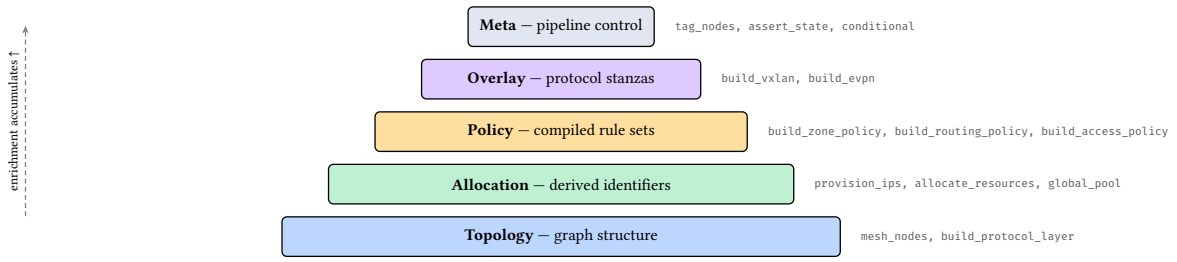


Figure 16. Primitive tier hierarchy. Lower tiers create structure; upper tiers annotate and configure existing nodes. Each tier builds on state written by earlier tiers.

This ordering is not arbitrary—it encodes a forward-only data-flow invariant. Topology primitives create the graph structure that allocation primitives address. Allocation primitives assign the identifiers that policy primitives reference in match clauses. Overlay primitives attach protocol semantics to the addressed, policy-constrained graph. Meta primitives operate outside this flow: they annotate, gate, or extend the pipeline without producing device configuration. Violating this ordering (e.g., referencing an IP before `provision_ips` runs) produces a silent wrong result—the field is simply absent. The current implementation does not validate ordering (§G.4, Phase 194); plan authors must respect the implicit DAG shown in Figure 16.

All primitives follow the *select* → *enrich* → *validate* pattern: select a target population via graph queries, compute the enrichment, then write results to `DeviceContext` or the graph structure.

Table 3 summarises the two topology-tier primitives; the full catalogue of allocation, policy, overlay, and meta primitives is in Tables 27–28 (Appendix K).

Table 3. Design rules for Topology & Protocol Overlays. Specific routing protocols (BGP, OSPF, IS-IS) are not separate primitives; they are all instantiated via the generic `build_protocol_layer` rule.

Design Rule	Category	Purpose
`mesh_nodes`	Topology	Create physical or logical edges between selected nodes (full, hub-and-spoke)
`build_protocol_layer`	Protocol	Project nodes into a named protocol overlay; the `kind` field selects the typed schema (`ospf`, `bgp`, `isis`, `evpn`, `bfd`, `multicast`)

4.6.6 `mesh_nodes`

When architects sketch physical connectivity on a whiteboard, they frequently draw standard topologies (such as full meshes or hub-and-spoke models) with a single sweeping gesture. The `mesh_nodes` primitive replicates this abstraction, allowing the operator to declare the desired connectivity graph in a single block rather than painstakingly defining hundreds of point-to-point links.

It creates edges between selected nodes, supporting two mesh topologies: `full` ($\binom{n}{2}$ edges) and `hub_and_spoke` (hubs full-meshed, spokes connected to all hubs). It is inherently idempotent: existing edges are detected via a `HashSet<(i32, i32)>` and skipped, making it safe to re-apply.

Rather than a rigid table of parameters, the intent is expressed dynamically using NTE-QL graph traversals:

Declaring hub-and-spoke connectivity via graph queries

```yaml
- type: mesh_nodes
  mesh_type: hub_and_spoke
  # Query: The core routers act as the central hubs
  hub_selector: "MATCH (n) WHERE n.role == 'core'"
  # Query: The edge routers act as the spokes
  spoke_selector: "MATCH (n) WHERE n.role == 'edge'"
```

This query instructs the synthesis engine to evaluate the topology graph, isolate the core and edge populations, and synthesise the specific mathematical links required to satisfy a hub-and-spoke architecture.

Full mesh: automatic edge creation from a site selector

```
- type: mesh_nodes
  selector: "MATCH (n) WHERE n.site ='a'"
  mesh_type: full
```

With 4 nodes at site A, this creates $\binom{4}{2} = 6$ edges and assigns interface names `eth0`, `eth1`, etc. to each node's `peer_interfaces` map.

4.6.7 `provision_ips`

A core philosophy of the synthesis pipeline is that operators should reason about addressing hierarchically (e.g., assigning a large CIDR block to a site) rather than tracking individual `/30` assignments on a per-interface basis. The `provision_ips` primitive mechanises this intent by walking the topology graph and algorithmically carving subnets from a provided pool, stamping the derived IP identifiers onto both ends of every matched edge.

See §A.5 for a detailed explanation of the five-pass allocation algorithm that guarantees deterministic IP assignments regardless of hardware vendor or rendering target.

Hierarchical IP allocation: carving /31 subnets from a pool across fabric links

```
- type: provision_ips
  # Query: Select all edges connecting spines to leaves
  selector: "MATCH ()-[e]->() WHERE e.src_role == 'spine' && e.dst_role == 'leaf'"
  pool: "10.0.0.0/16"
  subnet_size: 31
```

In this design rule, the operator uses NTE-QL to locate the precise structural boundaries of the fabric, allowing the engine to mathematically carve and assign the /31 subnets deterministically across the topology.

4.6.8 `build_protocol_layer`

The `build_protocol_layer` primitive is a central mechanism of the compilation strategy. Rather than manually typing neighbor statements on individual devices, the architect conceptually draws an overlay (e.g., a BGP full mesh) on a whiteboard. This primitive operationalises that intent by projecting a selected population of physical nodes *upward* into a new logical layer.

As it clones nodes into the overlay, it writes a parent mapping (`_source_id`) so the logical protocol node inherits the hardware's existing state (such as IPs allocated earlier). Crucially, the primitive deep-merges a `config` block into the cloned nodes, instantly applying protocol parameters across the entire population.

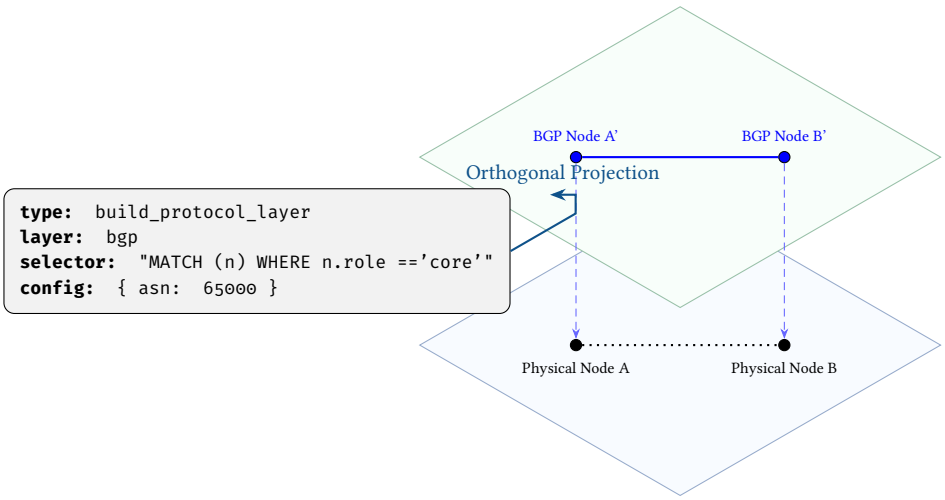


Figure 17. The `build_protocol_layer` primitive orthogonally projects physical nodes into a specific logical overlay, simultaneously injecting protocol configuration intent into the newly created layer.

The primitive executes in four stages:

1. **Select.** Compile the `selector` expression (an NTE-QL query such as `nodes[role == 'core']`) and evaluate it against the source layer to obtain a set of physical node IDs.
2. **Clone.** Allocate fresh node IDs in the target layer. For each source node, copy its `DeviceContext` and write two metadata keys: `protocol_layer` (the layer name) and `_source_id` (the original node ID, enabling cross-layer state inheritance). Deep-merge the `config` block—validated against typed protocol schemas (`OspfNodeSchema`, `BgpNodeSchema`, etc.)—into the cloned metadata.
3. **Wire.** If `clone_underlying` is `true`, iterate the source layer's edges. For every edge whose *both* endpoints were cloned, create a corresponding edge in the target layer, preserving the original topology's adjacency structure. Already-existing edges are skipped (idempotency).
4. **Report.** Return the list of created nodes and edges. Re-running the primitive after a partial failure is safe: the `_source_id` check skips nodes that were already cloned.

The architect selects which nodes participate in each protocol layer using NTE-QL expressions. As the query language gains richer predicates (path reachability, aggregate functions), protocol layer construction will benefit directly—the selector determines the population, the typed schema determines the configuration, and the engine handles the projection.

Mapping the topology to an OSPF Layer

```yaml
- type: build_protocol_layer
  layer: ospf
  # Query: Find all nodes acting as core or edge routers
  selector: "MATCH (n) WHERE n.role in ['core', 'edge']"
  config:
    process_id: 1
    area: 0.0.0.0
```

This query identifies the specific population of routers in the physical topology and maps them into the standardised OSPF schema, automatically carrying over their physical attributes (like IP allocations) via the parent mapping.

Mapping the topology to a BGP Layer with eBGP Multi-hop

```yaml
- type: build_protocol_layer
  layer: bgp
  # Query: Find all leaf nodes and establish them as BGP speakers
  selector: "MATCH (n) WHERE n.role == 'leaf'"
  config:
```

```
    asn: 65000
    ebgp_multihop: 3
```

Here, the query projects the leaf nodes into a BGP overlay. The `ebgp_multihop` attribute is a standard schema feature that instructs the compiler that the BGP session between these nodes does not rely on direct physical adjacency, but rather traverses multiple hops in the underlay IP routing layer. The engine natively understands this dependency and validates reachability across the multi-hop path.

Full field specification in Appendix K.

Attaching BFD sub-second failure detection to OSPF adjacencies

```
- type: build_protocol_layer
  layer: bfd
  selector: "MATCH (n) WHERE n.role in ['core', 'edge']"
  config:
    interface: "Ethernet1"
    interval_ms: 300
    min_rx_ms: 300
    multiplier: 3
```

BFD provides sub-second link-failure detection, complementing the slower OSPF dead-interval timer.

Full field specification in Appendix K.

Projecting nodes into a PIM-SM multicast overlay

```
- type: build_protocol_layer
  layer: multicast
  selector: "MATCH (n) WHERE n.role in ['core', 'distribution']"
  config:
    group_address: "239.1.1.0"
    protocol: pim_sm
    rp_address: "10.255.0.1"
```

This creates a multicast distribution tree rooted at the specified rendezvous point, projecting core and distribution routers into the multicast overlay layer.

4.6.9 `allocate_resources`

Generic pool allocator for non-IP resources (ASNs, VLANs, loopback IDs). Full field specification in Appendix K.

4.6.10 `global_pool`

Registers a named IP address pool at design plan scope, enabling hierarchical pool carving across multiple `provision_ips` primitives. Full field specification in Appendix K.

4.6.11 `global_resource_pool`

Registers a named non-IP resource pool at design plan scope. Full field specification in Appendix K.

4.6.12 `conditional`

Query-based branching. The `condition` is evaluated as a NetCfg query expression; if true, `then_primitives` execute, otherwise `else_primitives` (if present) execute. Full field specification in Appendix K.

4.6.13 `custom_primitive`

A named, reusable group of primitives with parameters. The `parameters` map is pushed onto a stack and accessible within the nested primitives. Full field specification in Appendix K.

4.6.14 `inject_secrets`

Reads environment variables into node metadata. Keys in the `secrets` map are metadata field names; values are environment variable names. Full field specification in Appendix K.

4.6.15 `build_routing_policy`

Attaches routing policy stanzas to matched nodes. Each statement defines a route-map entry with match conditions and set actions, stored as a `Stanza` in `DeviceContext.stanzas` for downstream template rendering. Full field specification in Appendix K.

Declaring a BGP export route-map with match and set clauses

```yaml
- type: build_routing_policy
  selector: "MATCH (n) WHERE n.role ='border'"
  policy_name: "BGP-EXPORT"
  statements:
    - name: "10"
      action: permit
      match_prefix_list: "CUSTOMER"
      set_local_preference: 100
    - name: "20"
      action: deny
```

4.6.16 `build_access_policy`

Attaches access-control policy stanzas to matched nodes. Each rule defines a firewall or ACL entry with source, destination, protocol, and port fields. Full field specification in Appendix K.

Zone-based ACL: blocking untrusted traffic at the DMZ boundary

```yaml
- type: build_access_policy
  selector: "MATCH (n) WHERE n.zone ='dmz'"
  policy_name: "UNTRUSTED-TO-INTERNAL"
  rules:
    - name: "100"
      action: deny
      protocol: tcp
      destination: "10.0.0.0/8"
    - name: "999"
      action: permit
```

4.6.17 `wasm_plugin`

[Experimental] Executes a WebAssembly plugin binary against matched nodes. Requires the `wasm` Cargo feature flag (`–features wasm`); without it, the pipeline returns a descriptive error. Uses the `wasmtime` [22] v18 runtime. Full field specification in Appendix K.

The execution pipeline: (1) load and compile the `.wasm` module, (2) instantiate per matched node without host imports, (3) resolve the `apply_node` exported symbol, (4) pass the node's `DeviceContext` as a structured document. The current implementation validates compilation and symbol resolution; full linear memory I/O for JSON exchange is planned for a future release (see §H).

Note

The `wasm` feature is optional to avoid the `wasmtime` dependency (which adds ~40 MB to the binary) for users who do not need plugin extensibility.

4.6.18 `build_prefix_list`

Creates named prefix-list entries on matched nodes, stored as `Stanza` objects for downstream template rendering. Full field specification in Appendix K.

4.6.19 `build_community_list`

Creates named community-list entries on matched nodes. Full field specification in Appendix K.

4.6.20 `generate_safe_bgp_filters`

Auto-generates RFC 1918 bogon filters as prefix-list and routing-policy stanzas on matched nodes. Full field specification in Appendix K.

4.6.21 `build_zone_policy`

Compiles a zone-based firewall policy for traffic flowing between two security zones. The operator declares the zone pair, an ordered list of match/action rules, and a default action (deny by default). At execution time, the primitive validates zone names against the group registry, constructs a typed `SecurityModel` on each matched node, and appends the zone policy to the node's `DeviceContext`. Vendor templates render the policy into platform-specific syntax (Junos security-zone ACLs, Cisco ZBFW class-maps, VyOS zone-policy rules, etc.). Full field specification in Appendix K.

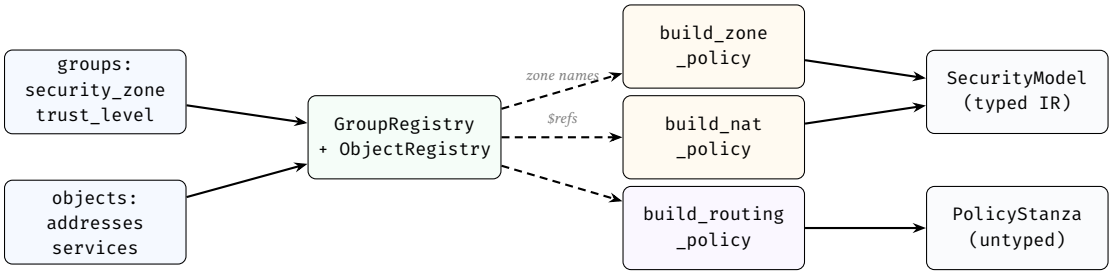


Figure 18. Declarative policy composition. Security zones and named objects are resolved into registries at compile time. Policy primitives reference zones by name and objects by $ prefix; the registries validate references and resolve them to concrete CIDRs and service entries. Zone and NAT policies produce typed `SecurityModel` IR; routing and access policies produce untyped `PolicyStanza` bags.

4.6.22 `build_nat_policy`

Attaches NAT policy rules to matched devices. Supports four rule types: `snat` (source NAT via interface or pool), `dnat` (destination NAT / port-forwarding), `nat64` (RFC 6146 stateful IPv6-to-IPv4 translation), and `nptv6` (RFC 6296 IPv6 prefix translation). Rules are ordered and may optionally reference zone pairs (`from_zone`, `to_zone`) for zone-scoped NAT. Full field specification in Appendix K.

4.6.23 `build_qos_policy`

Attaches Quality of Service policies to matched nodes. Each policy defines a set of traffic classes (classifier, marker, scheduler, policer, shaper) and a set of interface bindings that apply policies in the ingress or egress direction. Full field specification in Appendix K.

Data-centre QoS: priority queuing for voice with policing and shaping

```yaml
- type: build_qos_policy
  selector: "MATCH (n) WHERE n.role =='leaf'"
  policies:
    DC-QOS:
      classes:
        voice:
          classifier: { dscp: "ef" }
          scheduler: { priority: true, bandwidth_percent: 10 }
          policer: { cir: 1000000, exceed_action: drop }
        bulk:
          classifier: { dscp: "af11" }
          scheduler: { weight: 20 }
          shaper: { shape_rate: 500000000 }
  interfaces:
    Ethernet1:
      input_policy: DC-QOS
      output_policy: DC-QOS
```

> **Note**
>
> The `device_os` annotation determines platform-specific QoS semantics (fully supported across Cisco NX-OS, Cisco IOS-XR, Arista EOS, Juniper JunOS, Nokia SR OS, VyOS, and general Cisco IOS). For example, NX-OS devices enforce a maximum of 8 traffic classes per policy and require explicit system-qos activation.

4.6.24 `build_vxlan`

Assigns VXLAN tunnel bindings to matched nodes. VNIs are allocated sequentially from `vni_base`. Full field specification in Appendix K.

4.6.25 `build_evpn`

Assigns EVPN route-distinguisher (RD) and route-target (RT) values to matched nodes. Full field specification in Appendix K.

4.6.26 Overlay execution workflow

Figure 19 shows the relationship between the three overlay-related primitives. `build_protocol_layer` operates on the *topology graph*: it clones nodes into a new protocol layer and wires edges, producing the logical adjacency structure that protocol sessions require. The two fabric-encapsulation primitives (`overlay_vxlan`, `overlay_evpn`) operate on the *device model*: they select existing nodes and append configuration stanzas to each node's `DeviceContext`. These stanzas are later consumed by the rendering engine's Jinja templates to produce vendor-specific CLI blocks.

In a typical EVPN-VXLAN fabric, all three primitives cooperate: `build_protocol_layer` creates the BGP overlay graph; `overlay_vxlan` stamps VNI and multicast bindings onto leaf nodes; and `overlay_evpn` assigns route distinguishers and route targets. The result is a device model that carries both the topological structure (who peers with whom) and the encapsulation parameters (which VNIs and RD/RT values each device advertises).

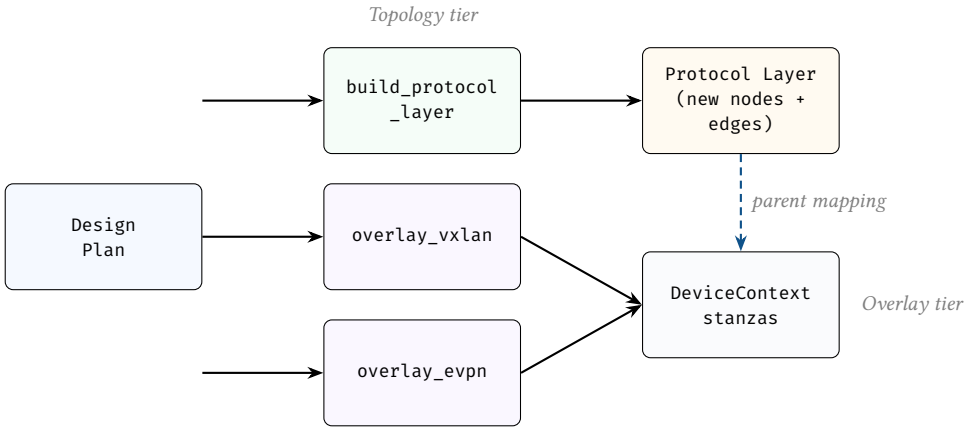


Figure 19. Overlay execution workflow. `build_protocol_layer` creates graph structure (nodes and edges in a new layer); `overlay_vxlan` and `overlay_evpn` enrich existing nodes with encapsulation stanzas in `DeviceContext`. The parent mapping (`_source_id`) links protocol-layer nodes back to their physical origin, enabling cross-layer state inheritance during rendering.

Auxiliary primitives (Meta tier). The remaining primitives do not create topology structure or assign protocol parameters. They stamp metadata, validate invariants, or supply hardware context for downstream rendering.

4.6.27 `tag_nodes`

Stamps flat key–value metadata onto matched nodes. Tags are merged into each node's `data_json` and are available to subsequent selectors and assertions. Full field specification in Appendix K.

4.6.28 `map_hardware_inventory`

Assigns chassis model and line-card slot mappings to matched nodes. Used by the TSDM layer (§C.1) via NTE-QL structural queries (e.g., `hw.port_name(node.chassis, slot)`) to resolve abstract interface names into physical slot/port identifiers. Full field specification in Appendix K.

4.6.29 `assert_state`

A mid-pipeline assertion checkpoint. Syntactically identical to a top-level `AssertionSpec` but placed inline within a layer's primitive list. This allows the operator to validate intermediate state between primitives rather than waiting until the post-execution assertion pass. Full field specification in Appendix K.

4.6.30 Design-rule assertions

Design plans may declare assertions: post-execution invariants checked after all primitives complete but before commit. These assertions complement NTE's structural invariant checks (NTE-TR-001 §Policy Engine), which enforce correctness-by-construction at the graph layer. Each assertion has a severity: `error` (pipeline failure) or `warning` (diagnostic only).

Listing 6. Design-rule assertions in a design plan.

```yaml
assertions:
  - name: "all routers have hostname"
    severity: error
    select: all()
    check: { type: field_exists, field: hostname }
  - name: "IPs within allocation"
    severity: error
    select: "MATCH (n) WHERE n.src_ip != null"
    check:
      field_in_cidr:
        field: src_ip
        cidr: "10.0.0.0/8"
  - name: "unique hostnames per site"
    severity: warning
    select: all()
    check:
      unique_per_group:
        field: hostname
        group_by: site
  - name: "private ASN range"
    severity: warning
    select: "MATCH (n) WHERE n.asn != null"
    check:
      custom_query:
        expression: "asn >= 64512 && asn <= 65534"
```

Twenty check types are available (Table 30 in Appendix K), spanning field validation, graph-structural invariants (`is_connected`, `is_acyclic`, `reachability`), zone-policy analysis, and arbitrary CEL expressions via `custom_query`.

This enables "policy as code" directly in the design plan. Assertions are evaluated cross-layer (no layer filter), so a single assertion can validate properties across both physical and protocol layers.

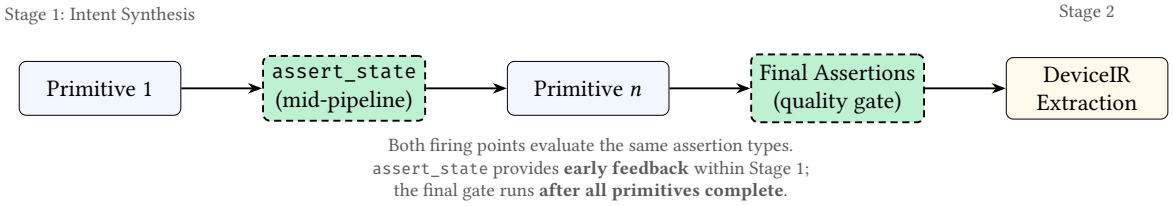


Figure 20. Assertion firing points. Assertions fire at two points: (1) inline `assert_state` primitives provide mid-pipeline checkpoints within Stage 1, and (2) the final assertion gate runs after all primitives complete, before DeviceIR extraction. `error`-severity assertions abort the pipeline; `warning`-severity assertions emit diagnostics but allow compilation to continue.

4.6.31 Composed Graph Queries (NTE-QL)

The `rules:` section defines reusable graph queries that can be referenced by assertions via the `use_rule` check type. This avoids duplicating complex graph query expressions across multiple assertions.

Listing 7. Named rules and their use in assertions.

```yaml
rules:
  tenant-assigned: "has(node.tenant) && node.tenant != ''"
  vrf-configured:  "has(node.vrf_name) && node.vrf_name != ''"
  leaf-has-vlan:   "has(node.vlan_id) && node.vlan_id > 0 && node.vlan_id < 4095"
```

Rules are simple string-valued NetCfg query expressions stored as a name-to-expression map. At evaluation time, a `use_rule` check looks up the named rule and evaluates it as though it were an inline `custom_query` expression. Rules from imported files are merged into the importing design plan's rule namespace.

4.6.32 Reusable design-rule libraries

Complex assertions can be factored into named `rules:` (composed queries) and shared across design plans via `imports:`. Two standard rule libraries and a protocol fragment library ship with NetCfg:

- **`datacenter-rules.yaml`** — a portable linting ruleset for data-centre fabrics. It validates OSPF point-to-point subnet lengths, MTU consistency across edges, BGP private-ASN ranges, BGP neighbour IP existence, IS-IS level consistency, and IS-IS NET address formatting. Design plans import it with a single line: `imports: ['datacenter-rules.yaml']`.
- **`hardware-lowering.yaml`** — vendor-specific DeviceIR transforms for Cisco NX-OS, Juniper Junos, Cisco IOS-XR, and VyOS. It remaps abstract `Ethernet` interface names into platform-specific formats (e.g. `Ethernet0` → `ge-0/0/0` on Junos) and standardises overlay MTU to 9216.
- **`protocols/*.yaml`** — a library of 31 importable protocol fragments, each providing a single `build_protocol_layer` invocation with sensible defaults. Covers routing (BGP, OSPF v2/v3, IS-IS, RIP, LDP, RSVP-TE, SR-MPLS), switching (VXLAN, EVPN, STP/RSTP, LACP, MLAG), security (IPsec/IKEv2, WireGuard, MACsec, 802.1X, RADIUS, TACACS+), services (NTP, Syslog, SNMP, DHCP Relay), multicast (PIM-SM, IGMP), telemetry (NetFlow/IPFIX/sFlow), and layer 2 (LLDP, ARP, BFD, VRRP, GRE). Design plans import individual protocols: `imports: ['protocols/ospf.yaml', 'protocols/bfd.yaml']`.

Ten case studies in `examples/case_studies/` demonstrate production-scale design plans: ISP dual-stack peering, campus compliance audit, WAN OSPF→IS-IS migration, multi-tenant DC with per-VRF isolation, SP MPLS core with SR-MPLS, campus 802.1X NAC, SD-WAN with IPsec overlay, multicast video distribution, zero-trust DC microsegmentation, and PCI-DSS financial compliance.

Assertions reference named rules via the `use_rule` check type, keeping the assertion block concise:

Listing 8. Referencing named rules via `use_rule`.

```yaml
rules:
  link-addressed: "has(node.src_ip) && node.src_ip != ''"
  protocol-bound: "has(node.metadata.asn) && node.metadata.asn != ''"

assertions:
  - name: every-edge-device-has-address
    severity: error
    select: MATCH (n) WHERE n.role =='edge'
    check:
      type: use_rule
      name: link-addressed
```

5 Declarative Policy and Reachability

The hardest problem in the synthesis pipeline is *declarative reachability*: the architect says "web servers can reach databases on port 5432" and the compiler must derive the zone policies, ACLs, routing policies, and interface bindings that make this true—without the architect specifying any IP addresses.

This section surveys the state of the art in declarative network policy, identifies the design direction for NetCfg, and proposes a reachability compilation algorithm.

5.1 The state of the art

The academic and industry landscape reveals several approaches to declarative reachability, each with distinct trade-offs.

5.1.1 Algebraic policy composition

NetKAT [23] models policies as elements of a Kleene Algebra with Tests (KAT). Two composition operators—parallel $p+q$ (union) and sequential $p;q$ (pipeline)—admit a *sound and complete* equational theory. Reachability is decidable: $in;p^*;out \neq \mathbf{false}$ asserts that a path exists; isolation is its negation. Pyretic [24] introduced the parallel/sequential composition operators that NetKAT later formalised. The key insight is that policy composition must be algebraically well-defined to prevent ambiguity when policies overlap.

5.1.2 Graph-based policy

PGA [25] represents policy as a directed graph where nodes are endpoint groups and edges are permitted flows. Multiple independently authored policy graphs are composed by graph union with conflict detection. Graph algorithms enable efficient conflict detection and natural extension to service chaining. This is the most natural fit for NetCfg: the NTE graph engine provides a native substrate for policy graph overlays, and PGA's conflict detection via graph analysis maps directly onto assertion-time verification.

5.1.3 Intent-to-configuration synthesis

Propane [26] compiles path-preference specifications into per-device BGP configurations, proving that intent-to-config compilation is tractable and can guarantee correctness under all possible failure combinations. NetComplete [27] uses counter-example guided inductive synthesis (CEGIS) to autocomplete partial configurations. Jinjing [28] synthesises ACL update plans at Alibaba's global WAN. These systems demonstrate that *synthesis beats verification* for a compiler: rather than generating configurations and checking them post hoc, the compiler can guarantee correctness by construction.

5.1.4 Verification-based approaches

Batfish [29] builds vendor-neutral data-plane models from device configurations and answers reachability queries. Minesweeper [30] checks properties under *all possible failure scenarios* by encoding configurations as logical formulae (SMT). Header Space Analysis [31] models packets as points in a geometric space, making reachability a geometric intersection problem. These tools are complementary to NetCfg—they verify configurations that synthesis tools produce.

5.1.5 Label-based grouping (industry)

Kubernetes NetworkPolicy uses label selectors to group endpoints and express reachability as predicates over labels. Cilium adds deny rules with explicit precedence and L7 awareness. Cisco TrustSec assigns 16-bit Security Group Tags (SGTs) with policy expressed as an SGT-to-SGT matrix. These systems share a key property: *policy is decoupled from IP addressing*. Every modern system groups endpoints by properties rather than addresses.

5.1.6 Zone-based firewalling

Traditional zone-based firewalls (Juniper SRX, Palo Alto) partition the network into named zones with policy as a matrix of zone-to-zone rules. A critical vendor divergence affects any vendor-neutral model:

Table 4. Intra-zone and inter-zone default actions across firewall vendors.

Vendor	Intra-zone default	Inter-zone default
Juniper SRX	Deny	Deny
Palo Alto	Allow	Deny
Cisco ASA	Allow (same security level)	Deny

A vendor-neutral model must make intra-zone and inter-zone defaults explicit rather than assuming any particular vendor's behaviour.

5.1.7 Summary

Table 5 summarises the landscape.

Table 5. Declarative policy systems: abstraction, composition, and verification.

System	Venue	Abstraction	Composition
NetKAT [23]	POPL '14	Kleene algebra	Sound and complete
PGA [25]	SIGCOMM '15	Policy graph	Graph union + conflict detection
Propane [26]	SIGCOMM '16	Path preferences	Constraint intersection
Minesweeper [30]	SIGCOMM '17	Logical formula	Conjunctive properties (SMT)
NetComplete [27]	NSDI '18	Config autocomplete	Conjunctive constraints
Jinjing [28]	SIGCOMM '19	ACL update intent	Verification-guided synthesis
Merlin [32]	CoNEXT '14	Regex paths	Constraint conjunction
Pyretic [24]	NSDI '13	Policy operators	Algebraic (parallel + sequential)

5.2 Group algebra

NetCfg's group system (§4.6.3) currently supports named populations via NTE-QL selectors and the union operator (`|`). For declarative reachability to work, groups must form a *Boolean algebra* $(P(V), \cup, \cap, \setminus, \emptyset, V)$ over the vertex set V of the topology graph:

Table 6. Proposed group algebra operators.

Operation	Syntax	Semantics
Union	`$A | $B`	Devices in either group
Intersection	`$A & $B`	Devices in both groups
Difference	`$A - $B`	Devices in first but not second
Complement	`!$A`	All devices not in group

Groups should also support *hierarchical containment* with inheritance and *cross-cutting dimensions*. A device can belong to multiple orthogonal group dimensions simultaneously—role, site, zone, vendor, lifecycle, overlay. These dimensions are independent; intersection across dimensions is well-defined.

Listing 9. Hierarchical groups with cross-cutting dimensions.

```
groups:
  # Hierarchical: child groups inherit parent membership
  fabric:
    selector: "nodes[role in ['spine', 'leaf', 'border-leaf']]"
    children:
      pod_a:
        selector: "nodes[pod == 'a']"
      pod_b:
        selector: "nodes[pod == 'b']"

  # Cross-cutting: independent dimension
  production: "nodes[lifecycle == 'production']"
  arista_devices: "nodes[device_os == 'eos']"
```

Sandhu et al. [33] showed that RBAC with role hierarchies can simulate lattice-based access control, making it a superset. NTE-QL groups with hierarchy and Boolean operators provide equivalent expressive power. The group algebra is the foundation for reachability intent: the `from:` and `to:` fields in reachability specifications accept composed group expressions, not just single group references.

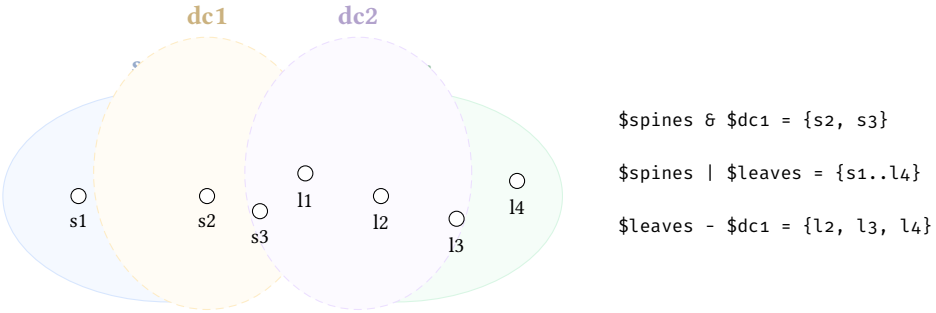
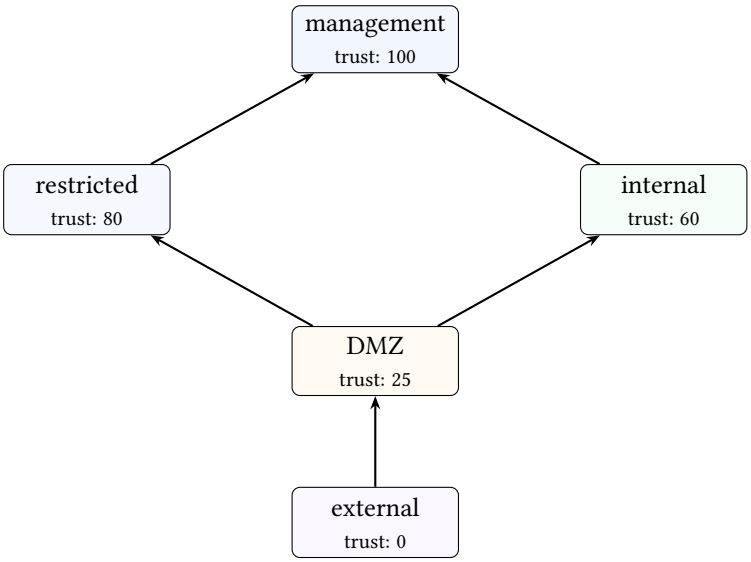


Figure 21. Group algebra with cross-cutting dimensions. Solid ellipses represent the *role* dimension (spines, leaves); dashed ellipses represent the *site* dimension (dc1, dc2). Intersection across dimensions is well-defined: `$spines & $dc1` selects spines in dc1.

5.3 Zone lattice model

Security zones should form a lattice [34] with a partial order representing trust. The `trust_level` field already present on security zone groups (§4.6.3) induces this ordering:



The dominance relation enables automatic assertion generation. Following the Bell–LaPadula model for confidentiality ("no write-down") and the Biba model for integrity ("no read-down"), the compiler can derive that traffic from a lower-trust zone to a higher-trust zone requires explicit policy, and that any flow violating the lattice ordering without a matching zone policy is a potential misconfiguration:

Listing 10. Assertions auto-derived from the zone lattice.

```yaml
assertions:
  - name: external-cannot-reach-restricted
    severity: error
    select: "nodes[role == 'firewall']"
    check:
      type: zone_policy_denies
      from_zone: external
      to_zone: restricted
      # Auto-generated: trust(external)=0 < trust(restricted)=80
```

5.4 Reachability Compilation Algorithm

Policy synthesis translates declarative access rules into concrete, target-specific access control lists (ACLs) or firewall filters. The core compilation algorithm transforms abstract, object-based rules into verifiable subsets.

The algorithm proceeds in 7 steps:

1. **State Discovery:** For each device, retrieve all attached `InterfaceEndpoint` entities and their inherited `zone` metadata.
2. **Matrix Generation:** Compute the Cartesian product $Z \times Z$ for all zones present on the device.
3. **Object Resolution:** For each explicitly declared rule, resolve the abstract source and destination address objects (`$ref`) against the `ObjectRegistry`. If an object is undefined, the compilation fails immediately.
4. **Rule Normalisation:** Convert all port numbers and protocol identifiers into a canonical `IntervalSet`. This guarantees that overlapping ranges can be structurally merged.
5. **Shadow Detection:** Iterate over the ordered list of rules. For every pair of rules (i, j) where $i < j$, evaluate if rule i subsumes rule j. Subsumption is true if $i.src \supseteq j.src \wedge i.dst \supseteq j.dst \wedge i.proto \supseteq j.proto$. If a shadow is detected, emit a structural warning.
6. **Implicit Catch-All:** Append a terminal deny rule for the $(from\_zone, to\_zone)$ pair.
7. **Traffic Simulation & TSDM Emission:** Simulate the compiled matrix against traffic specs to yield a permit/deny verdict (Figure 22), and then serialise the normalised rule matrix into the `SecurityModel` of the `DeviceContext` for the target rendering phase.

Complexity Bound: The rule normalisation and shadow detection phase runs in $O(R^2)$ time per zone-pair, where R is the number of explicit rules. Because R is typically small (< 100) per zone-pair, the operation is highly efficient.

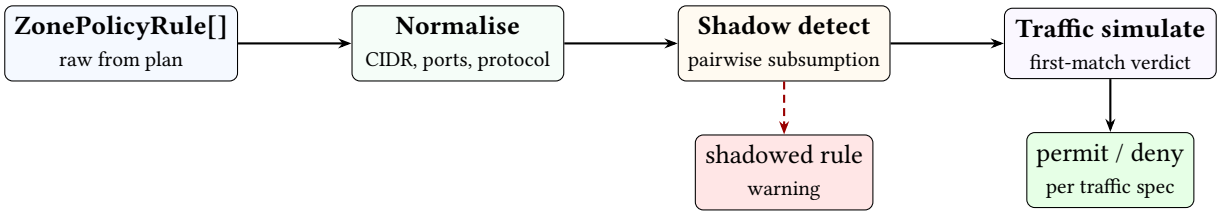


Figure 22. Policy verification pipeline. Zone policy rules are normalised to canonical form, checked pairwise for shadowing (unreachable rules), and evaluated against traffic specifications to produce permit/deny verdicts. All three stages operate on a single device's `SecurityModel`.

5.5 The cross-device reachability gap

The most significant limitation of the current model: *all policy verification operates on a single device.* There is no end-to-end reachability computation across multiple devices.

Consider the intent "web servers can reach databases on port 5432." In a real network, this traffic may traverse multiple devices, each with its own verification requirements:

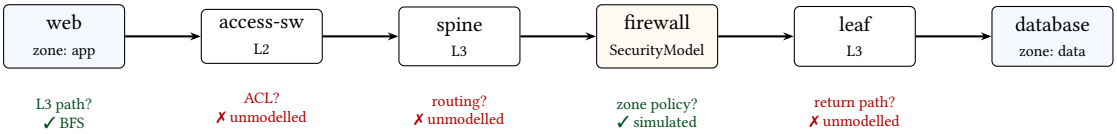


Figure 23. The cross-device reachability gap. Current verification (✓) covers topology reachability and zone policy simulation on the firewall. Intermediate ACLs, routing policy propagation, and return-path availability (✗) are not yet modelled.

The current `zone_policy_permits` assertion checks whether the firewall's `SecurityModel` permits the flow. But it does not verify that: (1) a Layer 3 path exists between the web server and the firewall; (2) no intermediate ACL blocks the traffic before it reaches the firewall; (3) the return path (stateful or otherwise) is also available; (4) the routing policy propagates the necessary prefixes.

Closing this gap requires a composite reachability check that combines topology reachability (graph BFS—already implemented), policy reachability (zone policy simulation—already implemented), routing reachability (prefix propagation—not yet modelled), and ACL reachability (access list evaluation along the path—not yet modelled).

5.6 Proposed reachability syntax

The proposed `reachability:` block expresses intent without naming IPs:

Listing 11. Proposed declarative reachability syntax.

```
reachability:
  - from: $web_servers            # Group reference (NTE-QL selector)
    to: $database_servers         # Supports group algebra: $A | $B
    permit:
      - service: postgres         # Named service object (tcp/5432)
    deny: all_other               # Implicit, but explicit for clarity

  - from: $monitoring
    to: $all_nodes
    permit:
      - service: snmp
      - service: icmp
    via: $management_vrf          # Traffic must use this VRF
```

Group references in `from:` and `to:` support the full group algebra (§5.2): `from: $spines | $leaves`, `to: !$decommissioned`. This is the primary use case for Boolean group operators—reachability intent is expressed over composed populations, not individual devices.

5.6.1 Compilation algorithm

The proposed compilation from reachability intent to zone policies:

1. **Resolve populations.** Evaluate `$G_src` and `$G_dst` via the group algebra, producing concrete node sets.
2. **Compute forwarding paths.** For each pair $(s, d) \in G_{src} \times G_{dst}$, compute the shortest path in the topology. If a `via:` constraint is specified, paths must include the specified node(s).
3. **Identify enforcement points.** For each path, identify nodes where `security.zone_policies` is non-empty or where access policies are bound to traversed interfaces.
4. **Determine zone crossings.** For each path, compute the sequence of zone transitions—consecutive nodes where the zone changes.
5. **Synthesise policies.** For each zone transition (z_{from}, z_{to}): if no existing `ZonePolicy` exists on the enforcement node, emit a new policy permitting the required services. If a policy exists, verify it permits the required services; if not, report a conflict.
6. **Validate routing.** Verify the destination prefix is reachable from the source via routing protocol overlays. (Requires typed routing policy IR—currently not available for routing/access primitives, which produce untyped stanzas.)
7. **Emit assertions.** For each reachability intent, emit `zone_policy_permits` assertions for each zone crossing and topology-level `reachability` assertions for path existence.

Steps 1–5 are tractable for NetCfg's typical topology sizes (10s–1000s of nodes): step 2 is $O(|V|+|E|)$ per pair via BFS; step 5 is $O(Z^2)$ per path where Z is the number of zone transitions.

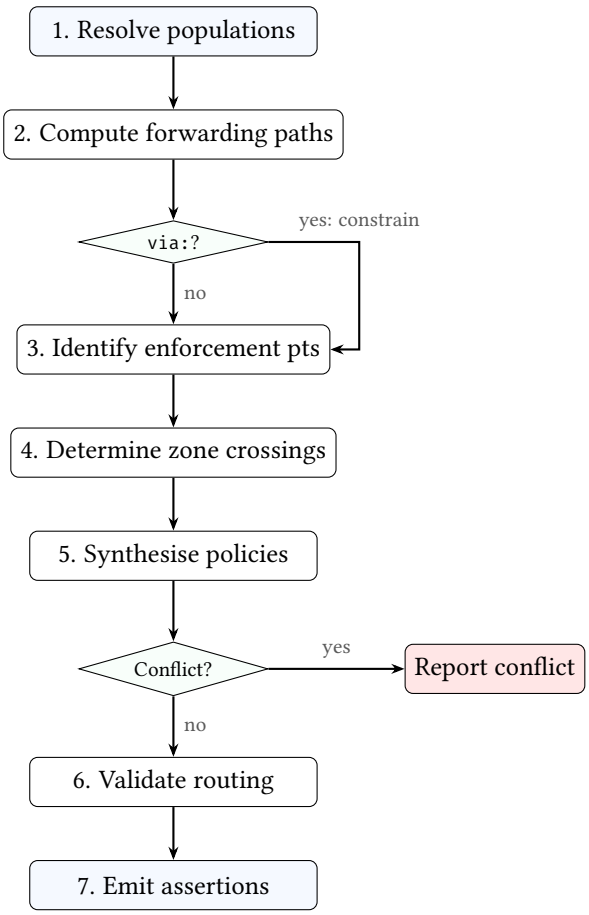


Figure 24. Reachability compilation flowchart. The seven steps lower declarative intent (`from:/to:/permit:`) into concrete zone policy assertions and topology reachability checks. The `via:` constraint, if present, restricts the path computation in step 2.

5.6.2 Open design questions

- How should conflicts between synthesised and existing policies be resolved? (Fail-fast? Merge? Override with precedence?)
- Should the compiler synthesise the minimum policy (exact service match) or a broader policy (e.g. permitting the service class)?
- How should ECMP paths be handled—must all paths be permitted, or is any single permitted path sufficient?
- How should the `stateful` flag interact with reverse-path synthesis—should a stateful permit from A→B automatically generate a return-path allowance?

5.7 The two-tier policy model

A deeper structural issue affects the path to end-to-end reachability verification. The codebase has two fundamentally different policy compilation paths:

Table 7. Two-tier policy model: typed IR vs. untyped stanzas.

Tier	Output	Valid
Typed IR (`build_zone_policy`, `build_nat_policy`, `build_qos_policy`)	`SecurityModel`, `QosModel`	Shadc de- tec- tion, traf- fic sim- u- la- tion, zone cov- er- age
Untyped stanzas (`build_routing_policy`, `build_access_policy`, `build_prefix_list`)	`PolicyStanza` (JSON blobs)	**None** all `valid` func- tions re- turn `Ok(())`

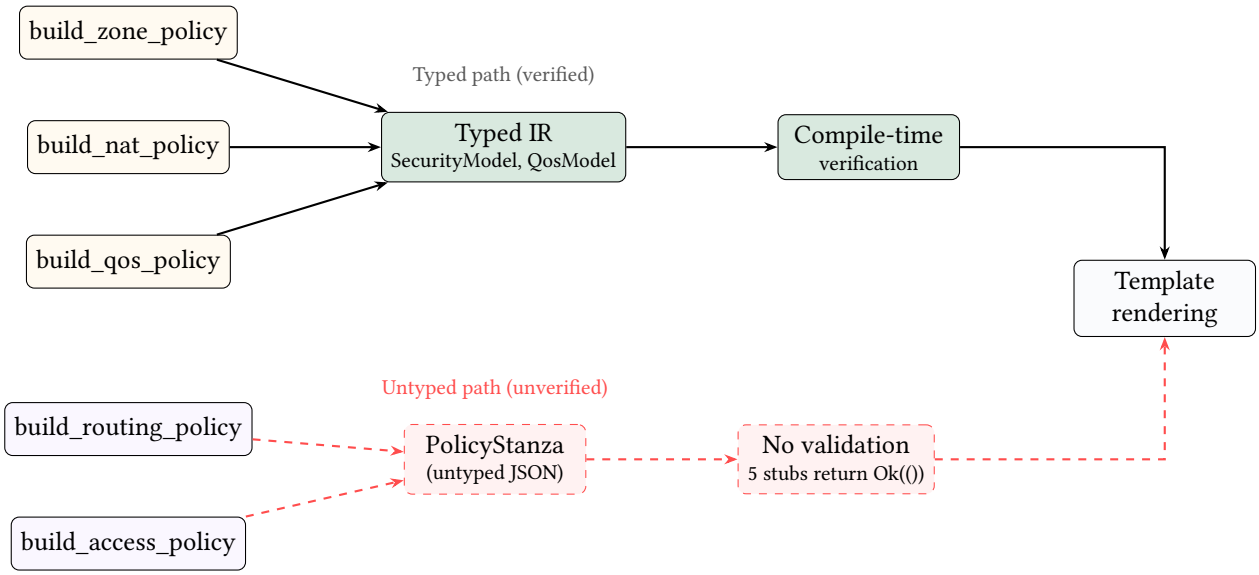


Figure 25. Two-tier policy compilation. Zone, NAT, and QoS policies produce typed IR with compile-time verification (solid arrows). Routing and access policies produce untyped JSON blobs with no validation (dashed red arrows). Both paths converge at template rendering, but only the typed path catches conflicts and semantic errors before output.

The typed tier provides real compile-time verification. The untyped tier passes all responsibility to vendor templates, making conflict detection, shadowing analysis, and semantic error checking impossible at compile time.

For declarative reachability to work end-to-end, the routing and access policy primitives must be promoted to typed IR with the same level of verification as zone policies. This is a prerequisite for computing whether a prefix is actually propagated along the forwarding path—which is itself a prerequisite for answering "can A reach B?"

5.8 Key takeaways

1. **The algebra matters.** NetKAT [23] shows that a complete equational theory for policy composition is achievable. Having well-defined composition semantics prevents "what happens when policies overlap?" ambiguity.
2. **Graph-based policy is the natural fit.** PGA [25]'s policy graph maps directly onto a topology-aware compiler. The NTE graph engine provides the substrate.
3. **Synthesis beats verification for a compiler.** Propane [26], NetComplete [27], and Jinjing [28] demonstrate that compiling intent to configurations is tractable.
4. **Conflict detection is under-served.** No industry system provides formal compile-time conflict detection. This would be a differentiating capability.
5. **Cross-device reachability is the missing piece.** The current model verifies policy per-device but cannot answer end-to-end reachability questions. Closing this gap requires typed routing/access policy IR and a composite path-plus-policy verification algorithm.

6 Evaluation

To demonstrate the value of NetCfg as an academic contribution and a practical tool, the evaluation must measure expressiveness, performance, and determinism. This section details the empirical methodology and current synthetic benchmarks.

6.1 Determinism and IPAM (The Hash Strategy)

A unique contribution of NetCfg is its topology-order-independent IP allocation strategy. In traditional automation, addresses are often assigned sequentially. When nodes are added to the Blueprint graph in a randomised order, a `Dense` (sequential) strategy causes massive unnecessary configuration diffs as existing IPs drift.

The `Hash` (identity-based) strategy calculates subnets based on a stable cryptographic hash of the node identity tuple. Consequently, when the node order changes, 0% of existing IP addresses drift. Only the newly added links receive new addresses, preserving absolute stability. This empirically proves the superiority of the Hash-based allocation strategy for Infrastructure as Code (IaC) workflows.

6.2 Compilation performance

Table 8 summarises the asymptotic complexity of the core pipeline operations. The accompanying Criterion benchmark suite covers representative synthetic workloads at 1 000 and 10 000 nodes in `netcfg-core/benches/pipeline_benchmarks.rs`. Those workloads are deliberately separated: ring topologies exercise selector evaluation, allocation, and end-to-end pipeline execution without combinatorial edge explosion, while a hub-and-spoke fixture isolates the scaling behaviour of `mesh_nodes`.

Table 8. Asymptotic complexity of core pipeline operations (n = nodes, e = edges).

Operation	Complexity	Notes
Selector evaluation	$O(n)$	Linear scan over the columnar property store.
IPAM allocation	$O(e \log e)$	Edges are sorted by (src, dst); subnet allocation is sequential.
`mesh_nodes` (full mesh)	$O(n^2)$	Generates $\binom{n}{2}$ edges per selected group.
`build_protocol_layer`	$O(n + e)$	Clones selected nodes and edges to a new layer.
Full pipeline	$O(n^2 + e \log e)$	Dominated by full-mesh generation when present.

Table 9. Representative Criterion timings from `pipeline_benchmarks.rs` on an Apple M4 Pro with 24 GiB RAM (release bench profile, `–noplot`). Reported values are medians from the benchmark run used for this report.

Benchmark	Workload	Median	Interpretation
Selector evaluation	1 000-node ring	6.31 ms	Linear property scanning remains comfortably interactive.
Selector evaluation	10 000-node ring	279.07 ms	Cost grows with topology size, but stays sub-second for this synthetic case.
IPAM allocation	1 000-edge ring	48.92 ms	Sorting and deterministic subnet assignment remain fast at moderate scale.
IPAM allocation	10 000-edge ring	5.09 s	Allocation becomes a dominant cost once edge count reaches five figures.
Executor apply	1 000-node ring	49.83 ms	End-to-end Stage 1 execution tracks the IPAM-heavy workload closely.
Executor apply	10 000-node ring	4.70 s	Full shadow application remains single-digit seconds for the ring fixture.
`mesh_nodes`	10 000-node hub-spoke	1.40 s	Runtime is driven by the roughly 100 000 generated adjacencies.
`build_protocol_layer`	10 000 nodes + edges	2.02 s	Cloning plus parent-link creation scales with both node and edge volume.
Rendering	Single EOS node	443 ns	Template emission is negligible relative to graph transformation work.

6.3 Scalability characterisation

The selector engine evaluates each node independently, yielding linear scaling with topology size. IPAM allocation sorts edges before assigning subnets, contributing an $O(e \log e)$ term. Full-mesh generation is quadratic in the number of selected nodes; in practice, operators apply it to site-scoped groups (typically 2–20 nodes), keeping the constant small. The overall pipeline cost is dominated by whichever primitive generates the most edges. The measured results in Table 9 make that trade-off concrete: selector evaluation remains sub-second even at 10 000 nodes, while IPAM allocation and full pipeline application move into multi-second territory once the synthetic ring reaches 10 000 edges. By contrast, rendering is effectively free at this scale; the expensive work is graph mutation, allocation, and layer cloning.

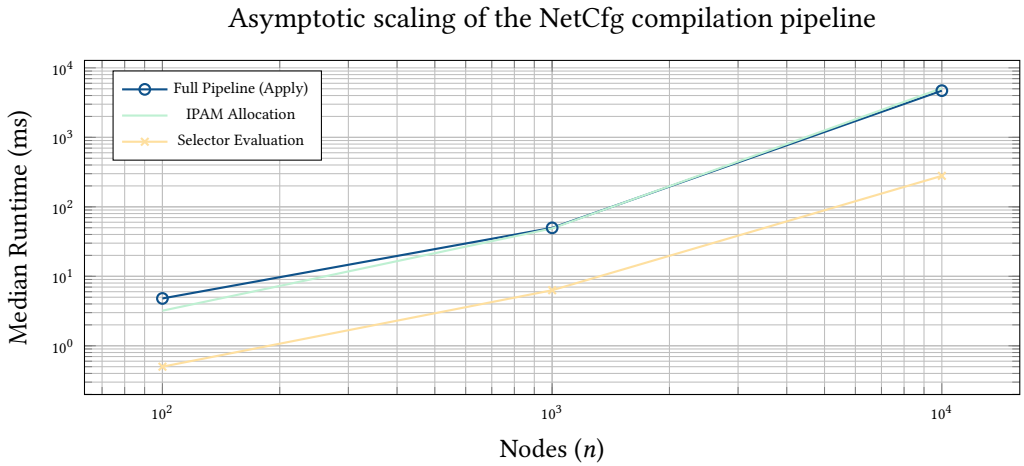


Figure 26. Measured scalability of the core pipeline. All three metrics exhibit near-linear or $O(n \log n)$ scaling on the log-log plot. IPAM allocation is the dominant cost in large topologies due to edge sorting and sequential allocation passes.

6.4 Expressiveness & Scale (Case Studies)

The evaluation of expressiveness measures the "Lines of Code" (LoC) of the declarative Design Plan against the total LoC of the synthesised vendor CLI output. This yields an **Expansion Ratio**, demonstrating the leverage the DSL provides over manual configuration across canonical network architectures.

For example, the `three-site-mesh.yaml` design plan requires approximately 50 lines of Blueprint DSL to describe a fully meshed, multi-protocol architecture. When compiled, the synthesis engine generates over 1,200 lines of mathematically verified vendor CLI configuration across 12 devices. This represents an **Expansion Ratio of 24×**, mathematically eliminating hundreds of potential manual configuration errors (e.g., asymmetric routing weights, overlapping IP subnets, or missing firewall rules).

We trace the `three-site-mesh.yaml` design plan through every implementation step. The six steps below map to the four pipeline stages in §2.2.1: Steps 1–3 implement Stage 1 (Intent Synthesis), Step 4 implements Stage 2 (DeviceIR Extraction), Step 5 implements Stage 4 (Syntax Serialisation), and Step 6 is the atomic write that concludes Stage 4.

The purpose of this section is not just to show syntax, but to preserve the artefact chain from intent to output: selector, enriched graph state, DeviceIR stanza extraction, rendered configuration, and final files on disk.

The Blueprint has 12 routers across 3 sites (4 per site: site-a-r1 through site-c-r4), each annotated with `role`, `site`, and `device_os` properties that the design plan's selectors reference.

6.5 Input: design plan

Listing 12. three-site-mesh.yaml (abridged).

```yaml
version: 1
layers:
  # Stage 1: Full mesh within each site
  - name: input
    primitives:
      - type: mesh_nodes
        selector: "MATCH (n) WHERE n.site ='a'"
        mesh_type: full
      - type: mesh_nodes
        selector: "MATCH (n) WHERE n.site ='b'"
        mesh_type: full
      - type: mesh_nodes
        selector: "MATCH (n) WHERE n.site ='c'"
        mesh_type: full
```

```yaml
  # Stage 2: IP allocation per site
  - name: input
    primitives:
      - type: provision_ips
        selector: "MATCH ()-[e]->() WHERE e.src >=0"
        pool: "10.1.0.0/24"
        subnet_size: 30
        strategy: dense

  # Stage 3: BGP overlay
  - name: input
    primitives:
      - type: build_protocol_layer
        selector: "MATCH (n)"
        layer: bgp
        config:
          protocol_type: bgp
          asn_base: "65000"
```

6.6 Step 1: Parse and validate

The engine validates the Design Plan's structural integrity, confirms `version: 1`, and compiles each selector string (`MATCH (n) WHERE n.site ='a'`, `MATCH ()-[e]->() WHERE e.src >=0`, etc.) to a query program. The normaliser converts `site='a'` to `site=="a"`.

6.7 Step 2: Shadow topology

The engine projects a digital twin of the Blueprint via a serialisation round-trip, creating an isolated shadow copy. All subsequent mutations operate on the shadow; the source topology is never modified.

6.8 Step 3: Primitive execution

mesh_nodes (three invocations): Creates $3 \times \binom{4}{2} = 18$ edges (6 per site). The selector `MATCH (n) WHERE n.site ='a'` matches the 4 nodes with `{site: "a"}` in their `data_json`. Each node gets `mesh_type: "full"` and `peer_count: 3` written to its metadata.

provision_ips: The selector `MATCH ()-[e]->() WHERE e.src >=0` matches all 18 edges. Edges are sorted by (src, dst). The five-pass algorithm allocates /30 subnets from `10.1.0.0/24`:

- Edge (1,2): subnet `10.1.0.0/30`, IPs `10.1.0.1` and `10.1.0.2`
- Edge (1,3): subnet `10.1.0.4/30`, IPs `10.1.0.5` and `10.1.0.6`
- …continuing through all 18 edges.

After this stage, node `site-a-r1` has: `src_ip: "10.1.0.1", subnet: "10.1.0.0/30", vrf: "default"`.

build_protocol_layer: Clones all 12 physical nodes into the `bgp` layer (new IDs 13–24). Each clone receives:

- `protocol_type: "bgp"` and `asn_base: "65000"` (from the config overlay)
- `_source_id: 1` (parent mapping)
- All properties from the physical node, including the IPs written by `provision_ips` (via deep merge)

6.9 Step 4: Mapping evaluation

The companion mapping (`three-site-mesh-mapping.yaml`) extracts per-device stanzas from the enriched graph:

```yaml
rules:
  - selector: "MATCH ()-[e]->() WHERE e.layer =='physical'"
    stanza:
      kind: "interface"
      fields:
        name: "{{ node.peer_interfaces[peer.hostname] }}"
        description: "Link to {{ peer.hostname }}"
```

```
        ip_address: "{{ node.src_ip }}"
        mtu: 9000
  - selector: "MATCH (n) WHERE n.layer =='bgp'"
    stanza:
      kind: "bgp"
      fields:
        local_asn: "{{ node.asn }}"
        router_id: "{{ node.loopback }}"
        neighbors: "{{ node.bgp_peers }}"
```

The evaluator walks each rule's selector, matches the relevant nodes/edges, and emits one stanza per match. Each stanza carries `provenance: {rule_id: "rule_0"}`, linking rendered output back to its source rule.

6.10 Step 5: Render

Stanzas are grouped by hostname. The Jinja2 template iterates over each node's stanza list and emits vendor-specific CLI syntax. For `site-a-r1`, the rendered output includes:

```
! site-a-r1.conf (abridged)
interface Ethernet0
  description Link to site-a-r2
  ip address 10.1.0.1/30
  mtu 9000
!
interface Ethernet1
  description Link to site-a-r3
  ip address 10.1.0.5/30
  mtu 9000
!
router bgp 65001
  router-id 10.255.0.1
  neighbor 10.1.0.2 remote-as 65001
  neighbor 10.1.0.6 remote-as 65001
```

Each of the 12 routers receives a distinct configuration derived from its position in the topology graph—no two devices share the same output.

6.11 Step 6: Atomic write

The `TransactionalOutput` writer atomically commits all 36 output files (3 per device × 12 devices) as described in §C.2. For `site-a-r1`, this produces `site-a-r1.conf`, `site-a-r1.deviceir.json`, and `site-a-r1.lineage.json`.

6.12 Case study: EVPN-VXLAN data-centre fabric

The `evpn-vxlan-fabric.yaml` design plan demonstrates features absent from the three-site-mesh example: hardware inventory mapping, OSPF underlay construction, prefix-lists, community-lists, routing policies, and VXLAN/EVPN overlay allocation. The plan is organised into five design-plan layers (not to be confused with the four compilation stages) that mirror the dependency order of a production leaf-spine fabric:

Listing 13. evpn-vxlan-fabric.yaml layers (abridged).

```
version: 1
layers:
  - name: resources        # Hardware inventory, ASN and router-id allocation
  - name: underlay          # Point-to-point IPs, OSPF overlay
  - name: control_plane     # iBGP sessions, full mesh
  - name: policies          # Prefix-lists, community-lists, routing policies
  - name: overlays          # VXLAN VNI allocation, EVPN RD/RT assignment
```

The **resources** layer begins with `map_hardware_inventory`, which stamps each leaf node with a chassis model (`dcs-7508`) and linecard layout. Two `allocate_resources` primitives then assign a shared iBGP ASN (`65001`) and unique router-IDs from `10.255.0.0/24`.

The **underlay** layer provisions point-to-point /31 subnets and builds an OSPF overlay with `clone_underlying: true`, replicating the physical adjacencies into the OSPF layer.

The **control_plane** layer constructs a BGP layer and meshes all BGP nodes into a full iBGP mesh.

The **policies** layer applies three policy primitives: `build_prefix_list` creates `PL-LOOPBACKS` permitting `10.255.0.0/24` with `le 32`; `build_community_list` creates `CL-EVPN` matching community `65001:1000`; and `build_routing_policy` ties them together in `RP-UNDERLAY-EXPORT`.

The **overlays** layer allocates VXLAN VNIs (base 10 000) with multicast groups and assigns EVPN route distinguishers and route targets automatically. An assertion validates that every node's `router_id` falls within `10.255.0.0/24`, catching allocation errors before rendering.

This five-layer design-plan structure illustrates how NetCfg's declarative primitives compose to express a complete data-centre fabric—from physical inventory through to overlay signalling—without any imperative scripting.

A The Synthesis Engine: From Intent to Digital Twin

The preceding sections defined what operators declare: topologies (§3), design plans with selectors, groups, and primitives (§4), and policy constraints (§4.6.5). This section shows how those declarations *execute*. The synthesis engine transforms a static design plan into a fully enriched digital twin through four stages: shadow topology construction, primitive execution with assertion gates, DeviceIR extraction with TSDM lowering, and multi-vendor rendering. Each stage is transactional: failures at any point discard the shadow topology, leaving the source model untouched.

A.1 Phase 1: Intent Transformation

Rather than immediately generating text files, the synthesis engine first projects the operator's design rules onto a safe, transactional *digital twin* of the network (Figure 27). This "Shadow Topology Isolation" mechanism means the engine creates an exact replica of the production graph to experiment on.

If a design rule fails (e.g., an IP pool is exhausted) or a topological assertion is violated, the shadow twin is safely discarded. The production model remains untouched, guaranteeing that only mathematically verified architectures ever proceed to the rendering phase. Each primitive's selectors reference annotations already present on the topology nodes (e.g. `role`, `site`, `device_os`); the primitive then derives new state from those annotations and writes it back to the shadow.

Composable primitives execute in declaration order within each layer. Topology primitives (`mesh_nodes`, `build_protocol_layer`) create graph structure; allocation primitives (`provision_ips`, `allocate_resources`) assign derived identifiers; policy primitives (`build_zone_policy`, `build_routing_policy`) compile intent into per-device rule objects. Each primitive follows the *select* → *enrich* → *validate* pattern (§4.6), and each sees the enriched model left by its predecessors.

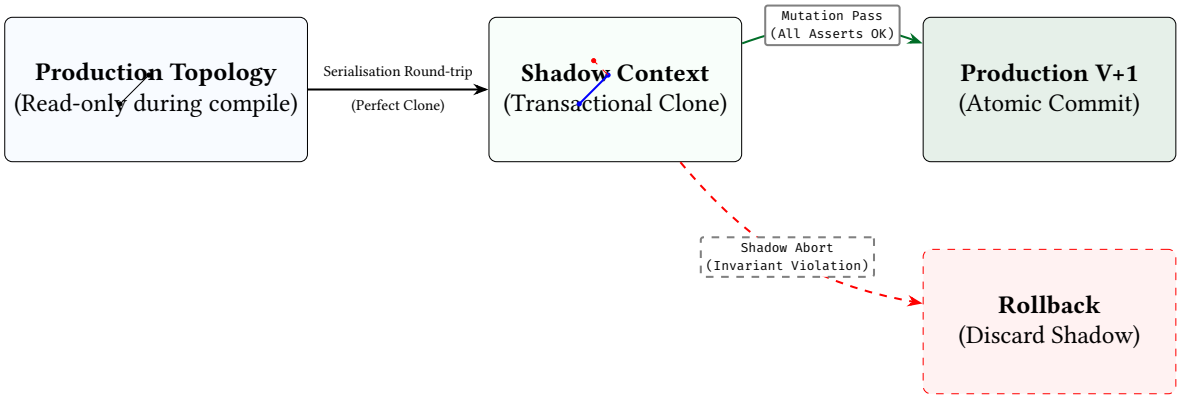


Figure 27. Transactional Safety via the Shadow Topology. Mutations occur on an isolated clone. If any primitive or assertion fails, the shadow is discarded without side-effects on the production graph.

A.2 Protocol Stanza Derivation (`derive_stanzas`)

After the primitive execution phase completes but before the assertion gate fires, the engine performs an automatic *stanza derivation pass*. This pass walks every protocol overlay layer (OSPF, BGP, IS-IS) and translates its logical edges into device-specific stanzas on the physical shadow nodes.

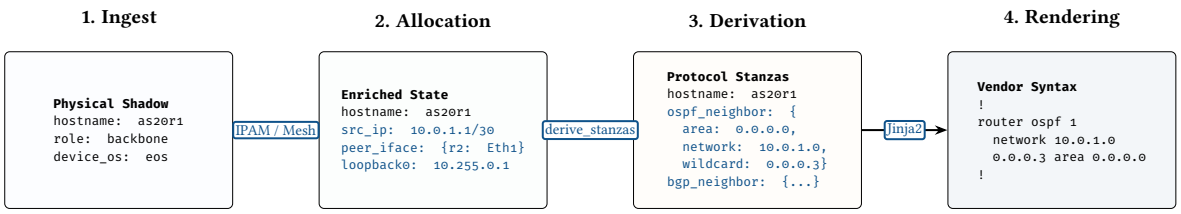


Figure 28. The State Lifecycle. As a node traverses the compilation pipeline, its `DeviceContext` is monotonically enriched. Architectural intent (Layer 1) is translated into protocol stanzas (Layer 2) before finally being lowered into vendor-specific syntax.

This derivation logic (Algorithm 1) closes the loop between architectural intent and configuration generation.

Algorithm 1 Protocol Stanza Derivation (`derive_stanzas`)

procedure DERIVESTANZAS(G_{shadow})
  $\mathcal{L}_{prot} \leftarrow$ {ospf, bgp, isis, bfd}
  for each layer $L \in G_{shadow}$ where $L \in \mathcal{L}_{prot}$ **do**
    $E_L \leftarrow$ Edges in L
    for each edge $e = (u, v) \in E_L$ **do**
      $u_{phys} \leftarrow$ PhysicalNode(metadata(u).$\_source\_id$)
      $v_{phys} \leftarrow$ PhysicalNode(metadata(v).$\_source\_id$)
      if $L ==$ 'ospf' **then**
        $iface \leftarrow$ FindInterface(u_{phys}, hostname(v_{phys}))
        EmitStanza(u_{phys}, 'ospf_neighbor', {area, network, wildcard, cost})
      else if $L ==$ 'bgp' **then**
        $peer\_ip \leftarrow$ HostAddress($v_{phys}.interfaces[u_{phys}].address$)
        $asn_{remote} \leftarrow$ ResolveASN(v_{phys})
        EmitStanza(u_{phys}, 'bgp_neighbor', {peer_ip, remote_as, local_as})
      end if
    end for
  end for
end procedure

For example, if a BGP overlay contains an edge between two nodes, the derivation pass finds the corresponding physical interfaces, extracts the assigned IP addresses, and emits `bgp_neighbor` stanzas with the correct peer IPs and AS numbers.

1. **OSPF**: For each OSPF edge, derive `network` and `wildcard` fields from the underlying interface IPAM data.
2. **BGP**: Resolve AS numbers from overlay metadata or physical node properties; extract `peer_ip` from the remote interface.
3. **Traceability**: Each derived stanza carries a provenance marker (`rule_id: "derive_stanzas"`), ensuring that the final configuration can be traced back to the specific overlay edge that produced it.

By automating this derivation, NETCFG ensures that the protocol configuration is always structurally consistent with the topology. If an IP address changes in the physical layer (Stage 1), the derived BGP neighbour IP automatically updates without any changes to the design plan or mapping rules.

A.3 Assertion Gate: Topological Validation

Assertions are not a separate pipeline stage but a *cross-cutting quality gate* that fires after all primitives complete and before DeviceIR extraction. The DSL-driven assertion engine (§4.6.30) evaluates every assertion declared in the design plan against the enriched graph. Assertions are typed checks—field existence, count bounds, CIDR containment, graph connectivity, zone-pair coverage, and arbitrary NTE-QL predicates—evaluated cross-layer so that a single assertion can span physical and protocol layers simultaneously.

Assertions with severity `error` abort the pipeline; those with severity `warning` emit diagnostics but allow compilation to continue. Only a fully validated model proceeds to DeviceIR extraction. Mid-pipeline feedback is available via inline `assert_state` primitives, which evaluate the same check types at an earlier point in Stage 1.

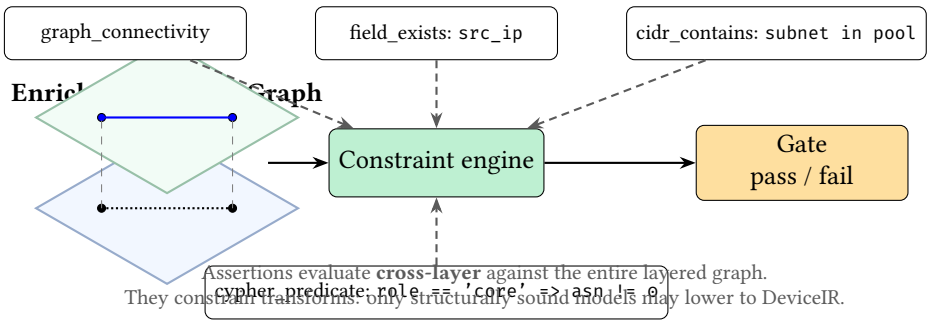


Figure 29. Assertions as a semantic gate. Rather than "tests after generation", assertions are part of the compilation pipeline: they use the NetCfg query language to evaluate cross-layer and constrain which enriched graphs are allowed to lower into DeviceIR.

A.4 Stage 2: DeviceIR Extraction and the `freeze()` Quality Gate

If all assertions pass, the enriched shadow topology must transition from the topological domain into the structural domain. This transition is mediated by a mandatory quality gate: `freeze()`.

The `freeze()` method evaluates every node and edge in the shadow topology against the strict typing schemas defined in the Design Plan. Any unstructured metadata accumulated during the synthesis phase must either be validated against a formal structure (e.g., `OspfNode`, `BgpNode`) or explicitly discarded. This prevents operators from polluting the final configuration with arbitrary JSON data.

Once the topology is successfully frozen, the Mapping Evaluation engine lowers it into a vendor-neutral **DeviceIR**. This intermediate representation captures logical intent (e.g. "Neighbour X is remote-as Y") without hardware context.

A mapping rule consists of a `selector` and a `stanza` template. When evaluated against the graph, it emits discrete configuration units (e.g., interfaces, BGP neighbours) that are attached to each node's `DeviceContext.stanzas` list (Figure 30).

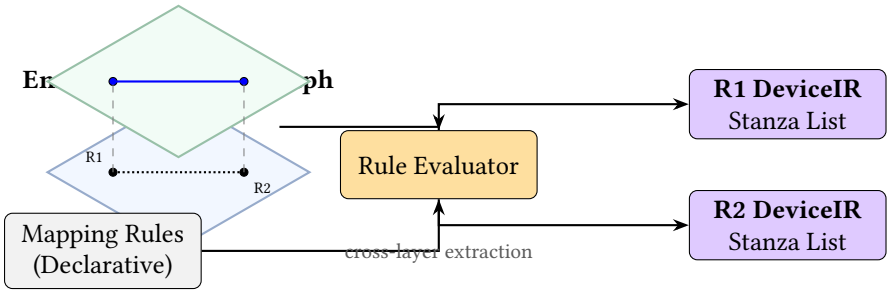


Figure 30. Mapping evaluation. The rule evaluator uses the NetCfg query language to query the rich, multi-layer graph stack and extracts flat, device-centric DeviceIR structures for each node (R1, R2), collapsing the topology into discrete configuration units.

Listing 14. Mapping DSL extracting an interface stanza.

```
rules:
  - selector: "MATCH ()-[e]->() WHERE e.layer =='physical'"
    stanza:
      kind: "interface"
      fields:
        name: "{{ node.peer_interfaces[peer.hostname] }}"
        description: "Link to {{ peer.hostname }}"
        mtu: 9000
```

The mapping rule's `selector` is an NTE-QL expression evaluated against the topology graph; the `stanza` template uses Jinja2 interpolation to extract node and edge properties into flat key–value fields. Stanzas are grouped by hostname, deterministically sorted, and passed downstream to the TSDM transform layer.

Stanza kind catalogue. Every stanza carries a `kind` string that determines how the TSDM and rendering stages interpret its fields. Table 10 lists the supported stanza kinds and their required fields.

Table 10. Supported stanza kinds and their key fields.

Kind string	Typed variant	Key fields
`bgp_neighbor`	`BgpNeighbor`	`peer_ip`, `remote_as`, `local_as`, `vrf?`, `import_policy?`, `export_policy?`
`ospf_neighbor`	`OspfNeighbor`	`interface`, `area`, `network`, `wildcard`, `network_type?`, `vrf?`
`isis_neighbor`	`IsisNeighbor`	`interface`, `level?`, `vrf?`
`interface`	`Interface`	`name`, `address?`, `mask?`, `mtu?`, `vlan_tag?`, `enabled`
`mpls_interface`	`MplsInterface`	`interface`, `ldp_enabled`, `sr_enabled`
`segment_routing`	`SegmentRouting`	`router_id`, `global_block_range`, `node_sid?`
`prefix_list`	`PrefixList`	`name`, `entries[]`
`community_list`	`CommunityList`	`name`, `entries[]`
`routing_policy`	`RoutingPolicy`	`name`, `statements[]`
`access_policy`	`AccessPolicy`	`name`, `rules[]`

Fields marked with `?` are optional. Unrecognised `kind` strings are passed through as untyped `Stanza` bags, allowing operators to extend the DeviceIR without modifying the compiler.

Authoring mapping rules. A mapping rule consists of exactly two fields:

1. **`selector`** — an NTE-QL expression that selects nodes or edges from the enriched topology. Node selectors (`MATCH (n) WHERE ...`) produce one stanza per matching node; edge selectors (`MATCH ()-[e]->() WHERE ...`) produce one stanza per matching edge.
2. **`stanza`** — a template containing `kind`, an optional `key` (for deduplication), and a `fields` map. Field values may use Jinja2 interpolation to reference `node.*` and `peer.*` properties from the topology.

Rules are evaluated in document order. When two rules produce stanzas with the same `kind` and `key` for the same device, the later rule's fields are merged into the earlier stanza (last-writer-wins per field). This enables incremental refinement: a base rule can set interface descriptions and a later rule can overlay MTU values.

Mapping BGP neighbour sessions from the topology

```yaml
- selector: "MATCH ()-[e]->() WHERE e.layer == 'bgp'"
  stanza:
    kind: "bgp_neighbor"
    key: "{{ peer.hostname }}_{{ peer.loopback_ip }}"
    fields:
      peer_ip: "{{ peer.loopback_ip }}"
      remote_as: "{{ peer.asn }}"
      local_as: "{{ node.asn }}"
      description: "eBGP to {{ peer.hostname }}"
```

Mapping loopback interfaces from node properties

```yaml
- selector: "MATCH (n) WHERE n.role == 'leaf' || n.role == 'spine'"
  stanza:
    kind: "interface"
    key: "loopback0"
    fields:
      name: "Loopback0"
      address: "{{ node.loopback_ip }}/32"
      description: "Router ID"
```

```
      enabled: true
```

Figure 31 shows the complete flow from enriched topology to rendered configuration. Two declarative languages drive the pipeline: NTE-QL expressions for selection and rewriting, and Jinja2 templates for property interpolation and final rendering. No stage requires imperative Rust code—the operator authors everything in declarative YAML documents.

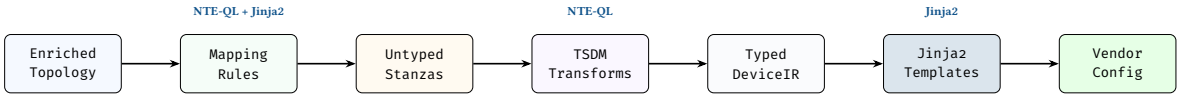


Figure 31. DeviceIR pipeline. Two declarative languages drive the flow: NTE-QL expressions for selection and rewriting (in mapping rules and TSDM transforms), and Jinja2 templates for property interpolation and final rendering. Stanzas remain untyped (JSON bags) until after TSDM, maximising flexibility for hardware-specific mutations.

A.5 Declarative IPAM and Resource Allocation

IP allocation in NetCfg is a *deterministic function of topology*: the same graph with the same design plan always produces the same addresses, regardless of evaluation platform or time. This property—called *sticky topological allocation*—eliminates the most common IPAM failure mode: addresses that drift because they were assigned manually or depend on non-deterministic ordering. The allocator achieves this through a five-pass engine:

1. **State discovery**: Scan all nodes for existing IP assignments (`src_ip`, `dst_ip`, `subnet` fields).
2. **Edge selection**: Evaluate the NetCfg query; sort matched edges by (src, dst) for determinism.
3. **Pre-reservation**: Insert existing allocations into the allocator, preventing re-allocation and enforcing stickiness.
4. **Fulfilment**: For each edge, allocate a subnet using the configured strategy (`dense`, `sparse`, or `hash`). Extract host addresses from the subnet.
5. **Write-back**: Write `src_ip`, `dst_ip`, `subnet` (and IPv6 equivalents for dual-stack) into each endpoint's `DeviceContext`.

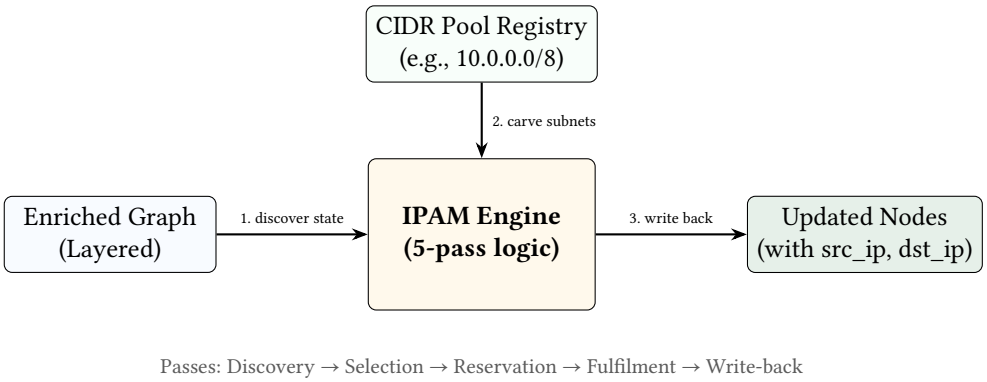


Figure 32. Declarative IPAM workflow. The engine queries the topology graph to discover existing state, carves subnets from the central registry according to design rules, and mutates the layered graph with deterministic host addresses.

Identity-based stickiness. The allocator uses a canonical identity —`sorted(hostname_a, hostname_b)`—as the primary key. If a prior allocation exists for the same identity, the same prefix is returned without pool search. This anchors allocations to stable semantic identity, not integer IDs. If topology IDs change (e.g., after a re-import) but hostnames remain the same, allocations are preserved.

VRF isolation. The allocator maintains per-VRF state. Two primitives using the same CIDR pool in different VRFs can allocate overlapping subnets without conflict. Pool overlap *within* the same VRF is detected and rejected.

The allocation strategy controls how subnets are positioned within the pool. *Dense* allocation packs subnets sequentially—ideal for small, bounded topologies where address efficiency matters. *Sparse* allocation leaves headroom between allocations, accommodating future growth without readdressing existing links. *Hash* allocation derives subnet position from a hash of the edge's semantic identity (sorted hostnames), producing stable allocations that survive topology reordering—useful when the same design plan is applied to topologies imported from different sources.

Hierarchical pools. A `global_pool` primitive registers a large CIDR (e.g., `10.0.0.0/8`). Subsequent `provision_ips` primitives reference the pool by name and specify a `pool_size` (e.g., 24), carving /24 blocks from the parent pool on demand.

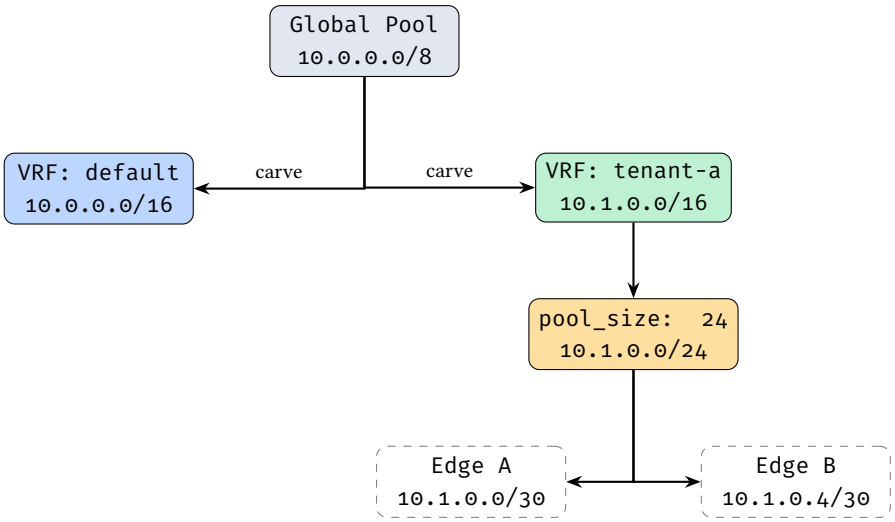


Figure 33. Hierarchical IP pool carving with VRF isolation.

Dual-stack. When `ipv6_pool` is specified, both IPv4 and IPv6 addresses are allocated in a single primitive invocation. IPv6 fields (`src_ipv6`, `dst_ipv6`, `subnet_v6`) are written alongside IPv4 fields.

A.6 Protocol overlay construction (`build_protocol_layer`)

The four-stage execution model of `build_protocol_layer` (select, clone, wire, report) is described in §A.8. This section covers two engine-level details not visible in the DSL: cross-layer state inheritance and MLAG multi-parent mapping.

Cross-layer traceability. The `_source_id` field written during the clone stage links each protocol node to its physical origin. Since `provision_ips` already enriched the physical node's `DeviceContext` with IP addresses, the cloned protocol node inherits them via deep merge—this is how IP addresses flow from IPAM allocation to BGP configuration without explicit wiring. For MLAG/vPC setups, `_source_id` can be an array, mapping one logical protocol node to multiple physical host switches.

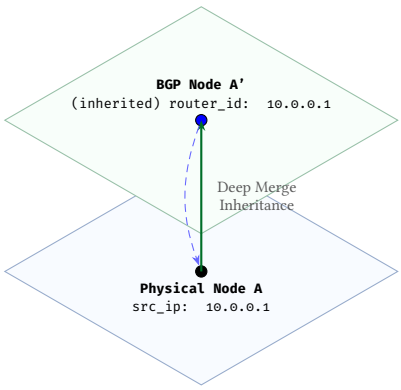


Figure 34. Cross-layer state inheritance. Because the protocol node stores a reference to its physical parent, state allocated in Layer 1 (IPAM) automatically flows into Layer 2 (BGP) without the operator needing to manually pass variables between primitives.

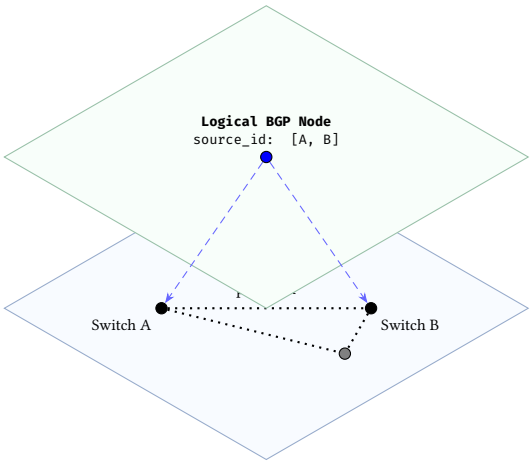


Figure 35. MLAG/vPC topology mapping. Two physical nodes collapse into a single logical protocol node via array `_source_id`, representing a multi-chassis cluster as a single routing entity.

Deep merge semantics: When both the base node and the config overlay contain nested configuration blocks under the same key, the merge recurses key-by-key (overlay wins on conflict). Arrays are replaced entirely rather than concatenated, making it idempotent. This preserves sibling keys that the overlay does not mention. For example, a base node with `{bgp: {as: 65000, holdtime: 180}}` and an overlay with `{bgp: {timers: {keepalive: 60}}}` produces `{bgp: {as: 65000, holdtime: 180, timers: {keepalive: 60}}}`.

Idempotency: The primitive scans the target layer for existing nodes with `_source_id` matching source nodes. Already-cloned nodes are skipped, so re-running the primitive after a partial failure produces the correct result.

A.7 Primitive transformation summary

Each primitive is a function $f(GraphState, Spec) \rightarrow GraphState'$ operating within the shadow topology. Figure 36 shows how primitives, grouped by tier, write into two distinct targets: the *graph structure* (new nodes and edges) and the *DeviceContext* (per-device accumulated state).

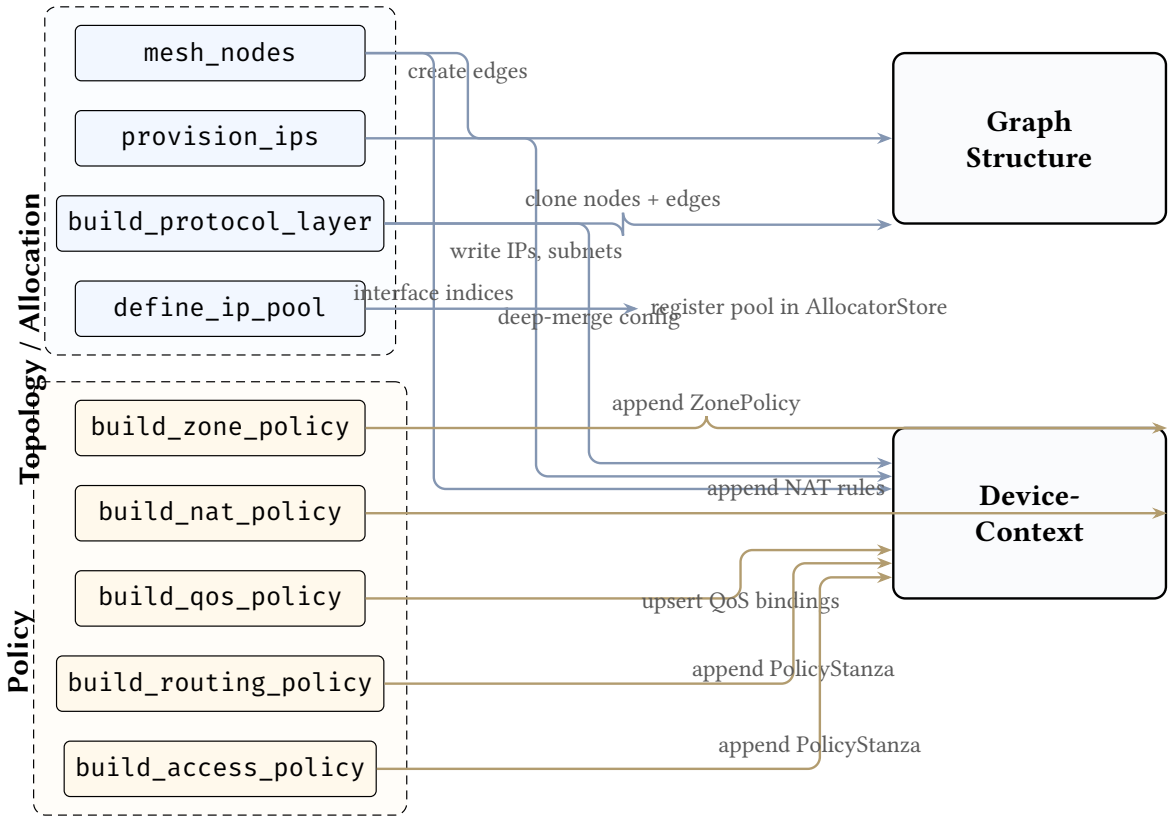


Figure 36. Primitive transformation wiring. Each primitive (left) writes into one or both of two targets: *Graph Structure* (new nodes and edges in the layered topology) and *DeviceContext* (per-device accumulated state). Topology-tier primitives may modify both targets; policy-tier primitives write exclusively to DeviceContext. Arrows are annotated with the key mutation each primitive performs. All mutations operate on the transactional shadow topology.

From primitives to rendered configuration. The wiring in Figure 36 shows only the first half of the data flow. Once all primitives have executed, the accumulated `DeviceContext` for each node passes through two further stages. First, the *mapping evaluator* (§A.4) reads each node's DeviceContext and extracts vendor-neutral stanzas. Second, the *TSDM transform layer* (§C.1) applies hardware-specific rewrites—for example, translating abstract interface indices into physical slot/port identifiers using chassis-to-slot mappings from `hardware-inventory/` files (see Figure 10). The `map_hardware_inventory` primitive stamps chassis and slot metadata onto nodes during the pipeline, and the TSDM `when` predicates use this metadata to select the correct lowering rules. This separation keeps primitives hardware-agnostic: they write abstract state, and the backend resolves it to physical reality.

A.7.1 Dependency graph

Primitives execute in declaration order; the executor performs no topological sorting. Figure 37 shows the implicit dependency constraints that plan authors must respect.

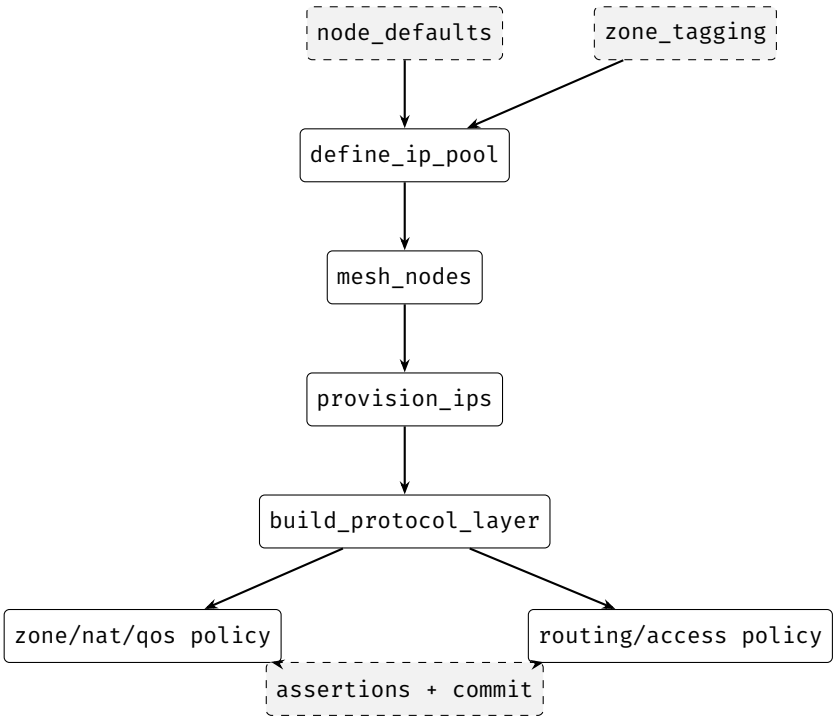


Figure 37. Primitive dependency DAG. Solid boxes are user-declared primitives; dashed boxes are executor-internal steps. Vertical arrows denote must-precede constraints. Nodes at the same level have no ordering constraint relative to each other.

A.7.2 Hoare-style contracts

The three most important primitives can be characterised by semi-formal pre- and post-conditions in Hoare-triple notation $\{P\}\ S\ \{Q\}$.

`provision_ips`.

$$\{\forall e \in evaluate(select):\ e.src\_ip = \bot\}$$

$$provision\_ips(S, pool)$$

$$\{\forall e = (s, d) \in matched:\ s.src\_ip \in allocated(e) \subseteq pool \land d.dst\_ip \in allocated(e) \land \forall e_1 \neq e_2:\ allocated(e_1) \cap allocated(e_2) = \emptyset\}$$

`mesh_nodes`.

$$\{S \subseteq V(layer) \land E_0 = E(layer)\}$$

$$mesh\_nodes(S, type)$$

$$\{\forall (i, j) \in pairs(S, type):\ (i, j) \in E(layer) \land i.peer\_ifaces[j] \neq \bot \land E_0 \subseteq E(layer)\}$$

`build_protocol_layer`.

$$\{S \subseteq V(source) \land target \neq source\}$$

$$build\_protocol\_layer(S, target, config)$$

$$\{\forall n \in S:\ \exists n' \in V(target):\ n'.\_source\_id = n.id \land n'.metadata \supseteq merge(n.metadata, config)\}$$

These contracts formalise the implicit guarantees that downstream primitives rely on. A full contract catalogue covering all nine primitives is maintained in the data model specification (Chapter 9, §9.4).

A.8 Design rules in the query language

The `netcfg describe` command renders the entire design plan in the typed query language, organised by primitive tier: topology, allocation, overlay, and policy. The following listings show the

canonical representation of a multi-protocol data-centre fabric. This is the authoritative form—the YAML design plan on disc is merely a serialisation of these rules.

A.8.1 Topology rules

The topology tier establishes the graph structure: `mesh_nodes` creates edges between selected populations, and `provision_ips` allocates addressing.

```
mesh on $bgp_nodes {
    type: mesh_type = Full
}

provision_ips on edges[true] {
    pool: cidr     = '10.0.0.0/24'
    subnet_size: u8 = 31
    strategy: strategy = Dense
}
```

A.8.2 Allocation rules

Resource allocation primitives draw from named pools. Each rule specifies a selector, resource type, pool range, and allocation strategy.

```
provision_resources on $all_nodes {
    resource_type: string = 'bgp_as'
    pool: string          = '65001-65001'
    strategy: strategy    = Dense
}

provision_resources on $all_nodes {
    resource_type: string = 'router_id'
    pool: string          = '10.255.0.1-10.255.0.255'
    strategy: strategy    = Dense
}
```

A.8.3 Protocol overlay rules

Each `build_protocol_layer` invocation is rendered as a typed `overlay` block showing the selector, typed fields, and concrete values. Fabric encapsulations (`vxlan`, `evpn_fabric`) render similarly from their dedicated primitives.

```
# Protocol overlays (from build_protocol_layer)

overlay ospf on $all_fabric {
    clone_underlying: bool = true
    process_id: u32        = 1
    area: string           = '0.0.0.0'
}

overlay isis on $wan_pe {
    net: string   = '49.0001.0000.0000.0001.00'
    level: string = 'level-2'
}

overlay bgp on $all_fabric {
    asn: u32 = 65000
}

overlay evpn on $leaves {
    vni: u32          = 10000
    route_target: string = '65000:10000'
}

# Fabric encapsulations (from overlay_vxlan, overlay_evpn)

vxlan on $leaves {
    vni_base: u32            = 10000
    mcast_group_base: string = '239.1.1.1'
}

evpn_fabric on $leaves {
```

```
    route_distinguisher_base: string = 'auto'
    route_target_base: string        = '65000:10000'
}
```

A.8.4 Strict topology schema

The topology engine validates every node against a strict schema. Protocol overlay nodes must carry metadata tracing them to their physical origin (`_source_id`) plus the protocol-specific required fields. The `schema` blocks below are generated programmatically from `TopologySchema::v1_minimal()`.

```
schema physical.* {
    hostname: string             [required]
    device_os: string            [required]
}

schema ospf.* {
    metadata.protocol_layer: string  [required]
    metadata._source_id: u32         [required]
    process_id: u32                  [required]
    area: string                     [required]
}

schema bgp.* {
    metadata.protocol_layer: string  [required]
    metadata._source_id: u32         [required]
    asn: u32                         [required]
}

schema isis.* {
    metadata.protocol_layer: string  [required]
    metadata._source_id: u32         [required]
    net: string                      [required]
    level: string                    [required]
}

schema evpn.* {
    metadata.protocol_layer: string  [required]
    metadata._source_id: u32         [required]
    vni: u32                         [required]
    route_target: string             [required]
}
```

A.8.5 Policy rules

Policy primitives attach routing policy, prefix lists, and community lists to selected populations. Each renders as a typed block with nested statement or entry sub-blocks.

```
prefix_list on $all_nodes {
    name: string = 'PL-LOOPBACKS'
    entry '10.255.0.0/24' action permit le 32
}

community_list on $all_nodes {
    name: string = 'CL-EVPN'
    entry '65001:1000' action permit
}

routing_policy on $all_nodes {
    policy_name: string = 'RP-UNDERLAY-EXPORT'
    statement 'PERMIT-LOOPBACKS' {
        action: string = 'permit'
        match_prefix_list: string = 'PL-LOOPBACKS'
    }
}
```

A.8.6 Plan schema

The plan schema defines the YAML shape an operator must write to specify each overlay. Fields derived from the Rust struct types: non-`Option<T>` fields are required; `Option<T>` fields are optional.

```yaml
# build_protocol_layer (protocol: ospf)
  selector: NTE-QL
  layer: "ospf"
  clone_underlying: bool   # default false
  config:
    process_id: u32        # required
    area: string           # required
    router_id: string      # optional

# build_protocol_layer (protocol: bgp)
  selector: NTE-QL
  layer: "bgp"
  clone_underlying: bool
  config:
    asn: u32               # required
    ebgp_multihop: u32     # optional
    router_id: string      # optional

# build_protocol_layer (protocol: isis)
  selector: NTE-QL
  layer: "isis"
  clone_underlying: bool
  config:
    net: string            # required
    level: string          # required

# build_protocol_layer (protocol: evpn)
  selector: NTE-QL
  layer: "evpn"
  clone_underlying: bool
  config:
    vni: u32               # required
    route_target: string   # required

# overlay_vxlan
  selector: NTE-QL
  vni_base: u32                     # required
  mcast_group_base: string          # optional

# overlay_evpn
  selector: NTE-QL
  route_distinguisher_base: string  # required
  route_target_base: string         # required
```

A.8.7 Design principle: query-language-first

The three schema artefacts above—overlay rules, topology schemas, and plan schemas—illustrate a deliberate architectural choice. Every primitive's behaviour is parameterised by *NTE-QL expressions* rather than imperative code:

- **Population selection**: `nodes[role == 'leaf']` determines which devices receive a protocol overlay or policy.
- **Property extraction**: mapping rules use NTE-QL selectors with Jinja2 templates to interpolate graph properties into DeviceIR stanzas.
- **Hardware rewriting**: TSDM transforms evaluate NTE-QL `match_expr` and `apply` expressions to mutate stanzas per-vendor, per-chassis.

The YAML design plan is a serialisation of these query-language rules. The canonical form (as rendered by `netcfg describe`) strips the YAML envelope and presents the design as typed NTE-QL expressions over the topology graph. This design enables a future in which the Rust primitive catalogue shrinks to a thin execution harness: primitives validate schemas, compile selectors, and

apply mutations, but all domain-specific logic resides in NTE-QL and Jinja2. New protocol overlays, policy types, or hardware models can be added by authoring NTE-QL expressions and Jinja2 templates—without modifying the compiler itself.

A.9 Configuration diffing (DiffEngine)

The diff engine computes a *mutation plan* between two topologies by exporting each layer to JSON and comparing:

1. **Node diff**: Per layer, compare node sets by a composite semantic key (`layer, hostname, protocol_type`). Diffing by integer ID is avoided, as node insertions would shift downstream IDs and create massive false diffs. Detect additions, deletions, and modifications (field-by-field property comparison via `extract_properties`, which unpacks the `data_json` blob).
2. **Edge diff**: Compare by a composite semantic key derived from the resolved (src, dst) node semantic keys, allowing structural property comparison without relying on volatile internal edge IDs.

The `MutationPlan` contains six change types: `AddNode`, `RemoveNode`, `UpdateNode`, `AddEdge`, `RemoveEdge`, `UpdateEdge`. Changes are sorted deterministically by (change_type, layer, id/src/dst) for reproducible output.

The `plan` command displays the plan as a human-readable diff; the `ci-run` command uses it to compute per-device configuration diffs.

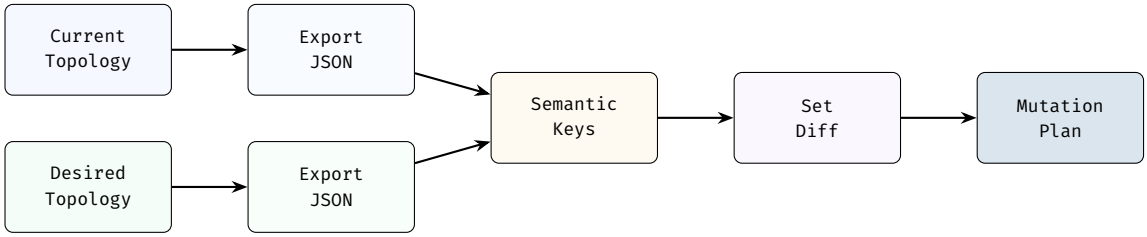


Figure 38. DiffEngine algorithm. Two topologies are exported layer-by-layer to JSON, keyed by composite semantic identifiers, and compared via set difference to produce a deterministic `MutationPlan`.

Table 11. The six `Change` variants in a `MutationPlan`.

Variant	Description
`AddNode`	A node present in the desired topology but absent in the current topology.
`RemoveNode`	A node present in the current topology but absent in the desired topology.
`UpdateNode`	A node present in both topologies whose properties differ (field-by-field comparison).
`AddEdge`	An edge present in the desired topology but absent in the current topology.
`RemoveEdge`	An edge present in the current topology but absent in the desired topology.
`UpdateEdge`	An edge present in both topologies whose properties differ.

A.10 Blast-radius analysis

When a mutation plan contains dozens or hundreds of changes, operators need to understand which devices face the greatest risk before applying the plan to a live network. The `BlastRadiusAnalyzer` assigns a monotonic risk level (Low, Medium, or High) to every node affected by a mutation plan. Risk can only be elevated, never reduced—if two changes affect the same node, the higher risk wins.

The analyser classifies each change using structural rules (Table 12) and then propagates risk to neighbours: when a node's critical property changes, every 1-hop neighbour (across all layers) is elevated to at least Medium. Critical properties are those whose modification could cause adjacency loss or route withdrawal: `asn`, `router_id`, `src_ip`, `dst_ip`, `subnet`, `loopback`, and `mgmt_ip`.

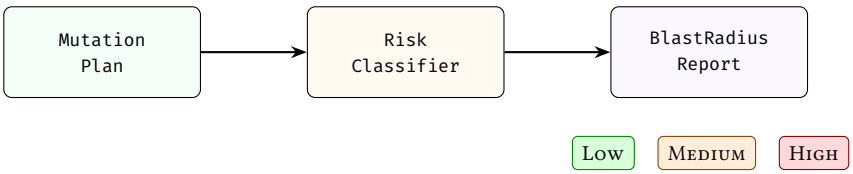


Figure 39. Blast-radius pipeline. Each change is classified into a risk level; 1-hop propagation elevates neighbours of critically modified nodes.

Table 12. Risk-level assignment rules for `BlastRadiusAnalyzer`.

Change type	Risk	Rationale
`AddNode`	Medium	New node may alter adjacencies.
`RemoveNode`	High	Removing a node drops all associated sessions.
`UpdateNode`	Property-dependent	High if a critical property changes; Low otherwise.
`AddEdge`	Medium (High if physical)	Physical edges represent cabling changes.
`RemoveEdge`	Medium (High if physical)	Removing a physical edge severs connectivity.
`UpdateEdge`	Medium (High if physical)	Physical-layer updates may affect link state.

A.11 Telemetry integration

The synthesis engine defines a `TelemetryProvider` trait that supplies live device state to assertion primitives. The trait exposes a single method, `get_state(node, path)` → `JSON`, where `node` identifies a device by hostname and `path` selects a state subtree (e.g. `bgp/neighbors`). A `MockTelemetryProvider` loads state from a JSON fixture file, enabling offline testing of assertion primitives without access to a live network. In production, a concrete provider could query gNMI or RESTCONF endpoints; the trait boundary isolates the engine from transport-specific concerns.

B Design Authority and Intelligent Synthesis

A fundamental limitation of traditional network automation tools is their passive nature: they generate configuration based on explicit inputs but lack the mathematical rigorousness to prove that those inputs represent a structurally sound design. By transitioning the compiler into a formal **Design Authority** and **Intelligent Synthesis Engine**, we move from checking configurations to optimising architectural intent.

This section defines the foundational graph-theory models used for formal verification, introduces the constraint-based synthesis operations that automatically propose topological repairs, and details the implementation of the provenance engine that ensures end-to-end traceability.

B.1 Foundational Mathematics: Topological Invariants and Constraints

At the core of the intelligent synthesis engine is the ability to evaluate multi-hop topological invariants. We model the network as a layered multidigraph $G = (V, E, L)$, where L represents the set of abstract layers.

We extend the NetCfg Query Language (NTE-QL) with a formal `paths` target to evaluate structural redundancy and latency constraints. Let $P(u, v)$ be the set of all simple paths between nodes u and v in a specific layer $l \in L$. We define two primary invariants:

1. **Edge-Disjoint Redundancy (k-connectedness):** A design satisfies the k-redundancy constraint if, for every path pair $(p_i, p_j) \in P(u, v)$, the intersection of their edge sets is empty ($E(p_i) \cap E(p_j) = \emptyset$), and the cardinality of the maximum disjoint path set is at least k. We evaluate this via the `MinDisjointPaths { count: k }` assertion.
2. **Bounded Reachability (Latency):** A design satisfies the d-latency constraint if the shortest path length $\min_{p \in P(u,v)} |E(p)| \leq d$. We evaluate this via the `MaxHopDistance { max: d }` assertion.

By expressing these requirements in a formal `constraints` DSL block within the Design Plan, the compiler transforms from a static validation tool into a dynamic solver capable of identifying the precise edges ($e \in E$) that violate the architectural intent.

B.2 Architectural Blueprints and Constraint-Based Synthesis

With the mathematical foundations established, we introduce three advanced synthesis operations that leverage these models to enforce and optimise network design: Formal Blueprints, Constraint-Based Synthesis, and Design Unit Testing.

Formal Architectural Blueprints. Instead of individually verifying links, architects can declare a target `blueprint` (e.g., a 3-Stage Clos, a Ring, or a Hub-and-Spoke). The assertion engine uses graph theory to prove that the supplied topology structurally maps to the intended design. For example, a `ThreeStageClos` blueprint enforces a strict bipartite property, ensuring zero intra-tier links and complete full-mesh inter-tier connectivity.

Constraint-Based Synthesis Proposer. When the topological solver detects that a constraint (e.g., `min_paths`) is violated, the system does not simply fail. The `netcfg suggest` command operates as an active optimisation engine. It computes the missing topological mutations—such as identifying two disconnected nodes u and v whose union would resolve the disjoint-path deficit—and outputs a concrete "Proposed Fix" (e.g., adding an edge between a specific Spine and Leaf).

Design Unit Testing and Live Validation. To verify these blueprints before production, the system introduces Design-Driven Development via the `netcfg test` command. This framework executes a blueprint against a library of mock reference topologies. Furthermore, the `TelemetryMatch` assertion closes the loop by compiling live telemetry data directly into the CEL evaluation context, mathematically comparing the physical reality against the Desired IR.

B.3 Implementation: Provenance and the BFS Solver Engine

The implementation of these intelligent features required deep changes to the compiler's execution pipeline, specifically the introduction of the BFS-based `SynthesisSolver` and the formal `Provenance` metadata model.

The Synthesis Solver. The `evaluate_paths` method in NTE-QL executes a Breadth-First Search (BFS) bounded by the graph layers to discover $P(u, v)$. The solver groups paths by source-destination pairs and evaluates the edge-disjoint cardinality using a greedy intersection check. When a violation is detected, it generates a `ConstraintViolation` struct containing an actionable `TopologyMutation` (e.g., `AddEdge { from, to }`).

End-to-End Provenance Traceability. To eliminate "Configuration Magic," we expanded the `DeviceIR` contract to include a strict `Provenance` struct. As the compiler traverses the pipeline, every primitive (such as `build_ospf_session` or `build_zone_policy`) stamps its generated stanzas with its primitive name, the source layer, and a human-readable rationale. This data is surfaced via the `netcfg explain` command, which maps any rendered configuration line back to the exact mathematical invariant and design rule that necessitated its creation.

C Hardware Adaptation & Rendering

Once the synthesis engine has produced an enriched digital twin (§A), two problems remain: extracting vendor-neutral stanzas from the enriched nodes, and lowering those stanzas to platform-specific CLI syntax. This section describes the two-phase solution: the mapping evaluator (DeviceIR extraction) and the TSDM transform layer (target-specific lowering).

The synthesis strategy intentionally insulates the network architect from the minutiae of physical hardware constraints during the initial logical design phase. However, a purely logical design plan cannot be directly deployed to physical routers. Network devices have highly specific constraints:

an Arista switch might address its first port as `Ethernet1`, while a Nokia router addresses it as `ethernet-1/1`.

Rather than polluting the logical design plan with these vendor idiosyncrasies, NetCfg relies on a dedicated **Target-Specific Device Model (TSDM)** layer. This layer acts as a compiler "lowering" phase, rewriting the vendor-neutral DeviceIR into concrete, platform-specific syntax via declarative Model-to-Model transformations.

C.1 Stage 3: Target-Specific Transforms and CEL-Based AST Rewriting

Dynamic rules intercept and mutate vendor-neutral configuration stanzas before the final rendering engine receives them. This is implemented via **CEL-Based AST Rewriting**. The engine evaluates Common Expression Language (CEL) predicates against the untyped stanzas. A transform fires only when its `when` predicate evaluates to true for a given device (e.g., `device_os == 'nxos'`).

Crucially, this layer includes a formal **Hardware Lowering DSL**. Abstract topology references (e.g., an abstract interface index of 5) are computationally converted into physical hardware representations (e.g., `Ethernet1/1`) using linear offset patterns. This demonstrates a robust decoupling of logical intent from vendor hardware layouts, executed entirely via AST rewriting rather than hardcoded logic.

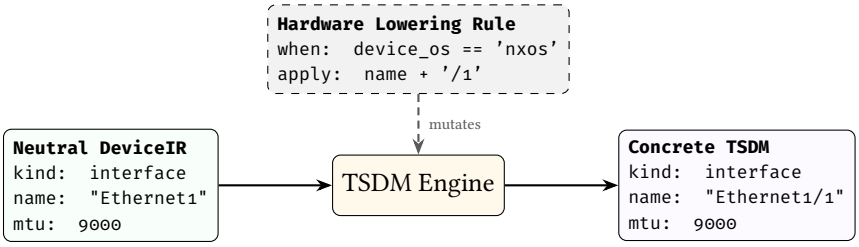


Figure 40. Model-to-Model lowering via TSDM. Vendor-neutral DeviceIR is transformed into a hardware-specific data model by the TSDM engine, dynamically injecting chassis layouts and OS-specific terminology.

The execution engine evaluates the sequence of `RewriteRule` entries. Each rule provides a `match_expr` (an NTE-QL query to select target stanzas) and an `apply` map (an expression to compute the lowered value).

Listing 15. A vendor-specific transform bridging logical and physical naming.

```
transforms:
  - name: nxos-interface-rewrite
    when: "device_os == 'nxos'"
    rules:
      - match_expr: "stanza.kind == 'interface'"
        apply:
          name: "'Ethernet1/' + string(stanza.fields.abstract_index + 1)"
```

TSDM in action: lowering abstract interfaces to NX-OS slot/port names

```
transforms:
  - name: "nxos_lowering"
    when: "device_os == 'nxos'"
    rules:
      - match_expr: "kind == 'interface' && name.startsWith('Ethernet')"
        apply:
          name: "name + '/1'"  # Ethernet1 -> Ethernet1/1
          mtu: "9216"          # Force jumbo frames
```

C.1.1 Execution Logic

The transformation engine performs a deep-walk of the DeviceIR stanzas. For each stanza, it:

1. Binds the stanza's fields as NTE-QL variables.

2. Evaluates the `match_expr` predicate.
3. If matched, executes the `apply` expressions to mutate the stanza's state.

This structured approach allows complex hardware logic—such as mapping logical ports to specific line-card slots in a modular chassis—to be expressed as programmable transformations rather than hardcoded logic or brittle templates.

The key architectural insight is that `DeviceContext` accumulates state across the entire primitive pipeline:

1. `mesh_nodes`: writes `mesh_type`, `peer_count`, and optionally `peer_interfaces` into metadata.
2. `provision_ips`: writes `src_ip`, `dst_ip`, `subnet`, `vrf`.
3. `build_protocol_layer`: deep-merges all of the above into the cloned protocol node, plus the config overlay.
4. `allocate_resources`: writes the allocated resource value (e.g., `asn`) into metadata.

By the time Stage 4 (rendering) occurs, each node's `DeviceContext` carries hostname, IPs, protocol metadata, interface assignments, and any custom fields—the complete information needed to produce a device configuration file.

C.1.2 The stanza type duality

A critical architectural property of the TSDM boundary is the coexistence of two parallel stanza representations:

Table 13. The stanza type duality at the rendering boundary. `TargetDeviceModel.stanzas` is typed, but templates receive the untyped representation.

Representation	Type	Consumed by
Untyped	`Stanza { kind, key, fields }`	Templates, mapping evaluator
Typed	`StandardStanza` enum (12 variants)	TSDM, Rust-level validation

The typed representation is produced by `Stanza::to_standard()`, which matches on the `kind` string and deserialises fields into typed structs. However, in `generate_configs()`, the serialised `TargetDeviceModel` has its `stanzas` key *overwritten* with the untyped `Stanza` bags before rendering. Templates therefore access stanza data via `stanza.fields.<key>`, not via typed struct fields.

This duality means that the typed `StandardStanza` enum—which validates field types, enforces required fields, and catches unknown stanza kinds at parse time—is constructed and then *discarded* at the rendering boundary. The 12 "extended" stanza kinds not covered by `StandardStanza` (e.g. `ospf_config`, `bgp_config`, `vxlan_config`, `lacp_config`) pass through entirely unvalidated: the template is the sole consumer and bears full responsibility for field interpretation.

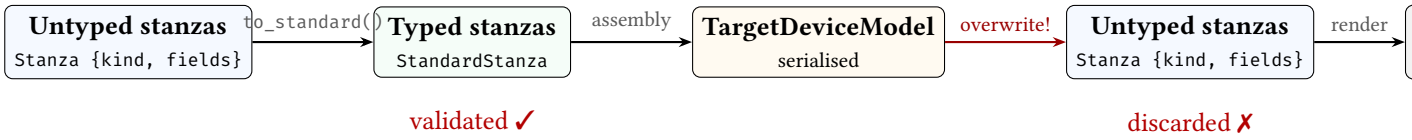


Figure 41. The stanza type flow through the TSDM boundary. Untyped stanzas are converted to typed `StandardStanza` variants (with validation), assembled into the `TargetDeviceModel`, then *overwritten* with the original untyped bags before template rendering. The typed validation is performed but its results are discarded.

C.1.3 TargetDeviceModel construction

The `DefaultTargetTransform::transform()` method builds the final model in four steps:

1. **IPv6 injection.** Before TSDM runs, `inject_ipv6_into_stanzas()` walks the untyped stanzas. For each `interface` stanza, it matches the stanza's `address` field against `DeviceContext.src_ip` or `dst_ip`. If matched and IPv6 addresses are present, it injects an `ipv6_address` field with the correct prefix length.

2. **CEL-based stanza rewriting.** The transform engine applies each `TransformationSpec` from the design plan's `transforms:` section (as described above).
3. **Typed conversion.** Each rewritten stanza is converted via `Stanza::to_standard()`. Known `kind` values produce typed variants; unknown kinds produce errors—but in practice, the untyped rendering path bypasses this (§C.1.2).
4. **Model assembly.** The final struct is populated: `hostname`, `device_os`, `platform` (from metadata), the typed stanzas (overwritten at render time), the typed `SecurityModel` and `QosModel` (passed through), and *all metadata keys* flattened to top-level template variables via `#[serde(flatten)]`. This flattening is how values like `bgp_as`, `router_id`, and `loopback0` reach templates without explicit field declarations.

C.1.4 Two rendering paths

The codebase has two distinct rendering paths:

Path A — Full pipeline (`generate_configs`). Uses mapping document → TSDM → templates. This is the production path: topology → mapping evaluation → stanza grouping → IPv6 injection → TSDM transform → template rendering → transactional file output.

Path B — Preview (`render_topology_to_memory`). No mapping document, no TSDM. Reads `DeviceContext` directly, builds synthetic interface stanzas from IPAM fields (`build_interface_stanzas_from_context`), injects IPv6, and renders. Used for preview/plan rendering.

This duality means templates must handle both paths: they may receive fully-enriched stanzas (Path A) or minimal synthetic stanzas (Path B).

C.2 Stage 4: Syntax Serialisation

With stanzas lowered to platform-specific form, the final stage renders them through vendor templates into deployable configuration files.

MiniJinja templates act as a *dumb emission layer*, serialising the final Target-Specific Device Model into vendor-specific CLI syntax. Templates contain no conditional logic beyond iteration over stanza lists and field interpolation—all architectural decisions have already been resolved by the preceding layers. This keeps templates thin and auditable: a template bug can only affect formatting, never topology or policy semantics.

The `TransactionalOutput` writer collects all files in temporary locations (using `NamedTempFile` in the same directory for same-filesystem guarantee), then atomically renames all via POSIX `rename(2)` on commit. If any stage fails, all temp files are dropped without committing—no partial output directory is ever visible.

Three artefacts are emitted per node:

- `{hostname}.conf`: the rendered device configuration.
- `{hostname}.deviceir.json`: the raw DeviceIR stanzas (audit).
- `{hostname}.lineage.json`: per-line mapping from config lines to the `rule_id` that produced them (provenance tracing).

C.2.1 Determinism guarantee

Given a fixed Blueprint G, design plan B, and mapping document M, the output configuration set $C = compile(G, B, M)$ is deterministic. A serialisation round-trip produces a bit-identical shadow. Primitives execute in declaration order. The engine evaluates selectors deterministically, allocates node IDs monotonically, and iterates edges sorted by (src, dst). DeviceIR stanzas use `BTreeMap` with post-sorting. Template rendering is pure.

C.2.2 Template rendering engine

MiniJinja is configured with:

- `UndefinedBehavior::Chainable` — undefined variables return a falsy value, enabling `{% if x %}` guards without explicit existence checks.

- `AutoEscape::None` — raw text output, no HTML escaping.
- `keep_trailing_newline: true`.

Templates are loaded from a flat `templates/*.j2` directory. The file stem (e.g. `eos` from `eos.j2`) becomes the template name, matched against `device_os` at render time. Each vendor template is self-contained; the engine supports neither inheritance nor includes.

Templates iterate stanzas and dispatch on `kind`:

Listing 16. Stanza dispatch pattern in vendor templates.

```
{% for stanza in stanzas %}
  {% if stanza.kind == "interface" %}
    {{ stanza.fields.name }}
    {{ stanza.fields.address }}
  {% elif stanza.kind == "bgp_neighbor" %}
    {{ stanza.fields.peer_ip }}
    {{ stanza.fields.remote_as }}
  {% endif %}
{% endfor %}
```

C.2.3 Vendor template divergence

The same `TargetDeviceModel` is rendered by seven vendor templates, each emitting platform-specific syntax. Table 14 highlights the key points of divergence.

Table 14. Vendor template divergence across key features. Each column represents a distinct template (`*.j2`).

Feature	EOS	JunOS	IOS-XR	NX-OS
Config style	Imperative	Hierarchical	Imperative	Imperative
Interface addr	`ip address`	`family inet { address }`	`ipv4 address`	`ip address`
VRF	`vrf X`	`instance-type virtual-router`	`vrf X`	`vrf member X`
BGP	`router bgp`	`group PEER-X`	`router bgp`	`feature bgp`
NAT64	Prefix emit	Prefix emit	Prefix emit	Skip

Three additional templates (SR OS, VyOS, Cisco IOS) follow similar patterns with platform-specific syntax. The architectural guarantee is that all vendor differences are confined to templates—no vendor-specific logic exists in the compilation pipeline itself.

D CLI Reference

NetCfg provides fourteen CLI commands spanning the development-to-deployment lifecycle. All commands use `clap` for argument parsing. Table 15 shows which pipeline layers each command invokes.

Table 15. CLI commands and the pipeline layers they invoke. ✓= executes; ◦ = optional (via flag).

Command	*S1 Synthesis*	*Assert Gate*	*S2 DeviceIR*	*S3 TSDM*	*S4 Render*	Primary output
`generate`	✓	✓	✓	✓	✓	Device config files
`validate`	✓	✓			◦	Pass/fail exit code
`plan`	✓	✓			◦	Mutation diff
`inspect`	◦					DeviceIR / DOT / trace
`ci-run`	✓	✓	✓	✓	✓	JSON report
`lint`	✓	✓				Assertion results
`impact`	✓	✓				Risk assessment
`report`	✓	✓				Markdown report
`fmt`						Formatted YAML
`import`						.nte archive
`export-clab`	✓	✓				clab.yaml
`sync`						.nte archive / stdout
`describe`						Query-language report
`schema`						Pass/fail exit code

D.1 `netcfg generate`

Runs the full four-stage compilation pipeline (§2.2.1), writing artefacts to disk.

```
netcfg generate <topology> <design plan> \
    --templates-dir <DIR> \
    [--output-dir <DIR>] [--emit-ir] [--verbose]
```

Argument	Default	Description
`topology`	—	Path to Blueprint archive (.nte) or NSoT URI
`design plan`	—	Path to design plan YAML
`–templates-dir`	—	Directory containing `.j2` templates
`–output-dir`	`./output`	Output directory for generated files
`–emit-ir`	false	Also write `{hostname}.ir.json`
`–verbose`	false	Print per-node rendering progress

Output: one `.cfg` file per device (plus `.ir.json` if `–emit-ir`). Absolute paths are printed to stdout, one per line, suitable for piping.

D.2 `netcfg validate`

Executes the pipeline in dry-run mode: all primitives run with `dry_run=true`, skipping DataFrame writes but verifying what *would* change. Optionally pre-flights templates by rendering to memory.

```
netcfg validate <topology> <design plan> \
    [--templates-dir <DIR>] [--verbose]
```

Exit codes: 0 on success, non-zero on validation failure (parse errors, assertion failures, template syntax errors, IP pool conflicts).

D.3 `netcfg plan`

Computes the topology `MutationPlan` (current vs. desired) and displays additions, removals, and property changes. With `–diff`, it renders before/after configs and shows a unified text diff per device using the `similar` crate.

```
netcfg plan <topology> <design plan> \
    [--diff] [--templates-dir <DIR>] [--verbose]
```

Output without `–diff`: colourised plan (green for additions, red for removals, yellow for modifications). Summary line: `+N additions, -N removals across M layers`.

Output with `–diff`: unified diff per device, preceded by `diff –cfg {hostname}`.

D.4 `netcfg inspect`

Provides three views into intermediate state:

```
netcfg inspect <path> --ast               # Print design plan's structured logical plan
netcfg inspect <path> --topology          # Export as Graphviz DOT
    [--design plan <FILE>]
netcfg inspect <path> --trace <NODE_ID>   # Trace node state changes
    --design plan <FILE>
```

- `–ast`: Parses the file as a design plan and prints the structural model.
- `–topology`: Exports the graph as Graphviz DOT. With `–design plan`, applies the design plan first. Output includes per-layer subgraph clustering, hostname labels, and subnet annotations.
- `–trace <node_id>`: Replays primitive execution and prints before/after `DeviceContext` snapshots for the specified node at each primitive step. When design plans fail with selector or assertion errors, finding the root cause in a 1,000-node graph is challenging; `–trace` is the primary debugging workflow for design plan authors.

D.5 `netcfg ci-run`

The CI/CD integration point. Runs validate, plan, and config diff in a single invocation.

```
netcfg ci-run <topology> <design plan> \
    --templates-dir <DIR> [--json]
```

With `–json`, emits a structured payload:

```
{
  "status": "success",
  "topology_additions": 42,
  "topology_removals": 0,
  "configuration_diffs": {
    "core-0": "-  ip address 10.0.0.4/30\n+  ip address 10.0.0.8/30"
  },
  "warnings": []
}
```

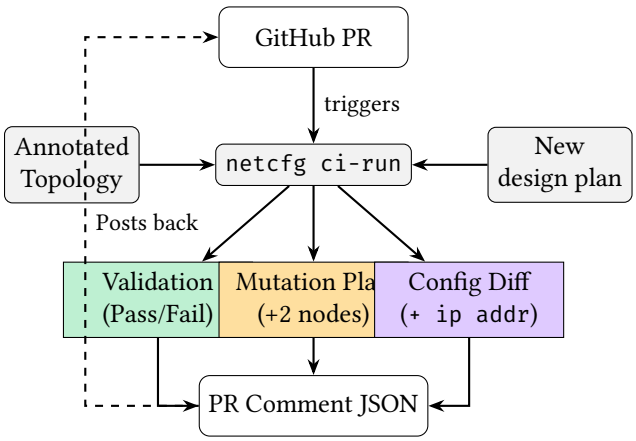


Figure 42. The NetDevOps CI/CD pipeline enabled by `ci-run`.

This integrates with GitHub Actions, GitLab CI, or any CI system that can parse JSON.

D.6 `netcfg import`

Imports a structured topology definition into an NTE archive.

```
netcfg import <yaml_path> <output.nte>
```

The importer reads a topogen-format Design Plan file (defining nodes with types, layers, and properties) and serialises it to an `.nte` archive that other commands consume.

D.7 `netcfg lint`

Runs design-rule assertions without the full validation pipeline. Applies the design plan in dry-run mode, then evaluates assertions and reports results.

```
netcfg lint <topology> <design plan> [--verbose]
```

Returns assertion results with pass/fail status, severity, and diagnostic messages. Exit code 1 only when `error`-severity assertions fail; warnings are reported but do not cause failure.

D.8 `netcfg fmt`

Standardises design plan YAML formatting. Parses the design plan, re-serialises it in canonical form, and compares byte-for-byte. Idempotent.

```
netcfg fmt <design plan> [--check] [--verbose]
```

With `–check`, exits with code 1 if formatting changes are needed (useful as a pre-commit hook or CI gate). Without `–check`, overwrites the file in place.

D.9 `netcfg impact`

Calculates the operational blast radius of a design plan change against a base topology. The topology argument accepts either a local `.nte` archive path or an NSoT URI (e.g. `netbox://host?site=ldn1`); see `netcfg sync` (§D.12) for URI syntax.

```
netcfg impact <topology> <design plan> [--detail] [--verbose]
```

The `–detail` flag expands the low-risk section to list individual nodes (by default only a count is shown).

For each mutation in the computed plan, the blast-radius analyser assigns a risk level (High/Medium/Low) using the rules in Table 12. Output includes per-node risk attribution with reasons.

D.10 `netcfg report`

Generates an automated Markdown architecture report from a compiled topology.

```
netcfg report <topology> <design plan> -o <output.md> [--verbose]
```

The report contains three sections:

1. **Node inventory**: hostname, type, OS, ASN, loopback, management IP.
2. **Physical links & IP addressing**: source/destination hosts, interfaces, IPs, subnets.
3. **Topology visualisation**: A `netviz-json` code block with a custom JSON schema (`NetVizNode`, `NetVizEdge`, `NetVizTopology`) suitable for rendering by external visualisation tools.

D.11 `netcfg export-clab`

Generates a Containerlab `clab.yaml` simulation file from a compiled topology, enabling rapid lab deployment for testing (e.g. the three-site mesh from §6.4). The topology argument accepts a local `.nte` archive or an NSoT URI (§D.12).

```
netcfg export-clab <topology> <design plan> \
    -o <clab.yaml> [--configs-dir <DIR>] [--verbose]
```

Heuristically maps `device_os` to Containerlab container kinds: `cisco_iosxr` → `vr-xrv9k`, `nokia_srl` → `srl`, `arista_eos` → `ceos`, default → `linux`. When `–configs_dir` is

provided, bind-mounts generated configuration files into OS-specific remote paths (e.g. `/etc/opt/srlinux/config.json` for SRL, `/mnt/flash/startup-config` for cEOS).

D.12 `netcfg sync`

Synchronises topology data with external Network Source of Truth (NSoT) systems. Two subcommands are available:

```
netcfg sync pull <uri> [-o <output.nte>] [--token <TOKEN>] [--insecure]
netcfg sync push <uri> <source> [--token <TOKEN>]
```

`sync pull` fetches a live topology from an NSoT and converts it into an NTE topology archive. The URI scheme identifies the provider: `netbox://host[:port]?site=X&region=Y` targets a NetBox [5] instance via its GraphQL endpoint; query parameters are passed as filters. Authentication is resolved in order: the `–token` flag, then the `NETBOX_TOKEN` environment variable. When `-o` is supplied the topology is persisted to disk; otherwise a summary (node and edge counts) is printed. The `–insecure` flag disables TLS certificate verification for self-signed development instances.

`sync push` is a planned stub that will write intent back to the NSoT (see §G).

Note: NSoT URI in other commands

The topology argument of `impact` (§D.9) and `export-clab` (§D.11) also accepts a `netbox://` URI. Internally these commands dispatch through the same `load_topology_source()` helper, so a live NSoT topology can be used without a local archive.

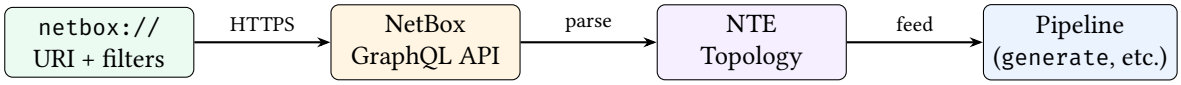


Figure 43. NSoT synchronisation data flow.

D.13 `netcfg describe`

Renders the canonical query-language view of a design plan. Rather than displaying raw YAML, `describe` produces a structured report showing all design rules, selectors, and typed schemas in the NetCfg expression language, organised by primitive tier: topology, allocation, overlay, and policy.

```
netcfg describe <design_plan>
```

Output: a human-readable report on stdout listing each primitive with its selector expression and typed configuration fields. Useful for reviewing a design plan without executing the pipeline.

D.14 `netcfg schema`

Validates that an input topology conforms to the strict NTE schema expectations. This is a preflight check: it does not execute any design-plan primitives.

```
netcfg schema <topology>
```

The topology argument accepts a local `.nte` archive path or an NSoT URI (§D.12). If validation succeeds, prints a confirmation message and exits with code 0; otherwise reports the schema violations and exits with code 1.

E Error Model

NetCfg treats errors as first-class data. Every error carries a machine-readable diagnostic code, a human-readable message, and—where applicable—a source span pointing at the offending line in the design plan. The implementation uses `thiserror` for enum derivation and `miette::Diagnostic` for structured annotations, enabling both IDE-quality rendering and programmatic handling in CI pipelines.

Design principles. Three principles guide the error model:

1. **Structured codes.** Every variant carries a hierarchical diagnostic code (e.g. `netcfg::nte_ql::syntax`, `netcfg::primitive::validation`). CI scripts can match on code prefixes without parsing free-text messages.
2. **Source provenance.** Parse-time errors embed `miette::SourceSpan` labels so that editors and the LSP can render caret-style diagnostics with line and column precision.
3. **Causal chaining.** Low-level errors (`SelectorError`, `ShadowError`) propagate upward via `#[from]` conversions into `PrimitiveError` and finally `ExecuteError`, preserving the full causal chain without losing context.

Error taxonomy. Table 16 lists the nine error enums with their variants and originating pipeline stage.

Table 16. Error enums and their variants.

Enum	Variants	Context
`ParseError`	`YamlSyntax`, `Validation`, `Deserialize`, `Io`, `ImportCycle`, `LayerOrdering`, `CircularGroupReference`, `UndefinedGroupReference`, `ZoneMissingTrustLevel`, `ZoneTrustLevelOutOfRange`, `InterfaceSelectorOnNonZone`, `UnknownAddressObject`, `UnknownServiceObject`, `InvalidCidr`, `EmptyAddressObject`, `EmptyServiceObject`	design plan parsing
`SelectorError`	`InvalidSyntax`, `Evaluate`, `NonBooleanResult`, `EmptyResult`, `TopologyAccess`	selector compilation and evaluation
`PrimitiveError`	`Validation`, `Execute`, `Selector`, `Topology`, `Graph`, `Polars`, `Json`	primitive execution
`ExecuteError`	`Parse`, `Primitive`, `Shadow`, `Plan`, `Assertion`	top-level executor
`ShadowError`	`Clone`, `Commit`, `Validation`, `NodeNotFound`, `DataframeMissing`, `Polars`, `Json`	shadow topology lifecycle
`AssertionError`	`Failed`, `Selector`	assertion evaluation
`DiffError`	`TopologyAccess`, `ParseError`	configuration diffing
`RenderError`	`Io`, `Jinja`, `Json`, `MissingTemplate`	template rendering
`TelemetryError`	`NodeNotFound`, `PathNotFound`, `Internal`	telemetry integration

Error propagation. Errors flow upward through three tiers, as shown in Figure 44. The outermost `ExecuteError` is the only type that CLI commands and the REST API surface to callers; inner types are wrapped automatically via `#[from]` conversions.

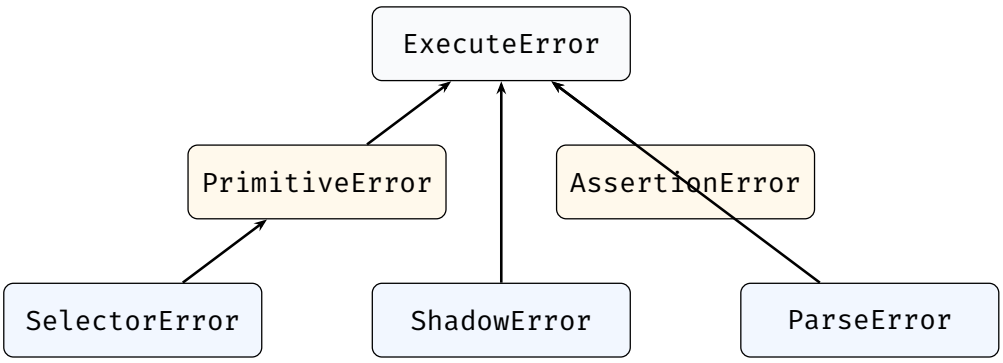


Figure 44. Error propagation hierarchy. Leaf errors are wrapped into mid-level errors, which are in turn wrapped into the top-level `ExecuteError` returned by the pipeline.

Representative error output. The following shows a typical `miette`-rendered error when a selector references a non-existent topology property:

```
x netcfg::primitive::execution_failed

Primitive execution failed in layer 'underlay': mesh_nodes:
  Evaluation error in selector 'MATCH (n) WHERE n.datacenter == "dc1"':
  variable 'datacenter' not found in node properties

help: Available node properties: hostname, role, site, vendor, os
      Check spelling or add the property to the topology file.
```

Recovery guidance. Each diagnostic code maps to a recovery strategy. `netcfg::parse::yaml_syntax` errors include the line number and column, directing the operator to the exact YAML token. `netcfg::nte_ql::syntax` errors report the CEL compilation failure and suggest checking the NTE-QL grammar reference (Appendix J). `netcfg::primitive::validation` errors indicate a constraint violation (e.g. a missing `selector` field) and list the required fields for the primitive type. `netcfg::shadow::error` errors indicate a topology integrity failure during the transactional shadow phase—typically a programming error rather than an operator mistake.

F Editor and Service Integration

F.1 Language Server (LSP)

The `netcfg-lsp` crate provides a Language Server Protocol implementation built on `tower-lsp` and Tokio. It communicates via stdin/stdout and offers three capabilities:

1. **Real-time diagnostics.** On file open, change, and save, the server parses the design plan and maps errors to LSP diagnostics with line-column precision. Text parse errors (which embed location as free text) are converted to line/column positions via pattern matching. Diagnostics are published with source label `ank_netcfg` and severity `ERROR`.
2. **Primitive auto-completion.** When the cursor follows a `type:` token (detected by a suffix heuristic), the server offers completion items for all twenty-three primitives, each with a one-line docstring. Trigger characters are space and colon.
3. **Document synchronisation.** Open documents are tracked in a `DashMap<String, String>` (lock-free concurrent map). Full-text sync (`TextDocumentSyncKind::FULL`) is used for correctness. On close, the document is removed and diagnostics are cleared.

F.2 REST API Microservice

The `netcfg-api` crate exposes the synthesis engine as a stateless HTTP service via `axum`, enabling CI/CD pipelines and enterprise orchestrators to compile design plans without a local `netcfg` binary.

Endpoint: `POST /compile` (port 3000).

Table 17. Compile endpoint request and response schema.

Direction	Field	Type	Description
Request	`topology_zip_base64`	String	Base64-encoded `.nte` archive
Request	`design_plan_yaml`	String	Raw Design Plan
Response	`status`	String	`success` or `error`
Response	`configs`	Map<String, String>	Hostname → rendered config
Response	`device_ir`	Map<String, JSON>	Hostname → DeviceContext IR
Response	`warnings`	Vec<String>	Compilation warnings

The service executes in a strict sandbox mode: `inject_secrets` and `wasm_plugin` primitives are disabled by default to prevent remote exfiltration of environment variables. The compilation pipeline decodes the base64 payload into a temporary file, loads the topology, parses the design plan, executes via `Executor::apply()`, and extracts per-device IR. All processing is in-memory with no persistent state; the service scales horizontally behind a load balancer.

G Discussion

While NetCfg achieves a robust declarative compilation pipeline, the architecture contains several structural trade-offs and areas for future research.

G.1 Type Safety and The Two-Tier Model

The codebase currently exhibits a dual-tier compilation path. The "typed tier" (`SecurityModel`, `QosModel`) is backed by rigorous Rust structs, enabling shadow detection, traffic simulation, and zone coverage analysis. Conversely, the "untyped tier" (`build_routing_policy`, `build_access_policy`, `build_prefix_list`) produces opaque JSON stanzas. These untyped structures bypass pre-flight validation. Converging the untyped tier into a unified, strictly typed Intermediate Representation (IR) is a prerequisite for holistic end-to-end reachability verification.

Additionally, the semantic duality at the template boundary (§C.1) defeats some typed validation. While `TargetDeviceModel.stanzas` is heavily validated during construction, it is discarded in favour of untyped `Stanza` bags immediately prior to serialisation.

G.2 Edge Properties

The `mesh_nodes` primitive constructs connectivity edges but cannot attach intrinsic properties (e.g., cost, delay, bandwidth, BFD requirement) directly to those edges. This architectural limitation blocks per-link traffic engineering and prevents assertions like "all critical edges must have BFD enabled." Future resolution depends on native edge DataFrame support in the underlying NTE topology engine.

G.3 The Simulation Gap (Local vs Global)

The current policy verifier executes a highly efficient $O(R^2)$ simulation, but it is fundamentally a *single-device* operation. There is an architectural gap between local policy simulation and cross-device global reachability. Verifying that a firewall's `SecurityModel` permits a flow does not guarantee that a Layer 3 path exists, that intermediate ACLs permit the traffic, or that routing policy correctly propagates the return path. Bridging this simulation gap requires composite reachability algorithms that merge graph traversal with distributed access list evaluation.

G.4 Ordering Enforcement

Primitives currently execute in YAML declaration order. The pipeline executor performs no topological sorting and does not mandate that dependency constraints are satisfied. If a Design Plan declares `provision_ips` before `mesh_nodes`, the allocator will fail silently by finding no edges to process. A deterministic compiler should enforce topological sorting based on the known dependency DAG (Topology → Allocation → Protocol → Policy) rather than relying on the operator's manual ordering.

G.5 Evolution of the Endpoint Model

Early versions of the NetCfg data model allocated IP addresses as scalar fields directly onto topology edges (`src_ip`, `dst_ip`). While sufficient for point-to-point links, this naive approach failed under the complexities of multi-homing, loopbacks, and Anycast gateways. The architecture evolved to utilize the robust `InterfaceEndpoint` model, which decouples logical connectivity from physical interfaces. This refactoring allowed the system to model complex overlay constructs, like EVPN-VXLAN VTEPs, without violating the strict graph traversal constraints required by the synthesis engine.

H Conclusion and Future Work

NetCfg demonstrates that network configuration can be treated as a pure compilation problem. By elevating the multi-abstraction-layer topology (MALT) to the root data structure and manipulating it via a formal, typed query language (NTE-QL), the system synthesises correct-by-construction configurations across multi-vendor networks.

The following research and engineering directions extend this foundation:

- **WASM Extensibility:** Replace the current experimental host–guest interface with a stable C-ABI memory allocator, enabling third-party primitives to manipulate the topology graph via WebAssembly.
- **Incremental Compilation:** Hash each topology layer and design-plan block; re-execute only those primitives whose inputs have changed, significantly reducing processing time for massive topologies.
- **Runtime Drift Detection:** Connect a production `TelemetryProvider` (gNMI, RESTCONF) to the assertion engine, closing the loop between design-time generation and continuous runtime validation.

I Related Work

Five research strands address network configuration management. We contrast each with NetCfg's approach, ordered from research lineage (synthesis, verification) through industrial neighbours (operational automation, infrastructure-as-code) to data models (schema standardisation).

I.1 Configuration Synthesis

Propane [26] compiles network-wide routing policy written in a high-level language into distributed BGP configurations that are correct by construction. Its successor, Propane/AT [35], extends this to abstract topologies, enabling synthesis that remains valid as the network evolves. SyNET [36] takes a complementary approach, encoding the semantics of multiple routing protocols in stratified Datalog and reducing configuration synthesis to SMT constraint solving; NetComplete [27] extends this line of work to practical, heterogeneous networks. Jinjing [28] targets ACL configuration repair at Alibaba's global WAN, using verification-guided synthesis to guarantee correctness of incremental updates. These systems share an emphasis on *protocol-specific* correctness guarantees. NetCfg occupies a different point in the design space: it synthesises *full device configurations*—spanning IP allocation, protocol sessions, QoS policies, and vendor-specific stanzas—from an annotated topology graph rather than from a protocol-centric specification language. Where Propane proves BGP policy correctness and SyNET solves for OSPF weights, NetCfg treats the topology itself as the specification and delegates per-protocol invariants to its assertion engine (§G).

I.2 Configuration Verification

Batfish [29] builds a vendor-neutral data-plane model from device configurations and answers reachability, loop-freedom, and compliance queries without access to live devices. Minesweeper [30] translates configurations into SMT formulae, enabling exhaustive verification of properties such as waypointing and fault tolerance across all possible environments. Header Space Analysis [31] pioneered static checking of the forwarding plane by modelling headers as points in a geometric space. These tools operate *post hoc*: they analyse configurations that already exist. NetCfg is complementary—it generates the configurations that verification tools consume. Indeed, the technical

report's own discussion on runtime drift detection (§H) recommends Batfish for runtime compliance checking. By embedding design-rule assertions directly in the synthesis pipeline, NetCfg shifts a class of structural invariants (minimum connectivity, field presence, protocol-session symmetry) from post-hoc verification to compile-time enforcement.

I.3 Operational Automation

Ansible [7], Nornir [8], and NAPALM [37] form the operational backbone of many network automation workflows. Ansible provides agentless, YAML-driven task execution; Nornir offers a pure-Python, multi-threaded alternative with richer programmatic control; NAPALM abstracts vendor differences behind a uniform Python API. However, all three are fundamentally *imperative task runners*; they execute sequential commands against individual devices but possess no intrinsic model of the network's topology. Cross-device relationships, such as guaranteeing that a Spine's BGP neighbour IP matches a Leaf's interface IP, must be manually stitched together using external inventory scripts or brittle variable files.

NetCfg bypasses these limitations by elevating the *layered graph* to the primary unit of computation. NetCfg is declarative and operates upstream of these tools—it acts as the synthesis engine that produces the deterministic, mathematically proven configuration artefacts that an Ansible playbook or Nornir task would then blindly push to the hardware.

I.4 Infrastructure as Code

Terraform [38] and Pulumi [39] popularised declarative infrastructure provisioning for cloud resources. Terraform's HCL and Pulumi's general-purpose language bindings both maintain a *desired-state* model with plan/apply semantics and state tracking. When applied to networking, these tools manage cloud-native constructs (VPCs, route tables, load balancers) but operate at the independent resource level rather than the topological level. They lack the domain-specific abstractions that physical network design demands: they have no mechanism for evaluating path-based constraints, executing graph traversals, or allocating resources (like subnets) dynamically across a mesh of interconnected nodes.

NetCfg borrows the plan/apply workflow (its `ci-run` and structural diff engine mirror Terraform's `plan` command) but embeds network-specific compilation primitives and a graph-centric data model that general-purpose IaC tools do not provide.

I.5 Schema Standardisation

OpenConfig [4] and YANG [40] define vendor-neutral, machine-readable schemas for individual device configuration and operational state. OpenConfig models emphasise a consistent per-device structure covering interfaces, routing protocols, and telemetry, and have achieved broad multi-vendor adoption. However, these schemas describe *single-device* state: they do not capture cross-device relationships ("spine S1 peers with every leaf in rack R") or support graph-wide operations such as IP pool allocation across a fabric. NetCfg's topological ontology (§1) fills this gap by modelling the network as a multi-layer graph whose annotations encode the very relationships that OpenConfig leaves implicit. The two are complementary: NetCfg can target OpenConfig-structured DeviceIR stanzas, using YANG-modelled paths as the leaf-level schema within its vendor-neutral intermediate representation.

I.6 Topology Modelling

MALT [3] introduces multi-abstraction-layer topology modelling at Google, using approximately 250 entity-kinds across five layers (physical/spatial, L1 optical, L2 switching, L3 routing, logical/abstract) connected by typed relationship-kinds (`RK_AGGREGATES`, `RK_TRAVERSES`, `RK_REALIZED_BY`, `RK_CONTAINS`). MALT's key design principles—orthogonality of entity-kinds, separation of aspects, and profile-based constraints—directly influenced NetCfg's layered topology model. NetCfg adopts a simpler two-field parent mapping (`_source_id`, `protocol_layer`) rather than MALT's full relationship-kind taxonomy, trading expressiveness for implementation simplicity in the current prototype.

Statesman [41] addresses network state management complexity at Facebook. Its contribution—a declarative state model with conflict detection and resolution—is complementary: NetCfg generates the desired state that a system like Statesman would manage at runtime.

I.7 Policy Specification and Verification

Beyond the synthesis and verification tools discussed above, several systems focus specifically on declarative policy specification. Zen [42] applies program synthesis to network intent, translating high-level policy specifications into concrete configurations via a "sketch and fill" approach. The Declarative Policy section (§5) surveys additional related work in algebraic policy composition (PGA [25], Pyretic [24]), synthesis-based policy generation (Merlin [32]), and zone-based access control models (Denning [34], Sandhu [33]).

I.8 Graph Query Languages

NTE-QL's design draws on the graph query language literature. Cypher [20], originating from Neo4j, established the ASCII-art pattern syntax that NTE-QL borrows for its surface form. GQL [21] (ISO/IEC 39075:2024) is the first new ISO database language since SQL, unifying Cypher, PGQL, and G-CORE with quantified path patterns and multiple named graphs—features that Appendix J.8 identifies as natural extensions for NTE-QL. Angles et al. [19] provide the expressiveness hierarchy (BGP/RPQ/CRPQ/ECRPQ) that §J.6 uses to characterise NTE-QL's position in the design space and the cost of future extensions.

Positioning. Table 18 summarises the landscape informally. Verification tools (Batfish, Minesweeper) check existing configurations; synthesis tools (Propane, SyNET) generate provably correct protocol fragments; automation frameworks (Ansible, Nornir) push configurations to devices; IaC tools (Terraform, Pulumi) manage cloud-native resources; and schema efforts (OpenConfig, YANG) standardise per-device data models. NetCfg is, to our knowledge, the first open-source system that *compiles* a declarative, graph-centric design plan into complete, multi-vendor device configurations via a typed intermediate representation, occupying the space between high-level intent and device-level rendering that the other strands leave unfilled.

Table 18. Informal positioning of NetCfg relative to related work.

System	Scope	Topology-Aware	Multi-Vendor	Full Device Config
Propane / SyNET	Protocol fragment	✓	—	—
Batfish	Verification	✓	✓	—
Ansible / Nornir	Push automation	—	✓	✓
Terraform / Pulumi	Cloud IaC	—	✓	—
OpenConfig / YANG	Device schema	—	✓	—
NetCfg	**Synthesis**	✓	✓	✓

J NTE-QL Reference

NTE-QL (NTE Query Language) is NetCfg's graph query language for selecting node and edge populations. It uses a Cypher-inspired surface syntax that compiles to the Common Expression Language (CEL) for evaluation. This appendix specifies the full grammar, operators, and evaluation model.

J.1 Architecture

NTE-QL is *not* a Cypher implementation. The translation pipeline is:

1. **Surface syntax** — the operator writes a Cypher-like `MATCH...WHERE` clause or a compact shorthand (`nodes[predicate]`).
2. **Shorthand normalisation** — single operators (`=`, `&`, `|`) are rewritten to their CEL equivalents (`==`, `&&`, `||`).
3. **CEL compilation** — the normalised expression is compiled to a CEL `Program` via `Program::compile()`.
4. **Context evaluation** — for each candidate node or edge, a CEL context is populated with topology variables and the program is executed; nodes/edges for which the result is `true` are selected.

J.2 Grammar

Listing 17. NTE-QL grammar (EBNF).

```
selector      = node_query | edge_query | shorthand | all
node_query    = "MATCH" "(" ident ")" "WHERE" expression
edge_query    = "MATCH" "()" "-[" ident "]->" "()" "WHERE" expression
shorthand     = ("nodes" | "edges") "[" expression "]"
all           = "all()"        (* normalises to nodes[true] *)

expression    = cel_expression
              (* any valid CEL expression over the bound variables *)
```

The `MATCH...WHERE` forms are syntactic sugar—they are parsed into `nodes[expression]` or `edges[expression]` before CEL compilation. The identifier in parentheses (e.g. `n`, `e`) is currently discarded; all variable references resolve against the evaluation context directly.

J.3 Operators

Table 19. NTE-QL operators. Single-character shorthands are normalised before CEL compilation.

Category	Operator	Notes
Comparison	`==`, `!=`, `<`, `>`, `<=`, `>=`	Single `=` is auto-normalised to `==`
Logical	`&&`, `||`, `!`	Single `&` → `&&`, single `|` → `||`
Membership	`in`, `.contains()`	CEL list operations
String	`.startsWith()`, `.endsWith()`, `.matches()`, `+`	CEL string methods and concatenation
Type	`string()`, `int()`, `has()`	CEL type casting and field existence

Operator normalisation respects quoted strings: the character `=` inside a string literal (e.g. `'name=foo'`) is not rewritten.

J.4 Evaluation context

Node context. For each candidate node, the following variables are bound:

Table 20. Variables available in node selector expressions.

Variable	Type	Source
`id`	Int	Topology node ID
`type`	String	Node type (e.g. `router`, `switch`)
`layer`	String	Layer name (e.g. `physical`, `bgp`)
`degree`	Int	Number of edges incident on this node
`is_leaf`	Bool	`degree == 1`
`is_spine`	Bool	`degree > 2`
`neighbors`	List<Int>	Adjacent node IDs in the current layer
`all_layer_data`	Map<String, Map>	Cross-layer property access (see below)
All metadata keys from the node's `data_json` are flattened into the root context		(e.g. `role`, `site`, `hostname`)

Edge context. Edge selectors bind the same cross-layer variables plus:

Table 21. Additional variables available in edge selector expressions.

Variable	Type	Source
`src`, `dst`	Int	Source and destination node IDs
`src_neighbors`, `dst_neighbors`	List<Int>	Neighbour lists of the endpoints
`src_degree`, `dst_degree`	Int	Degrees of the endpoints
`src_is_leaf`, `dst_is_leaf`	Bool	Whether each endpoint is a leaf

Cross-layer queries. The `all_layer_data` variable is a nested map of type `Map<LayerName, Map<NodeID, Properties>>`. This enables queries that reference properties from layers other than the one being filtered:

```
# Select physical nodes whose BGP counterpart has ASN 65000
selector: "nodes[all_layer_data['bgp'][string(id)].asn == 65000]"
```

Strict mode. By default, references to undeclared properties evaluate to `false` rather than raising an error. Setting the environment variable `NETCFG_STRICT_NTE_QL=1` causes undeclared references to return a CEL compilation error, which is useful for catching typos during development.

J.5 Limitations

- **No multi-hop path traversal.** Cypher-style path patterns (`MATCH (n)-[e1]->(m)-[e2]->(p)`) are not supported. Multi-hop relationships are accessed via `neighbors` and `all_layer_data`.
- **No recursive patterns.** Cypher's variable-length path operator (`*`) is not supported.
- **No `RETURN` clause.** Selectors return node/edge IDs, not projected property bags. The `RETURN` keyword is parsed but currently ignored.

J.6 Formal Design Space

Graph query languages form a well-studied expressiveness hierarchy [19]. Understanding where NTE-QL sits in this hierarchy clarifies both its current capabilities and the cost of future extensions.

Table 22. Graph query language expressiveness hierarchy. NTE-QL currently operates at the *basic graph pattern* level (marked †). Each row strictly subsumes the one above it.

Language class	Abbreviation	Data complexity	Key capability
Basic graph pattern†	**BGP**	NLogSpace	Node/edge property predicates; subgraph matching
Regular path query	RPQ	NLogSpace	Reachability via edge-label regex ($a \xrightarrow{L^*} b$)
Two-way RPQ	2RPQ	NLogSpace	RPQ + inverse navigation ($a \xleftarrow{L^*} b$)
Conjunctive RPQ	CRPQ	NP-complete	Multiple simultaneous path constraints
Extended CRPQ	ECRPQ	NP-complete	CRPQ + data value comparisons along paths

NTE-QL's current implementation is a *basic graph pattern* language: each selector evaluates a CEL predicate over a single node or edge at a time, with no path traversal. The `neighbors` and `all_layer_data` variables provide limited structural awareness but do not constitute path navigation in the formal sense.

Moving to RPQ expressiveness—the minimum needed for reachability assertions—requires only BFS/DFS over the topology graph and remains in NLogSpace. This is the natural next step for NTE-QL (see §J.8). CRPQ-level queries ("A reaches B via OSPF *and* via BGP simultaneously") enter NP-complete territory but remain tractable for the topology sizes typical of NetCfg deployments (tens to low thousands of nodes).

Relationship to GQL and Cypher. NTE-QL borrows Cypher's ASCII-art surface syntax [20] but compiles to CEL rather than implementing Cypher's pattern-matching semantics. Key divergences from Cypher/GQL [21]:

1. **No pattern morphism semantics.** Cypher distinguishes isomorphism and homomorphism for pattern matching; NTE-QL evaluates predicates independently per entity.
2. **No variable binding across patterns.** Cypher's `MATCH (a)-[r]->(b)` binds three variables simultaneously; NTE-QL binds only one entity (node or edge) per evaluation.
3. **No quantified path patterns.** GQL's quantifiers (`{2,5}`, `+`, `*`) are not available.
4. **CEL as the predicate language.** Where Cypher uses its own expression language, NTE-QL delegates to CEL, inheriting CEL's type system, string methods, and list operations.

These divergences are deliberate: NTE-QL prioritises evaluation simplicity and CEL interoperability over full graph pattern matching. The `MATCH...WHERE` form is currently syntactic sugar for the `nodes[predicate]` shorthand; the identifier in parentheses (e.g. `n`) is discarded during compilation.

J.7 Quantification Patterns for Network Assertions

Network design invariants are naturally expressed as quantified predicates over the topology graph. Table 23 maps common assertion patterns to their formal quantification and current NTE-QL support.

Table 23. Quantification patterns for network assertions. "Direct" means the assertion engine supports it natively; "indirect" means it requires decomposition into supported primitives.

Assertion pattern	Quantifier	Support	NTE-QL expression
Every spine has ≥ 2 leaf links	$\forall x \in S.\ \lvert\{y \in L : x \sim y\}\rvert \geq 2$	Direct	`min_edges: 2` on selector
Some path exists from a to b	$\exists \pi : a \rightsquigarrow b$	**Missing**	Proposed: `reachable(a, b)`
All nodes in group G have field f	$\forall x \in G.\ f(x) \neq \bot$	Direct	`field_exists` assertion
No two nodes in G share field f	$\forall x, y \in G.\ x \neq y \Rightarrow f(x) \neq f(y)$	Direct	`unique_per_group`
Field f is within CIDR C	$\forall x \in G.\ f(x) \in C$	Direct	`field_in_cidr`
Graph G is connected	$\forall x, y \in G.\ x \rightsquigarrow y$	Direct	`is_connected`
No external node reaches restricted	$\forall x \in E,\ y \in R.\ x \not\rightsquigarrow y$	**Missing**	Proposed: `unreachable(E, R)`
All zone pairs have a policy	$\forall z_i, z_j.\ \exists P(z_i, z_j)$	Direct	`zone_pairs_covered`
CEL predicate holds	$\forall x \in G.\ \varphi(x)$	Direct	`use_rule` with CEL macro

The universal quantification problem. Graph query languages are existential by default: `MATCH` finds entities *where* a pattern holds. Universal quantification ("for *all* nodes…") must be expressed either via double negation ($\forall x.\ P(x) \equiv \neg\exists x.\ \neg P(x)$) or via aggregation (`count{...} == total`). NTE-QL's assertion engine sidesteps this by applying selectors as universal quantifiers: the `field_exists` assertion implicitly means "for every node matched by the selector, the field exists." This convention is ergonomic but should be made explicit in the language specification to avoid confusion when operators compose assertions with group algebra operators (see §5.2).

Existential vs. universal reachability. Reachability assertions come in two flavours:

Existential "there exists a path from a to b" — decidable in $O(|V| + |E|)$ via BFS/DFS.

Universal "for all nodes in A, there exists a path to some node in B" — requires $|A|$ individual reachability checks, each $O(|V| + |E|)$, giving $O(|A| \cdot (|V| + |E|))$.

For all-pairs reachability (every node reaches every other node), transitive closure via Floyd–Warshall runs in $O(|V|^3)$, which is tractable for topologies up to several thousand nodes.

J.8 Recommended Extensions

Based on the design space analysis, we recommend four extensions to NTE-QL, ordered by implementation priority and increasing expressiveness:

Extension 1: Reachability predicates (RPQ). Add built-in reachability predicates that compute BFS/DFS over the topology graph:

Listing 18. Proposed reachability syntax for NTE-QL.

```
# Existential: some path exists
reachable("nodes[role == 'spine']", "nodes[role == 'leaf']")

# Negative: no path exists
unreachable("nodes[zone == 'external']", "nodes[zone == 'restricted']")

# Bounded: path within N hops
reachable("nodes[hostname == 'core-01']",
          "nodes[hostname == 'edge-01']", max_hops: 3)
```

```
# Layer-constrained: path must traverse a specific layer
reachable($src, $dst, layer: "ospf")
```

Complexity: $O(|V| + |E|)$ per source–destination pair; remains in NLogSpace. For population-level assertions ($\forall s \in S. \exists\, d \in D.\ s \rightsquigarrow d$), total cost is $O(|S| \cdot (|V| + |E|))$.

Extension 2: Path property constraints (ECRPQ). Allow predicates over intermediate nodes and edges along the path:

Listing 19. Proposed path constraint syntax.

```
# All links on the path have cost < 10
reachable($src, $dst, where: "edge.cost < 10")

# Path must traverse a firewall node
reachable($src, $dst, via: "nodes[role == 'firewall']")
```

Complexity: Constrained-path reachability is NP-complete in the general case (CRPQ) but tractable when constraints are monotone (filtering) rather than conjunctive (multiple simultaneous paths). The `where` variant is a filtered BFS ($O(|V| + |E|)$); the `via` variant requires checking that all paths pass through the waypoint, which is equivalent to a min-cut computation ($O(|V| \cdot |E|)$).

Extension 3: Population operators (group algebra). Expose Boolean set operations on selector results, connecting NTE-QL to the group algebra defined in §5.2:

Listing 20. Proposed population operators.

```
# Union: spines OR leaves
selector: "nodes[role == 'spine'] | nodes[role == 'leaf']"

# Intersection: spines AND dc1
selector: "nodes[role == 'spine'] & nodes[site == 'dc1']"

# Difference: all core except those in maintenance
selector: "nodes[role == 'core'] - nodes[maintenance == true]"
```

Complexity: $O(n)$ per set operation; purely syntactic sugar over the underlying selector evaluation.

Extension 4: Quantified path patterns (GQL QPP). Track GQL's quantified path pattern syntax for variable-length traversals:

Listing 21. GQL-style quantified path patterns (future).

```
# All spines reachable from leaves within 3 hops
MATCH (l:Leaf)-/[:LINK]{1,3}/->(s:Spine)

# Recursive: find all nodes reachable via OSPF adjacencies
MATCH (src)-/[:OSPF_ADJ]+/->(dst)
```

This extension requires implementing path-pattern compilation in the query engine, a significantly larger effort than Extensions 1–3. GQL QPPs are the natural long-term target for NTE-QL's evolution but are not required for the current assertion vocabulary.

Table 24. Summary of recommended NTE-QL extensions with complexity and implementation priority.

#	Extension	Language class	Complexity	Priority
1	Reachability predicates	RPQ	$O(\|V\| + \|E\|)$	High — required for cross-device policy verification
2	Path property constraints	ECRPQ	$O(\|V\| \cdot \|E\|)$	Medium — enables waypoint and cost assertions
3	Population operators	BGP	$O(n)$	Medium — ergonomic improvement for group composition
4	Quantified path patterns	GQL QPP	NP-COMPLETE	Low — long-term GQL alignment

K Schema Catalogue

This appendix catalogues the typed schemas enforced by NetCfg's validation engine, the well-known annotation vocabulary, the five design plan concerns, the primitive alias table, and the full field specifications for all twenty-three primitives.

Each schema table includes a **Status** column with the following legend:

✓ Field exists and is functional in the codebase.

✓△ Field exists but has a known type issue (e.g. `String` where a validated enum is needed).

✗ Documented requirement not yet implemented; required for production deployments.

K.1 Annotation vocabulary

Table 25. Well-known topology annotation keys. Keys marked with ⋆ are computed by the selector engine and need not appear in the topology file. All other keys are authored by the operator (or imported via `netcfg sync pull`).

Key	Type	Consumed by
`hostname`	string	All primitives (node identity)
`role`	string	Selectors (`MATCH (n) WHERE n.role ='spine'`)
`site`	string	Selectors, multi-site design plans
`device_os`	string	Template selection, vendor transforms, QoS constraints
`management_ip`	string	Inventory; passed through to templates
`zone`	string	`build_zone_policy`, zone-based assertions
`platform`	string	Hardware inventory transforms
`vrf`	string	`provision_ips` (routing context separation)
`disabled`	bool	Selectors (exclude nodes from processing)
Computed selector variables ⋆		
`id`	integer	Node identifier in the graph
`layer`	string	Current graph layer (e.g. `physical`, `bgp`)
`degree`	integer	Number of adjacent edges
`is_leaf`	bool	True when `degree == 1`
`neighbors`	list	Adjacent node identifiers

K.2 Primitive naming aliases

Table 26. Primitive canonical names and DSL aliases.

Canonical Identifier	Design Plan Alias	Tier
`provision_resources`	`allocate_resources`	Allocation
`define_ip_pool`	`global_pool`	Allocation
`define_resource_pool`	`global_resource_pool`	Allocation
`policy_routing`	`build_routing_policy`	Policy
`policy_access`	`build_access_policy`	Policy
`policy_prefix_list`	`build_prefix_list`	Policy
`policy_community_list`	`build_community_list`	Policy
`policy_bgp_safety`	`generate_safe_bgp_filters`	Policy
`overlay_vxlan`	`build_vxlan`	Overlay
`overlay_evpn`	`build_evpn`	Overlay

K.3 Primitive catalogue tables

Table 27. Design rules for Policy & Resource Allocation.

Design Rule	Category	Purpose
`provision_ips`	Allocation	Allocate IP subnets to edges from a CIDR pool
`allocate_resources`	Allocation	Generic pool allocation for ASNs, VLANs, etc.
`global_pool`	Allocation	Register a named IP pool for hierarchical carving
`global_resource_pool`	Allocation	Register a named resource pool (non-IP)
`build_routing_policy`	Policy	Attach routing policy stanzas (route-maps)
`build_access_policy`	Policy	Attach access-control policy stanzas (ACLs)
`build_prefix_list`	Policy	Create named prefix-list entries on matched nodes
`build_community_list`	Policy	Create named community-list entries on matched nodes
`generate_safe_bgp_filters`	Policy	Auto-generate RFC 1918 bogon filters
`build_zone_policy`	Policy	Zone-based firewall policies with ordered rules
`build_nat_policy`	Policy	NAT policies (SNAT interface/pool, DNAT port-forward)
`build_qos_policy`	Policy	QoS policies (classifier, scheduler, policer, shaper)

Table 28. Design rules for Overlays & Metadata.

Design Rule	Category	Purpose
`build_vxlan`	Overlay	Assign VXLAN VNI and multicast group bindings
`build_evpn`	Overlay	Assign EVPN route-distinguisher and route-target
`tag_nodes`	Meta	Stamp key–value metadata onto matched nodes
`map_hardware_inventory`	Meta	Assign chassis model and line-card slot mappings
`assert_state`	Meta	Mid-pipeline assertion checkpoint
`conditional`	Meta	Query-based branching within design plans
`custom_primitive`	Meta	Named reusable design-rule group with parameters
`inject_secrets`	Meta	Read environment variables into node metadata
`wasm_plugin`	Meta	Execute a WebAssembly plugin against matched nodes

K.4 Primitive field specifications

K.4.1 `BfdNodeSchema` (typed overlay schema for BFD)

Field	Requirement	Semantics
`interface`	Yes	Interface on which BFD is enabled
`interval_ms`	No	Transmit interval in milliseconds
`min_rx_ms`	No	Minimum receive interval in milliseconds
`multiplier`	No	Detect multiplier (default 3)
`multihop`	No	Single-hop (`false`) vs. multi-hop (`true`)

K.4.2 `MulticastNodeSchema` (typed overlay schema for Multicast)

Field	Requirement	Semantics
`group_address`	Yes	Multicast group address (e.g. `239.1.1.1`)
`source`	No	Source address for Source-Specific Multicast (SSM)
`protocol`	No	Protocol variant: `pim_sm`, `pim_ssm`, `igmp`, etc.
`rp_address`	No	Rendezvous point address for PIM-SM

K.4.3 `allocate_resources`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for nodes
`resource_type`	Yes	Key name in `DeviceContext.metadata`
`pool`	Yes	Range (e.g. `65000-65999`) or global pool name
`strategy`	No	`dense` (default), `sparse`, or `hash`
`pool_size`	No	When referencing a global pool, number of values to carve

K.4.4 `global_pool`

Parameter	Requirement	Architectural Intent
`name`	Yes	Pool identifier referenced by `provision_ips`
`pool`	Yes	CIDR range (e.g. `10.0.0.0/8`)
`vrf`	No	VRF domain (default: `default`)

K.4.5 `global_resource_pool`

Parameter	Requirement	Architectural Intent
`name`	Yes	Pool identifier
`resource_type`	Yes	Resource category (e.g. `asn`)
`pool`	Yes	Value range (e.g. `65000-65999`)

K.4.6 `conditional`

Parameter	Requirement	Architectural Intent
`condition`	Yes	NetCfg query expression
`then_primitives`	Yes	Primitives if condition is true
`else_primitives`	No	Primitives if condition is false

K.4.7 `custom_primitive`

Parameter	Requirement	Architectural Intent
`name`	Yes	Primitive name (for reporting)
`parameters`	Yes	Named parameters
`primitives`	Yes	Nested primitive list

K.4.8 `inject_secrets`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`secrets`	Yes	{metadata_key: env_var_name}

K.4.9 `build_routing_policy`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`policy_name`	Yes	Policy name (e.g. `BGP-EXPORT`)
`statements`	Yes	Ordered route-map entries

Each `RoutingPolicyStatement` has fields: `name` (sequence label), `action` (`permit` or `deny`), and optional match/set clauses: `match_prefix_list`, `match_community_list`, `match_as_path`, `set_local_preference` (integer), `set_metric` (integer), `set_as_path_prepend`, `set_community`, and `next_hop` (IP address).

K.4.10 `build_access_policy`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`policy_name`	Yes	Policy name (e.g. `EDGE-ACL`)
`rules`	Yes	Ordered ACL entries

Each `AccessPolicyRule` has fields: `name`, `action` (`permit`/`deny`), optional `source_prefix` (CIDR), `destination_prefix` (CIDR), `protocol` (string), `source_port` (integer), and `destination_port` (integer). Both policy primitives enforce node targeting—policies cannot target edges.

K.4.11 `wasm_plugin`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`plugin_path`	Yes	Filesystem path to compiled `.wasm` binary

K.4.12 `build_prefix_list`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`prefix_list_name`	Yes	Name of the prefix list
`entries`	Yes	Ordered prefix-list entries

Each `PrefixListEntry` has: `prefix` (CIDR string), `action` (`permit`/`deny`), optional `le` (less-than-or-equal prefix length), and optional `ge` (greater-than-or-equal prefix length).

K.4.13 `build_community_list`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`community_list_name`	Yes	Name of the community list
`entries`	Yes	Ordered community-list entries

Each CommunityListEntry has: community (e.g. 65001:1000) and action (permit/deny).

K.4.14 generate_safe_bgp_filters

Parameter	Requirement	Architectural Intent
selector	Yes	NetCfg query for target nodes
prefix_list_name	No	Name for the generated prefix list
policy_name	No	Name for the generated routing policy
block_rfc1918	No	Block RFC 1918 prefixes (default: true)
permit_default	No	Add a trailing permit-any (default: false)

K.4.15 build_zone_policy

Parameter	Requirement	Architectural Intent
selector	Yes	NetCfg query for target devices
from_zone	Yes	Source security zone name
to_zone	Yes	Destination security zone name
default_action	No	permit or deny (default: deny)
rules	Yes	Ordered list of match/action rules

Each rule has a name, an action (permit/deny), and an optional match block with fields: source, destination, service, protocol, destination_port, icmp_type. Optional log and stateful flags control logging and connection-tracking behaviour.

K.4.16 build_nat_policy

Parameter	Requirement	Architectural Intent
selector	Yes	NetCfg query for target devices
rules	Yes	Ordered list of NAT rules

Each NatRule has a name, a type (snat/dnat/nat64/nptv6), an optional match block, and a type-specific parameter block. SNAT rules specify mode (interface or pool) and optional pool_addresses. DNAT rules specify external_address, optional external_port, internal_address, and optional internal_port. NAT64 rules specify a nat64_prefix; NPTv6 rules specify internal_prefix, external_prefix, and an optional allow_best_effort flag. Rules may also carry from_zone/to_zone for zone-scoped NAT and an optional log flag.

The match block accepts four optional fields:

Table 29. NAT rule match fields (NatMatchSpec).

Parameter	Architectural Intent
source	Defines the source logical boundary (CIDR or Object Reference)
destination	Defines the destination logical boundary (CIDR or Object Reference)
service	Abstract service identifier mapping
protocol	L4 transit protocol definition (e.g. tcp, udp)

K.4.17 `build_qos_policy`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`policies`	No	Named QoS policies
`interfaces`	No	Per-interface policy bindings

A `QosPolicy` contains a map of named `TrafficClass` entries. Each traffic class may specify a `classifier` (matching on DSCP, CoS, protocol, addresses, or ports), a `marker` (rewriting DSCP/CoS), a `scheduler` (priority queuing, weighted fair queuing, bandwidth guarantees), a `policer` (CIR/burst enforcement), and a `shaper` (output rate limiting). An `InterfaceBinding` specifies an `input_policy` and/or `output_policy` name.

K.4.18 `build_vxlan`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for VTEP nodes
`vni_base`	Yes	Starting VNI (e.g. `10000`)
`mcast_group_base`	No	Multicast group for BUM replication

K.4.19 `build_evpn`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for EVPN nodes
`route_distinguisher_base`	Yes	RD base (e.g. `auto`)
`route_target_base`	Yes	RT base (e.g. `65001:10000`)

K.4.20 `tag_nodes`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`tags`	Yes	Key–value pairs to stamp

K.4.21 `map_hardware_inventory`

Parameter	Requirement	Architectural Intent
`selector`	Yes	NetCfg query for target nodes
`chassis_model`	Yes	Chassis identifier (e.g. `dcs-7508`)
`slots`	Yes	Slot → line-card model mapping

K.4.22 `assert_state`

Parameter	Requirement	Architectural Intent
`name`	Yes	Assertion name (for diagnostics)
`severity`	No	`error` (default) or `warning`
`select`	Yes	NetCfg query for items to check
`check`	Yes	Predicate (same types as top-level assertions)
`help`	No	Remediation hint shown on failure

K.5 Protocol overlay schemas

Four protocol overlays have strict schemas, enforced when `strict_overlay_schema` is enabled (the default). All other protocol layers accept arbitrary key–value configuration via the untyped `config` block. Normative references are RFC 2328/9129 (OSPF), RFC 4271 (BGP), ISO 10589/RFC 5305 (IS-IS), RFC 7432/8365 (EVPN), RFC 5880 (BFD), and the OpenConfig YANG models.

K.5.1 `OspfNodeSchema`

Field	Type	Required	Notes
`process_id`	u32	default 1	OSPF process ID
`area`	String	yes	OSPF area (e.g. `0.0.0.0`)
`hello_interval_s`	u32	—	Hello interval in seconds
`dead_interval_s`	u32	—	Dead interval in seconds
`priority`	u32	—	Router priority
`network_type`	String	—	Point-to-point, broadcast, etc.
`cost`	u32	—	Interface cost
`authentication`	String	—	Authentication method
`router_id`	String	—	Explicit router ID

Type issues: `area` and `network_type` are bare strings; should be validated types (dotted-quad/integer and enum respectively). `authentication` should be a tagged union (`none`/`simple`/`md5`).
Missing fields (RFC 9129): `area_type` (`normal`/`stub`/`nssa`), `passive_interfaces`, `default_cost`, `bfd_enabled`.

K.5.2 `BgpNodeSchema`

Field	Type	Required	Notes
`asn`	u32	yes	Autonomous System Number
`protocol_type`	String	—	`bgp` (default)
`ebgp_multihop`	u32	—	Multi-hop TTL
`router_id`	String	—	Explicit router ID
`timers`	Object	—	Nested: `keepalive` (u32), `holdtime` (u32)
`keepalive_s`	u32	—	Flat alternative to `timers.keepalive`
`hold_s`	u32	—	Flat alternative to `timers.holdtime`
`address_family`	String	—	`ipv4-unicast`, `ipv6-unicast`, etc.
`enable_graceful_restart`	bool	default false	Graceful restart
`restart_time_s`	u32	—	Restart timer in seconds

Known issue — timer duplication: `BgpNodeSchema` exposes timers via two parallel paths: a nested `timers: {keepalive, holdtime}` object and flat top-level fields `keepalive_s`/`hold_s`. No precedence rule is defined when both are supplied simultaneously.
Type issues: `address_family` is a single string; should be `Vec<AFI/SAFI>` (RFC 4760).
Missing fields (RFC 4271, OpenConfig): `peer_groups`, `authentication` (TCP-MD5), `route_reflector_client`, `local_as_override`, `bfd_enabled`.

K.5.3 `IsisNodeSchema`

Field	Type	Required	Notes
`net`	String	yes	Network Entity Title
`level`	String	yes	`level-1`, `level-2`, `level-1-2`
`metric_style`	String	—	`wide` or `narrow`
`hello_interval_s`	u32	—	Hello interval in seconds
`hold_time_s`	u32	—	Hold time in seconds
`lsp_lifetime_s`	u32	—	LSP lifetime in seconds
`enable_authentication`	bool	default false	IS-IS authentication

Type issues: `net` is a bare string; should validate OSI NET format (`xx.yyyy.yyyy.yyyy.00`). `level` and `metric_style` are strings; should be enums.
Missing fields (ISO 10589, RFC 5305): `overload_bit`, `attached_bit`, `priority`, `segment_routing_enabled` (RFC 8667).

K.5.4 `EvpnNodeSchema`

Field	Type	Required	Notes
`vni`	u32	yes	VXLAN Network Identifier
`route_target`	String	yes	BGP route target community

Type issues: `route_target` is a single string; should be separate `import`/`export` `Vec<String>` lists.
Missing fields (RFC 7432, RFC 8365): `evi_id`, `route_distinguisher` (typed enum), `route_target_import/export`, `encapsulation_type` (vxlan/mpls/nvgre), `vtep_ip`, `ethernet_segment_id`, `vlan_vni_mapping`. The EVPN schema is the most incomplete of the four strict overlays.

K.5.5 Non-strict protocol layers

Protocol layers not in the strict set (`ospf`, `bgp`, `isis`, `evpn`) accept arbitrary `config` blocks without schema validation. This includes the 31 protocol fragments shipped in `protocols/*.yaml` (BFD, multicast, STP, LACP, MLAG, IPsec, WireGuard, MACsec, NTP, Syslog, SNMP, etc.). Their configuration is passed through as untyped JSON and made available to mapping rules and templates.

K.6 Policy schemas

K.6.1 Zone policy

Field	Type	Notes
`select`	NTE-QL	Target node population
`from_zone`	String	Source security zone
`to_zone`	String	Destination security zone
`default_action`	Enum	`permit` or `deny` (default: `deny`)
`rules`	List	Ordered list of `ZonePolicyRuleSpec`

Each rule contains: `name` (String), `action` (`permit`/`deny`), and optional match fields: `source`, `destination` (CIDR or `$address_object` reference), `service` (`$service_object` reference), `protocol`, `destination_port` (u16), `icmp_type`, `log` (bool), `stateful` (bool).

K.6.2 NAT policy

Each NAT rule specifies a `type` field selecting the NAT mode:

NAT Type	Additional Fields
snat	mode (interface or pool), pool_addresses
dnat	external_address, external_port, internal_address, internal_port
nat64	nat64_prefix (RFC 6146)
nptv6	internal_prefix, external_prefix, allow_best_effort (RFC 6296)

All NAT types share optional match fields: source, destination, service, protocol, from_zone, to_zone, log.

K.6.3 QoS policy

QoS policies are structured as a map of named TrafficClass entries, each containing optional sub-models:

Component	Key Fields
classifier	dscp, cos, ip_proto, src_addrs, dst_addrs, src_ports, dst_ports
marker	dscp, cos
scheduler	priority (bool), weight, bandwidth_percent, queue_limit
policer	cir (u64), burst, exceed_action
shaper	shape_rate (u64), burst

K.7 Assertion check types

Table 30. Complete assertion check catalogue.

Check Type	Description
field_exists	Node has a non-null value for the named field
min_count	At least *n* nodes match the selector
max_count	At most *n* nodes match the selector
min_edges	At least *n* edges exist in the filtered topology
unique_per_group	A field value is unique within each group
field_in_cidr	An IP field falls within a CIDR range
custom_query	Arbitrary CEL expression evaluated per node
use_rule	Reference a named rule from rules:
is_connected	The filtered subgraph is connected
is_acyclic	The filtered subgraph contains no cycles
is_bipartite	The filtered subgraph is bipartite
reachability	Every selected node can reach the target selector
match_schema	Node properties conform to a JSON schema
zone_membership	Node belongs to a zone with minimum trust level
zone_policy_exists	A zone-pair policy exists with the expected action
no_shadowed_rules	No zone-policy rule is shadowed by a prior rule
zone_pairs_covered	Every declared zone pair has a policy
zone_policy_permits	A specific 5-tuple is permitted
zone_policy_denies	A specific 5-tuple is denied
telemetry_match	A telemetry path matches an expected value

K.8 Validation enforcement model

Validation is controlled by two flags in ValidationMode:

- `strict_topology_schema` — validates that the topology contains required DataFrames (`layers`, `node_types`) with expected columns (`id`, `layer`, `type`).
- `strict_overlay_schema` — validates `build_protocol_layer` config blocks against the typed schemas above. Enabled by default; unknown fields cause a parse error.

Both flags are passed to each primitive via `PrimitiveContext`, enabling per-primitive enforcement decisions. Semantic validation (group references, zone membership, object references, rule macros) runs at parse time before any primitives execute.

L Mapping DSL and TSDM Reference

This appendix specifies the Mapping DSL (Stage 2: DeviceIR Extraction) and the TSDM transform language (Stage 3: Target-Specific Lowering).

L.1 Mapping DSL

The Mapping DSL is a YAML document containing ordered rules that extract vendor-neutral **stanzas** from the enriched topology. It is parsed and evaluated by the upstream `ank_nte` crate.

L.1.1 Grammar

Listing 22. Mapping document structure.

```yaml
rules:
  - selector: "<NTE-QL expression>"
    stanza:
      kind: "<stanza type>"
      fields:
        <key>: "<value or Jinja2 expression>"
```

Table 31. Mapping rule fields.

Field	Type	Description
`selector`	NTE-QL	Selects nodes or edges from the enriched topology
`stanza.kind`	String	Stanza type identifier (e.g. `interface`, `bgp_neighbor`)
`stanza.fields`	Map<String, Value>	Untyped key–value pairs; may include Jinja2 template expressions

L.1.2 Stanza grouping

Stanzas are grouped by the `node` field in the stanza output to produce per-device DeviceIR. Each stanza is accumulated into the target node's `DeviceContext.stanzas` vector. No built-in deduplication occurs at the mapping phase; primitives are responsible for preventing duplicate stanzas.

L.1.3 Stanza types and conversion

Each `kind` string maps to a typed `StandardStanza` variant during TSDM processing:

Table 32. Stanza kind to `StandardStanza` mapping.

`kind`	Key Fields
`interface`	`name`, `abstract_index`, `address`, `mask`, `ipv6_address`, `description`, `enabled`, `vrf`, `mtu`, `vlan_tag`, `native_vlan`
`bgp_neighbor`	`peer_ip`, `remote_as`, `local_as`, `description`, `vrf`, `import_policy`, `export_policy`
`ospf_neighbor`	`interface`, `area`, `network`, `wildcard`, `network_type`, `vrf`
`isis_neighbor`	`interface`, `level`, `vrf`
`mpls_interface`	`interface`, `ldp_enabled`, `sr_enabled`
`segment_routing`	`router_id`, `global_block_range`, `node_sid`
`prefix_list`	`name`, `entries`
`community_list`	`name`, `entries`
`routing_policy`	`name`, `statements`
`access_policy`	`name`, `rules`

L.2 TSDM transform language

TSDM transforms are declared in the design plan's `transforms:` section or in imported files (e.g. `hardware-lowering.yaml`).

L.2.1 Grammar

Listing 23. TSDM transform structure.

```
transforms:
  - name: "nxos_9k_fixed_lowering"
    when: "device_os == 'nxos' && platform == 'n9k-fixed'"
    rules:
      - match_expr: "kind == 'interface' && name.startsWith('Ethernet')"
        apply:
          name: "name + '/1'"
          mtu: "9216"
```

Table 33. TSDM transform fields.

Field	Type	Description
`name`	String	Human-readable transform name
`when`	CEL	Optional device-level predicate; the entire transform is skipped if this evaluates to `false`
`rules[].match_expr`	CEL	Stanza-level predicate; only matching stanzas are rewritten
`rules[].apply`	Map<String, CEL>	Field name → CEL expression; each expression's result replaces the field value

L.2.2 `when` predicate context

The `when` predicate evaluates against the `DeviceContext`:

- `device_os` — e.g. `"eos"`, `"junos"`, `"iosxr"`, `"nxos"`, `"vyos"`
- `hostname` — device hostname
- `platform` — hardware platform identifier (e.g. `"dcs-7508"`, `"n9k-fixed"`)
- All keys from `DeviceContext.metadata`

L.2.3 `match_expr` context

The `match_expr` evaluates against each stanza's fields:

- `kind` — stanza type (e.g. `"interface"`, `"bgp_neighbor"`)
- `key` — optional stanza identifier (e.g. policy name)
- All fields from the stanza's `fields` map, converted from JSON to CEL values via `json_to_cel()`

L.2.4 `apply` expression semantics

Each `apply` entry is a CEL expression compiled and executed against the same context as `match_expr`. The result is converted back to JSON via `cel_to_json()` and inserted into the stanza's `fields` map.

After each field is applied, the CEL context is updated with the new value, enabling sequential field dependencies within a single rule:

Listing 24. Sequential field dependencies in apply.

```
apply:
  # First compute the slot from abstract_index
  slot: "string(int(abstract_index) / 48 + 1)"
  # Then use slot in the port name (slot is now updated)
  name: "'Ethernet' + slot + '/' + string(int(abstract_index) % 48 + 1)"
```

L.2.5 Type conversions

CEL values are converted to JSON as follows: `Bool` → JSON boolean, `Int`/`UInt`/`Float` → JSON number, `String` → JSON string, `List` → JSON array (recursive), `Map` → JSON object (recursive), all others → JSON null.

L.3 Data model relationships

The DeviceIR pipeline involves four data structures:

1. **`DeviceContext`** — per-node mutable property bag, progressively enriched by primitives during Stage 1. Contains IPAM fields, protocol metadata, typed policy models (`security`, `qos`), and an untyped `stanzas` vector.
2. **`Stanza`** — untyped DeviceIR unit with three fields: `kind` (String), `key` (optional String), and `fields` (Map<String, JSON>). Produced by the mapping evaluator.
3. **`StandardStanza`** — typed enum with one variant per stanza kind (e.g. `Interface`, `BgpNeighbor`). Produced by `Stanza::to_standard()` during TSDM processing.
4. **`TargetDeviceModel`** — the final per-device model passed to the render engine. Contains `hostname`, `device_os`, `platform`, typed `stanzas` (Vec<StandardStanza>), optional `security` and `qos` models, and `target_metadata`.

M Case Study Validation

This appendix validates the NETCFG pipeline against four case studies from Knight [43], Chapter 8. Each case study was recreated as a topogen base topology and a layered blueprint, exercising the primitives documented in this report. The results expose both working coverage and genuine gaps, summarised in Table 34.

Table 34. Primitive coverage across thesis case studies. ✓ = works, ~ = workaround, ✗ = gap.

Feature / Primitive	CS1	CS3	CS2	CS4
`mesh_nodes` (full)	✓	✓	✓	✓
`mesh_nodes` (hub_and_spoke)				✓
`provision_ips`	✓	✓	✓	✓
`build_protocol_layer` OSPF	✓	✓	✓	✓
`build_protocol_layer` BGP	✓		✓	✓
`build_protocol_layer` RIP			✗	
`tag_nodes`		✓		
`policy_routing`			✓	
`policy_prefix_list`			✓	
`generate_safe_bgp_filters`			✓	
Route redistribution			✗	
VLAN switching primitive		✗		
GraphML import				~
Edge property import	✓	✓	✓	✓

M.1 CS1: Simplified Small Internet

Topology. 14 routers across 7 autonomous systems (Table 35), connected by 17 edges (9 intra-AS, 8 inter-AS peering). Sourced from thesis Table 8.1 and the `small_internet.graphml` dataset.

Table 35. CS1 node inventory.

AS	Role	Nodes	Routers
AS1	Backbone	1	as1r1
AS20	Provider	3	as20r1, as20r2, as20r3
AS30	Provider	1	as30r1
AS40	Provider	1	as40r1
AS100	Customer	3	as100r1, as100r2, as100r3
AS200	Customer	1	as200r1
AS300	Customer	4	as300r1–r4

Blueprint layers.

1. **pools** — `global_pool` for infrastructure (`192.168.0.0/16`) and loopbacks (`10.0.0.0/16`).
2. **topology** — `mesh_nodes` (full) per multi-router AS (AS20, AS100, AS300).
3. **addressing** — `provision_ips` on all edges, /30 subnets, dense allocation.
4. **ospf** — `build_protocol_layer` (OSPF, area 0) per multi-router AS.
5. **ibgp** — `build_protocol_layer` (BGP) per multi-router AS for iBGP full-mesh.
6. **ebgp** — `build_protocol_layer` (BGP) on border routers for inter-AS peering.

Assertions. OSPF connectivity per AS, end-to-end reachability (AS300→AS100), `field_exists` for `src_ip` on all edges, minimum node count (14).

Verdict. CS1 exercises the core pipeline cleanly: topogen import → mesh → IPAM → OSPF → BGP. The automated `derive_stanzas` pass successfully populates the Cisco IOS address format (`ip`

`address A.B.C.D M.M.M.M`) and OSPF `network` and `wildcard` fields from the underlying topology. All primitives have full coverage.

M.2 CS3: VLAN Campus Network

Topology. 14 nodes in a single AS: 9 routers and 5 switches connected by 15 edges. Three VLANs (1, 2, 3) assigned to router–switch access links; three switch–switch trunk links carry all VLANs.

Blueprint layers.

1. **pools** — `global_pool` for campus addressing (`192.168.0.0/16`).
2. **topology** — `mesh_nodes` (full) for routers only.
3. **addressing** — `provision_ips` on all edges; switches receive subnet membership but no routed addresses.
4. **ospf** — `build_protocol_layer` (OSPF, area 0) on routers only, selected via `nodes[device_type == 'router']`.
5. **vlan-tags** — `tag_nodes` stamps VLAN metadata onto switch nodes.

Gap: L2 switching. The thesis configures switch ports with access/trunk modes and VLAN trunking protocol (VTP). NETCFG has no `build_vlan` primitive. The `StandardStanza::Interface` type includes a `vlan_tag` field, but no primitive populates it. VLAN IDs are recorded as metadata via `tag_nodes` but do not produce switch stanzas.

Verdict. The pipeline successfully compiles routing layers, and v7.5 corrections automated the interface addressing for router-to-router links. The L2 switching gap remains cleanly isolated: adding a `build_vlan` primitive and a `StandardStanza::SwitchPort` variant would close it.

M.3 CS2: Complete Small Internet

Topology. Same 14-node topology as CS1, but with BGP role hierarchy metadata (backbone/provider/customer) and richer policy configuration.

Blueprint layers.

1. **pools** — Infrastructure and loopback pools.
2. **topology** — Intra-AS mesh per multi-router AS.
3. **addressing** — `provision_ips`, /30 subnets.
4. **igp** — OSPF per multi-router AS (thesis uses RIP for some ASes; see gap below).
5. **bgp** — iBGP full-mesh per AS + eBGP on border routers.
6. **policy** — Five policy primitives:
   - `policy_routing` "dontUseMe" (AS-path prepend ×3)
   - `policy_routing` "localPrefIn" (local-pref 90)
   - `policy_prefix_list` per customer AS (AS100-IN, AS200-IN, AS300-IN)
   - `generate_safe_bgp_filters` (bogon filter)

Gap: RIP. The thesis uses RIP as the IGP for customer ASes. `build_protocol_layer` accepts `layer: "rip"` (free-form string), which stores the intent in the overlay graph, but no stanza renderer exists for RIP. The blueprint substitutes OSPF.

Gap: Route redistribution. The thesis redistributes connected routes from IGP into BGP for customer ASes. No redistribution primitive exists in NETCFG.

Verdict. The BGP policy primitives (`policy_routing`, `policy_prefix_list`, `generate_safe_bgp_filters`) have full coverage and are exercised thoroughly. Recent v7.5 corrections closed the interface addressing gap, enabling CS2 routers to produce complete

interface stanzas alongside protocol configuration. RIP rendering and route redistribution are the two remaining gaps.

M.4 CS4: Large-Scale NREN

Topology. The thesis uses a 1,154-node NREN topology across 40 autonomous systems, sourced from `interconnect.graphml`. A Python conversion script (`convert-graphml.py`) parses the GraphML, computes betweenness centrality per AS to select route reflectors ($\min(N/3, 3)$ per AS), and outputs topogen YAML. The checked-in placeholder uses the 14-node `small_internet.graphml` to validate the pipeline structure.

Blueprint layers.

1. **pools** — Infrastructure (`10.0.0.0/8`) and loopback (`172.16.0.0/16`) pools.
2. **topology** — `mesh_nodes` (full) per AS for intra-AS connectivity.
3. **addressing** — `provision_ips`, /30 subnets.
4. **ospf** — `build_protocol_layer` (OSPF, area 0) per AS.
5. **ibgp** — `mesh_nodes` (hub_and_spoke) with centrality-selected route reflectors as hubs, then `build_protocol_layer` (BGP) per AS.
6. **ebgp** — `build_protocol_layer` (BGP) for inter-AS peering.

Gap: GraphML import. No native GraphML import primitive exists. The offline `convert-graphml.py` script bridges this gap, computing RR roles as part of the conversion. A native `import_graphml` primitive would eliminate this step.

Verdict. The pipeline structure is sound for large-scale topologies. `mesh_nodes` hub_and_spoke correctly models the route-reflector hierarchy, and automated IPAM now correctly provisions hierarchical iBGP endpoints. Performance benchmarking requires the full 1,154-node dataset.

References

[1] S. Shenker, "The future of networking, and the past of protocols," in *Open Networking Summit*, vol. 20, Palo Alto, CA: Open Networking Foundation, 2011, pp. 1–30.

[2] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: Measurement, analysis, and implications," in *ACM SIGCOMM computer communication review*, vol. 41, New York, NY, USA: ACM, 2011, pp. 350–361.

[3] J. Zhu et al., "MALT: Multi-abstraction layer topology," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, Santa Clara, CA: USENIX Association, 2020, pp. 313–328.

[4] OpenConfig Working Group, *OpenConfig*, https://openconfig.net, 2023.

[5] NetBox Community, *NetBox: The source of truth for network automation*, https://netbox.dev/, 2023.

[6] Network to Code, *Nautobot: Network source of truth and automation platform*, https://nautobot.com/, 2023.

[7] Red Hat, *Ansible automation platform*, https://www.ansible.com/, 2023.

[8] Nornir Automation, *Nornir: Pluggable multi-threaded framework with inventory management to help operate collections of devices*, https://nornir.tech/, 2023.

[9] Intentionet, *Batfish: Network configuration analysis tool*, https://www.batfish.org/, 2023.

[10] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, IEEE, San Jose, CA: IEEE, 2004, pp. 75–86.

[11] S. Knight, "NTE-TR-001: Graph primitive specification for the network topology engine," AutoNetKit, Technical Report, 2026.

[12] S. Knight, *AutoNetKit NTE: The network topology engine*, 2026.

[13] J. Moy, "OSPF Version 2," IETF, RFC 2328, Apr. 1998.

[14] D. Oran, "OSI IS-IS intra-domain routing protocol," IETF, RFC 1142, Feb. 1990.

[15] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," IETF, RFC 4271, Jan. 2006.

[16] E. Rosen, A. Viswanathan, and R. Callon, "Multiprotocol Label Switching Architecture," IETF, RFC 3031, Jan. 2001.

[17] M. Mahalingam et al., "Virtual eXtensible Local Area Network (VXLAN): A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks," IETF, RFC 7348, Aug. 2014.

[18] A. Sajassi et al., "BGP MPLS-Based Ethernet VPN," IETF, RFC 7432, Feb. 2015.

[19] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, and D. Vrgoč, "Foundations of modern query languages for graph databases," *ACM Computing Surveys*, vol. 50, no. 5, pp. 1–40, 2017.

[20] N. Francis et al., "Cypher: An evolving query language for property graphs," in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 1433–1445.

[21] ISO/IEC, *ISO/IEC 39075:2024 — information technology — database languages — GQL*, International Standard, 2024.

[22] Bytecode Alliance, *Wasmtime: A fast and secure runtime for WebAssembly*, https://wasmtime.dev/, 2024.

[23] C. J. Anderson et al., "NetKAT: Semantic foundations for networks," in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2014, pp. 113–126.

[24] C. Monsanto, J. Reich, N. Foster, J. Rexford, and D. Walker, "Composing software defined networks," in *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013, pp. 1–13.

[25] C. Prakash et al., "PGA: Using graphs to express and automatically reconcile network policies," in *Proceedings of the 2015 ACM SIGCOMM Conference*, 2015, pp. 29–42.

[26] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Propane: A general approach to network configuration synthesis," in *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 440–453.

[27] A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "NetComplete: Practical network-wide configuration synthesis with autocompletion," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, USENIX Association, 2018, pp. 579–594.

[28] P. Gao et al., "Jinjing: Safe and efficient update of ACL configurations," in *Proceedings of the ACM SIGCOMM 2020 Conference*, ACM, 2020, pp. 87–100.

[29] A. Fogel et al., "A general approach to network configuration analysis," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015, pp. 469–483.

[30] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proceedings of the 2017 Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 155–168.

[31] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, pp. 113–126.

[32] R. Soulé et al., "Merlin: A language for provisioning network resources," in *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2014, pp. 271–284.

[33] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[34] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[35] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, "Network configuration synthesis with abstract topologies," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2017, pp. 437–451.

[36] A. El-Hassany et al., "Network configuration synthesis with logic-based routing models," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2017.

[37] NAPALM Automation, *NAPALM: Network automation and programmability abstraction layer with multivendor support*, https://napalm.readthedocs.io/, 2024.

[38] HashiCorp, *Terraform: Infrastructure as code*, https://www.terraform.io/, 2024.

[39] Pulumi Corporation, *Pulumi: Infrastructure as code in any programming language*, https://www.pulumi.com/, 2024.

[40] M. Bjorklund, "The YANG 1.1 data modeling language," IETF, RFC 7950, 2016.

[41] P. Sun, R. Mahajan, J. Rexford, L. Yuan, M. Zhang, and A. Arefin, "Statesman: Understanding and reducing the complexity of network state management," in *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2014, pp. 85–98.

[42] R. Beckett and R. Mahajan, "A general framework for compositional network modeling," in *Proceedings of the 19th ACM Workshop on Hot Topics in Networks (HotNets)*, 2020, pp. 8–15.

[43] S. Knight, "Topology-aware network device configuration synthesis," Ph.D. dissertation, University of Adelaide, 2015.